

Агенты и вайб-кодинг

Практический курс: как работать с ИИ-агентами
и не обманывать себя результатом

Собрано по материалам полугода профессиональной переписки практиков.
42 главы, десять модулей, приложения с чек-листами и шаблонами.

Содержание

Введение	4
Модуль 1. Устройство	7
Глава 1. Что такое агент	8
Глава 2. Анатомия агента	11
Глава 3. Харнесс: слово без одного значения	15
Глава 4. Как устроена модель: минимум механики	19
Глава 5. Маршрут новичка	23
Модуль 2. Постановка задачи	26
Глава 6. Спецификация вместо задания	27
Глава 7. Требования, понятные агенту	31
Глава 8. Диалог, который превращается в план	35
Глава 9. Выбор модели, уровень усилий и оценка сроков	39
Модуль 3. Обязка	43
Глава 10. Файл правил	44
Глава 11. Правила, которые работают	48
Глава 12. Скиллы: механика и жизненный цикл	53
Глава 13. Хуки и детерминированные гарантии	58
Глава 14. Инструменты и когда обязка вредит	62
Модуль 4. Контекст и память	66
Глава 15. Контекст как единственный ресурс	67
Глава 16. Сжатие и что должно его пережить	71
Глава 17. Память: файлы, векторы, нити	75
Глава 18. Поиск по коду и по документам	79
Модуль 5. Делегирование	83
Глава 19. Субагент: когда помогает, когда мешает	84
Глава 20. Многоагентность и рои	88
Глава 21. Роли и конвейер	92
Модуль 6. Проверка	96

Глава 22. Ревью	97
Глава 23. Эвалы	101
Глава 24. Детерминированные проверки вместо уговоров	105
Глава 25. Сколько полировать	109
Модуль 7. Автономность	113
Глава 26. Ночной прогон	114
Глава 27. Дрейф намерения	118
Глава 28. Автономный режим: что он реально делает	122
Глава 29. Постоянно работающий агент	126
Модуль 8. Люди и компании	130
Глава 30. AI-first процессы	131
Глава 31. Внедрение и неявное знание	135
Глава 32. Владение скиллами в компании	139
Глава 33. Роли: что теперь делает разработчик	143
Модуль 9. Безопасность и риски	147
Глава 34. Разрушительные действия и заборы	148
Глава 35. Изоляция, права, песочница	152
Глава 36. Почему правила не удерживают агента	156
Глава 37. Приватность и утечки	160
Модуль 10. Экономика и практика	164
Глава 38. Токеномика	165
Глава 39. Подписки, лимиты и горизонт	169
Глава 40. Личный агент	172
Глава 41. Агент в групповом чате	176
Глава 42. Документы, дизайн, интерфейсы	180
Заключение. Что устоялось, а что нет	184
Приложение А. Словарь терминов	187
Приложение Б. Чек-листы	189
Приложение В. Шаблоны запросов	192

Введение

Эта книга собрана из полугода переписки людей, которые каждый день ебошат с ИИ-агентами. Не из документации поставщиков, не из докладов на конференциях и не из постов, написанных ради охвата. Из рабочего разговора, где человек приходит с ошибкой, получает три разных совета, пробует, возвращается и честно рассказывает, что сработало, а что наебнулось.

У такого источника есть минус: причёсанных методологий здесь нет и не будет ни хуя. И есть плюс, ради которого всё затевалось: здесь видна цена. Сколько стоит ошибка, сколько времени уходит на проверку, где красивая схема разваливается на второй неделе и с хуя ли совет, который звучит разумно, у половины людей не работает.

Про язык

Книга написана так, как об этом говорят между собой. С матом, потому что без него получается вежливая вода, в которой тонет вся конкретика.

Технические термины, цифры, команды и названия остаются как есть, байт в байт. Шаблоны запросов и куски кода чистые: их копируют и вставляют в работу, а туда мат не течёт.

Для кого

Для человека, который начинает с нуля.

Программировать уметь не надо: первые три модуля читаются и без этого, и после них агентом можно пользоваться осмысленно. Дальше пригодится хотя бы поверхностное понимание, как устроена разработка, но каждое новое понятие вводится отдельно и разжёвывается на пальцах.

Уже работаешь с агентами, получишь другое: сведённые вместе споры, которые ты наверняка вёл сам, с их разрешением или с честным признанием, что разрешения нет.

Что здесь есть и чего нет

Есть: приёмы, обкатанные на настоящих проектах, с указанием, где они наебнутся. Разобранные инциденты. Измерения, которые люди делали для себя и выкладывали. Несколько десятков готовых формулировок, которые можно вставить в свои правила и запросы прямо сегодня.

Нет: сравнений конкретных моделей и цен. Такое протухает за месяц, и книга, набитая этой поебенью, превращается в архив. Названия инструментов встречаются только там, где без них не объяснить суть, и почти всегда описаны как класс, а не как бренд.

ТРИ СЛОВА, КОТОРЫЕ НУЖНЫ С ПЕРВОЙ СТРАНИЦЫ

Модель это то, что генерирует текст. Между вызовами она не помнит ни хуя и руками делать ничего не умеет.

Агент это модель, которую посадили в цикл и дали инструменты. Он может прочитать файл, запустить команду, посмотреть результат и решить, что делать дальше.

Обязка, она же харнесс, это всё, что вокруг: инструкции, набор инструментов, память, проверки. Именно она превращает модель в работника, и именно про неё большая часть этой книги.

Почему область выглядит такой запутанной

Потому что молодая. Термины появляются быстрее, чем успевают получить общепринятое значение. Одно и то же слово у трёх собеседников означает три разные вещи, и они часами пиздят друг с другом, не заметив этого.

Книга не делает вид, что консенсус есть там, где его нет. Где мнение общее, так и написано. Где идёт спор, приведены обе стороны с их аргументами, и сказано, от чего зависит выбор. Таких мест до хуя, и это не слабость материала, а его состояние.

Как устроена книга

Десять модулей. Внутри каждого главы по три-пять страниц. Модули идут по нарастанию: сначала устройство, потом постановка задачи, потом обязка, дальше темы посложнее. Но главы самостоятельны, и если тебе прямо сейчас нужен ответ про память или про то, сколько полировать результат, иди сразу туда.

Модуль	О чём	Кому нужен
1. Устройство	Что такое агент и из чего состоит	Всем, с этого начинается счёт
2. Постановка задачи	Как объяснить, чтобы получи-лось	Всем
3. Обязка	Правила, скиллы, хуки, инстру-менты	Всем
4. Контекст и память	Единственный ресурс и как им платят	Всем
5. Делегирование	Субагенты, рои, роли	Когда задача не влезает в разго-вор
6. Проверка	Ревью, эвалы, детерминирован-ные тесты	Когда результат идёт наружу
7. Автономность	Долгие прогоны и дрейф	Когда хочется отойти от компью-тера
8. Люди и компании	Внедрение, роли, владение	Кто отвечает не только за себя
9. Безопасность	Инциденты, изоляция, приват-ность	Перед первым доступом в прод

Модуль	О чём	Кому нужен
10. Экономика и практика	Счета и первые проекты	В любой момент

Первые четыре модуля стоит прочитать подряд: дальше всё на них опирается. Модули с пятого по седьмой нужны, когда задачи перестали влезать в один разговор. Восьмой и девятый пригодятся, если отвечаешь не только за себя. Десятый практический, его можно читать в любой момент, когда захочется что-нибудь захуячить.

Пять типов врезок

Текст размечен так, чтобы читать можно было по-разному.

Врезка	Что внутри	Как читать
Главное	Сформулированный вывод раз-дела	Торопись, читай только их
Для новичка	Разбор базового понятия	Пропускай, если понятие знако-мо
Случай из практики	Разобранный инцидент	Здесь видна цена ошибки
Разные взгляды	Спор без победителя	Выбирать придётся тебе
Что делать	Готовый чек-лист или шаблон	Можно применять сразу

Одна просьба

Не воспринимай написанное как свод законов, высеченный в камне. Почти каждый приём в этой книге кто-то опроверг на своём проекте, и там, где это известно, так и сказано. Область меняется быстрее, чем печатается тираж.

Правильное отношение такое: здесь собрано то, что работало у многих на настоящих задачах. Проверь на своей. Единственное, что действительно универсально, это привычка измерять, а не верить на слово.

Модуль 1. Устройство

Пять глав про то, из чего вся эта хуёвина собрана.

Здесь вводятся понятия, на которые опирается остальная книга: что называют агентом и с хуя ли об этом до сих пор спорят, из каких слоёв он сложен, что за слово такое харнесс, как модель работает внутри и с чего начинать человеку, у которого пока там ещё хуй не валялся.

Читаешь подряд, значит модуль обязательный. Дальше без него будет хуй просышь.

Что такое агент

Спроси пятерых практиков, что такое агент, и получишь пять определений. Причём каждый будет свято уверен, что уж его-то определение общепринятое, а остальные путаются в терминах, как слепые котята.

Это не повод махнуть рукой. Определение нужно, иначе на простейшие рабочие вопросы ответить хуй: считать ли агентом то, что ты собрал, платить ли за инструмент, который сам себя обозвал агентской платформой, и с хуя ли совет из чужой статьи у тебя не работает.

Разберём спор до конца, а потом дадим порог, которым можно пользоваться.

Четыре определения и четыре контрпримера

ЧТО ВООБЩЕ ПРОИСХОДИТ ВНУТРИ

Модель принимает текст и выдаёт текст. Всё. Никаких программ она не запускает, файлов не читает, вчерашний разговор не помнит ни хуя.

Чтобы она что-то сделала, кто-то должен: собрать текст запроса, отправить, разобрать ответ, увидеть там просьбу вроде прочитай файл, выполнить эту просьбу своими руками и отправить результат обратно новым запросом. Этот кто-то и есть программа-обвязка. Повторяющиеся круги такого обмена называют циклом.

Первое определение самое ходовое: агент это модель плюс инструкции плюс инструменты плюс контекст, работающие в цикле.

Контрпример: процессный агент, собранный из жёстко заданных шагов, цикла не имеет вообще. Ход исполнения задан заранее, модель дёргают на каждом шаге по одному разу. Но все вокруг называют эту хуёвину агентом, и спорить с этим бесполезно.

Второе определение: агент это модель плюс память плюс планирование плюс использование инструментов.

Контрпример: исследовательский режим поисковика. Там и планирование есть, и инструменты, и обвязка, собирающая контекст. А результат один: сообщение в чат, и всё только на чтение. Многие откажутся называть это агентом, потому что он ни хуя не делает.

Третье определение шире: агент это вся система, которая реально выполняет за тебя работу, и похуй, как физически разнесены её части.

Контрпример: короткий ответ поисковика, собранный моделью из трёх найденных страниц. Работу за тебя он тоже выполнил. Значит, и он агент? Тогда слово вообще перестаёт что-либо означать.

Четвёртое, самое требовательное: агент это когда обязательно есть цикл и свобода выбора инструментов, плюс какое-то управление контекстом.

Контрпример: тот же процессный агент. И тот же исследовательский режим. Оба выпадают нахуй.

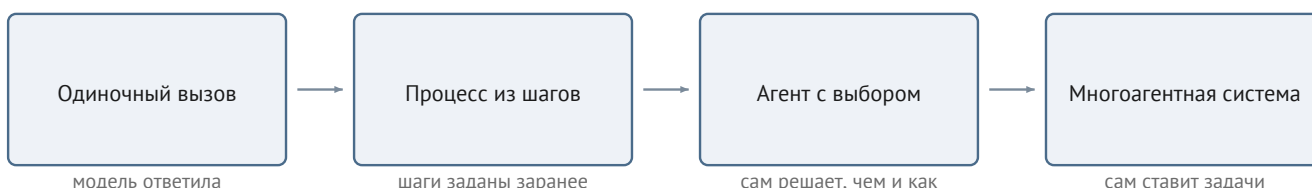
ГЛАВНОЕ ПРО ОПРЕДЕЛЕНИЕ

Спор не разрешается, потому что агентность это не свойство, а спектр. На одном конце одиночный вызов модели, на другом система, которая сама берёт задачи и сама их закрывает. Между ними десятки промежуточных форм, и каждый ставит порог там, где ему удобно. Требовать от собеседника твоего порога бессмысленно. Спрашивать, где стоит его, полезно.

Спектр вместо категории

Хорошая аналогия прозвучала в разговоре: солёная ли еда. Курица в меду солёная? А хлеб? А чиабатта? Соль есть везде, вопрос только в том, где ты проводишь черту.

СПЕКТР АГЕНТНОСТИ



Слева направо растут три вещи разом: свобода агента, непредсказуемость результата и цена ошибки. Это единственная закономерность, которая работает независимо от определения, и опираться при выборе стоит именно на неё.

Рабочий порог

Нужно одно предложение, вот оно.

Агент это посредник, который сам собирает контекст и сам выбирает инструменты.

Слово посредник тут ключевое. В обычном языке агент это тот, кто действует по твоему поручению: агент по недвижимости ищет квартиру, актёрский агент договаривается о работе. Гениальности от него никто не требует, требуют, чтобы он понял задачу и довёл дело до конца, обращаясь по пути к нужным людям и инструментам.

Отсюда практический вывод, который переворачивает половину разговоров. Вынеси модель за скобки, и качество агента определяется качеством его инструментов и тем, насколько эти инструменты понятны модели. Не размером модели. Не количеством инструкций, которых ты туда невъебенно нахуевертил. Инструментами и их описанием.

ТРИ ВЗГЛЯДА НА ГРАНИЦУ

Строгий. Нет цикла и свободы выбора, значит не агент, а процесс. Удобен, когда надо оценить риск: чем больше свободы, тем строже проверка.

Функциональный. Делает работу за человека, значит агент. Удобен в разговоре с бизнесом, где важен результат, а не устройство.

Инженерный. Агент это система с состоянием, идентичностью, памятью и границами. Удобен при проектировании, потому что сразу называет, что придётся строить.

Зачем это знать новичку

Затем, что от выбранного порога зависят три решения.

Сколько свободы дать. Чем правее по спектру, тем меньше ты можешь предсказать. Задачу с высокой ценой ошибки лучше зажать в процесс с заданными шагами, чем отдать агенту, который сам решит, как её сделать, и ты будешь потом хуй проссышь разгребать.

Что проверять. Процесс проверяется по шагам, агент только по результату и по траектории. Это разные затраты и разные инструменты, о чём отдельный модуль.

За что платить. Половина продуктов, которые называют себя агентскими платформами, продают тебе процесс. Это не наебало: процесс часто именно то, что нужно. Но платить за него как за агента не стоит.

КАК РАЗОБРАТЬСЯ В ЧУЖОМ ПРОДУКТЕ ЗА ПЯТЬ МИНУТ

1. Спроси, есть ли цикл. Модель вызывается один раз за шаг или столько, сколько сама решит?
2. Спроси про инструменты. Набор фиксирован для шага или агент выбирает из общего списка?
3. Спроси, кто собирает контекст. Шаблон подставляет заранее известные поля или система решает, что положить?
4. Спроси, что происходит при ошибке. Повтор, откат, остановка или продолжение как ни в чём не бывало?
5. Три ответа из четырёх в пользу свободы означают агента. Иначе перед тобой процесс, и это нормально.

Что запомнить

Единого определения нет и в ближайшее время не будет, хоть ты тресни. Есть спектр, на котором растёт свобода, а вместе с ней непредсказуемость и цена ошибки.

Рабочий порог: сам собирает контекст, сам выбирает инструменты.

И главное следствие, к которому мы будем возвращаться всю книгу: агент настолько хорош, насколько хороши его инструменты и описания этих инструментов. Всё остальное надстройка.

Анатомия агента

Чтобы дальше говорить предметно, агента надо расхуячить на части. Иначе разговор всё время сваливается в общие слова: он тупит, он не слушается, он выдумывает. Тупит не он целиком, а какой-то один слой, и почти всегда можно сказать, какой именно.

Шесть слоёв

ЧТО ПРОИСХОДИТ ЗА ОДИН ХОД

Ты написал фразу. Дальше происходит вот что.

Программа собирает большой текст: правила проекта, описания доступных инструментов, историю разговора, твою фразу. Отправляет это модели. Модель возвращает текст, в котором может быть просьба вызвать инструмент. Программа выполняет просьбу, кладёт результат в тот же большой текст и отправляет заново.

Так повторяется, пока модель не скажет, что закончила. Каждый круг стоит денег, потому что весь накопленный текст уезжает целиком.

Первое, что вводит новичка в ступор: модель не помнит ни хуя. Каждый круг ей заново пересказывают всё, что было. Именно поэтому контекст оказывается главной валютой, и целый модуль этой книги про то, как им распоряжаться.

Слой	За что отвечает	Кто им управляет	Как ломается
Модель	Догадка, рассуждение, текст	Поставщик	Выдумывает, срезает углы
Цикл	Повторение ходов, остановка	Обязка	Зацикливается или бросает
Инструменты	Действия во внешнем мире	Ты	Плохое описание, нет ограничений
Контекст	Что модель видит сейчас	Обязка и ты	Переполнение, мусор, противоречия
Память	Что переживает разговор	Ты	Свалка фактов, устаревшее
Проверка	Признание работы годной	Ты	Её просто нет

Шестой слой стоит особняком. Он не внутри агента, он снаружи. И это единственный слой, который целиком твой: поставщик его не сделает, фреймворк не подскажет.

ОДИН ХОД АГЕНТА



Модель: догадка и больше ни хуя

Модель хороша ровно в одном: угадывать, что уместно дальше. Это невъебенная сила, когда задача требует понимания, и полная беспомощность, когда задача требует точности.

Отсюда правило, которое стоит запомнить до всякой практики.

ГЛАВНОЕ ПРО РАЗДЕЛЕНИЕ ТРУДА

Модели отдавай то, где нужна догадка. Всё, что требует надёжности, пиши кодом.

Формат ответа, описанный словами в запросе, нарушается примерно в одном случае из десяти. Это не лечится формулировкой, не лечится строгим тоном и не лечится большой моделью. Контракт обязан обеспечивать код, который вызывает модель, а не абзац внутри запроса.

Пример, который разбирают постоянно. Есть инструмент распознавания блюд на подносе. Модель просят вернуть ответ в определённом формате, и формат описан прямо в тексте запроса. В девяти случаях из десяти всё заебись, в десятом формат нарушен, и всё разваливается к ёбаной бабушке. Правильное решение не в том, чтобы написать формат покрупнее, а в том, чтобы обвязка сама принуждала к структуре ответа.

Цикл: что заставляет всё работать

Цикл это простая петля: спросили, получили просьбу, выполнили, спросили снова. Вся сложность в двух вопросах, которые вслух задают редко.

Когда останавливаться? Нет критерия, значит агент либо закончит слишком рано и объявит готовым недоделанное, либо будет коноёбиться, пока не кончатся деньги.

Что делать при ошибке? Повторить, откатиться, спросить человека или пойти дальше как ни в чём не бывало. Четвёртый вариант выбирается по умолчанию, если ты не выбрал иначе, и именно он приносит больше всего сюрпризов.

МИНИМУМ, КОТОРЫЙ ДОЛЖЕН БЫТЬ В ЛЮБОМ ЦИКЛЕ

1. Явный признак завершения. Не молчание модели, а проверяемое условие.
2. Ограничение на число ходов. Иначе одна неудачная задача сожрёт дневной бюджет.
3. Правило при ошибке инструмента: повтор с ограничением, откат, остановка.
4. Запрет объявлять готовым то, что не прошло проверку.

Последний пункт стоит выделить отдельно. Не может агент доказать, что задача выполнена, значит не имеет права говорить, что она выполнена. Это правило вылезает в каждом

серьёзном разборе, потому что нарушают его постоянно и с удовольствием.

Инструменты: интерфейс важнее уговоров

Инструмент это не функция, а контракт. У него есть схема входа, предусловия, побочные эффекты, таймаут, поведение при повторном вызове и права.

Слово идемпотентность звучит страшно, а значит простую вещь: повторный вызов не должен ломать мир. Отправить письмо дважды плохо. Пересчитать сумму дважды нормально. Инструмент не идемпотентен, значит повтор при ошибке становится опасным, и знать это надо заранее, а не выяснять на проде, когда уже понеслась пизда по кочкам.

Второе важное свойство: описание инструмента это часть его работы. Модель выбирает инструмент по описанию, и хуёвое описание означает, что инструмент не вызовет вовремя или вызовет не туда. Вернёмся к этому в главе про скиллы, там та же закономерность видна ещё резче.

СПОРНАЯ ГРАНИЦА: ИНСТРУМЕНТЫ ИЛИ ПОДКЛЮЧАЕМЫЕ СЕРВЕРЫ

За отдельные утилиты. Их проще тестировать, они не занимают место в контексте постоянно и работают в любом окружении. Практики, которые упаковывали внешние интерфейсы в утилиты, отмечали, что фон выдумок пропал совсем.

За протокольные подключения. Единый способ подключить чужой сервис, не описывая его руками. Удобно, когда сервисов до хуя и они чужие.

Что решает на практике. Скорость появления: утилиту пишут за десять минут, подключение согласовывают неделями. Поэтому набор утилит растёт быстрее, и большинство команд живут на нём.

Контекст: то, за что ты платишь

Контекст это весь текст, который уходит модели на каждом круге. Правила проекта, описания инструментов, история, результаты вызовов, твоя фраза.

Два свойства делают его главной темой книги.

Он платный. Каждый круг оплачивается целиком, а не по приросту. Длинный разговор порождает нелинейно.

Он ограничен. И даже внутри лимита работает неравномерно: начало и конец модель использует лучше, чем середину. Чем больше ты туда напихал, тем меньше внимания достаётся каждой части.

Память: то, что переживает разговор

Память это всё, что должно сохраниться, когда разговор кончится. Бывает трёх видов, и все три нужны в разных случаях: файлы, векторный поиск и нити с историей решений. Целая глава разбирает, какой вид что решает и какими болезнями болеет.

Пока хватит одного различия. Контекст это то, что модель видит сейчас. Память это то, откуда в контекст попадает нужное. Путать их дорого: люди сваливают в правила проекта то,

чему место в памяти, а потом хватаются за голову от счёта.

Проверка: слой, которого обычно нет

Разбирая чужие сборки, практики чаще всего находят одно и то же: модель выбрана, инструменты написаны, правила расписаны на трёх страницах, а как понять, что стало лучше, не знает ни одна живая душа.

ГЛАВНОЕ ПРО АНАТОМИЮ

Пять слоёв внутри агента делают работу. Шестой, снаружи, отвечает на вопрос, годится ли она.

Первые пять можно взять готовыми: модель у поставщика, цикл у фреймворка, инструменты у сообщества. Шестой придётся строить самому, потому что он зависит от твоей задачи. И именно он отличает работающего агента от красивой демки.

Кто чем управляет

Полезно понимать, где проходит граница твоей ответственности.

Поставщик даёт модель и её поведение. Ты на это не влияешь, кроме выбора модели и уровня усилий, и должен быть готов к тому, что поведение поменяется без предупреждения.

Фреймворк даёт цикл, вызовы инструментов, базовое сжатие. Это можно взять готовым, и почти всегда стоит.

Твоя зона: описания инструментов, правила, память, проверки. Здесь готового нет ни хуя, и здесь решается, будет ли всё это работать.

Что запомнить

Агент это шесть слоёв, а не монолит. Когда что-то пошло по пизде, определи слой: выдумки это модель, бесконечное кручение это цикл, не тот инструмент это описание, забыл сказанное это контекст или память, объявил готовым недоделанное это отсутствие проверки.

Модели отдавай догадку, всё остальное коду. Формат, описанный словами, нарушается в одном случае из десяти, и хуй ты это уговорами вылечишь.

И последнее, к чему мы будем возвращаться: агент это не веса и не обвязка, а модель вместе с инструкциями, памятью и историей. Убери любой слой, и перед тобой другой агент.

Харнесс: слово без одного значения

Есть слово, которое за полгода стало главным в профессиональном разговоре и при этом так и не получило общепринятого значения. Харнесс. Дословно упряжь.

Разбираться с ним нужно не из любви к терминологии. Половина непоняток между людьми, которые вроде бы говорят об одном, происходит ровно здесь. Один имеет в виду автономность, другой набор инструментов, третий систему целиком, и они час толкут воду в ступе, не заметив расхождения.

Три определения, которые ходят рядом

Автономность. Харнесс не про быструю и качественную разработку кода, а про то, чтобы агент работал сам, с долгим временем цикла, особенно на отладке. При таком взгляде харнесс это возможность отойти от компьютера и не бояться.

Полная автономка. Версия порадикальнее: агент сам забрал задачу, сам разбил, сам сделал, сам закрыл, взял следующую. Тут харнесс уже почти синоним самостоятельной системы.

Упряжь. Не автономность, а оснастка: инструменты, скиллы, правила, проверки. Модель это лошадь, упряжь это то, что на неё надето, агент это тот, кто едет. Чем лучше лошадь и упряжь, тем дальше и бодрее едешь.

КАКОЕ ОПРЕДЕЛЕНИЕ ВЫБРАТЬ

Оцениваешь риск, бери определение через автономность. Оно сразу говорит, сколько свободы ты отдаёшь.

Строишь, бери определение через упряжь. Оно называет части, которые придётся захуячить руками.

Объясняешь бизнесу, бери определение через результат. Остальное только запутает.

Главное не спорить о слове, а спросить собеседника, что он имеет в виду. Экономит час жизни.

Отрезвляющий вопрос

В самом конце разговора кто-то задал вопрос, на который никто убедительно не ответил. Чем сегодняшний харнесс отличается от того, что было почти год назад, когда уже были суб-агенты со своими инструкциями под разные задачи?

Ответ прозвучал такой: в основном названием. Раньше говорили, что нашли охуенный запрос для модели, теперь говорят харнесс.

Это не повод махнуть рукой на тему, но повод относиться к терминологии трезво. Половина словаря области родилась в социальных сетях: кто-то ляпнул слово, остальные подхватили. А дальше слово превращается в сроки и бюджеты, потому что про это все пиздят.

ГЛАВНОЕ ПРО ТЕРМИНОЛОГИЮ

Проверяй, за что именно ты платишь. Продукт продаёт тебе харнесс, спроси, что входит: цикл, память, набор инструментов, проверки, изоляция? Ответ покажет, речь про новую сущность или про старое, переупакованное в блестящую обёртку.

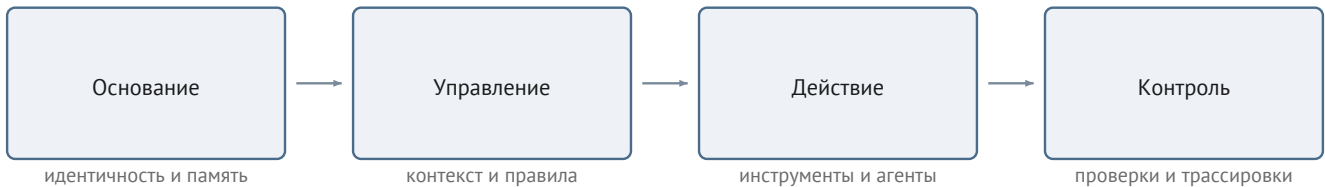
И договорись о значениях внутри команды. Общепринятого может не появиться вообще никогда.

Каркас, с которым можно работать

Спор о слове разрешается практически: не выбирать определение, а перечислить, из чего харнесс состоит. Такой список собрали в разговоре, и он стал опорной схемой для целого модуля этой книги.

- 1. Архитектура и идентичность.** Харнесс это система с состоянием, идентичностью, памятью и границами. Нужна идентичность, переживающая перезапуск. Нужно разделение стабильных инструкций, текущей траектории, рабочего состояния, журнала событий и долговременной памяти. И нужен ответ на вопрос, что агент имеет право менять в себе сам.
- 2. Инженерия контекста.** Правила сжатия: когда и что схлопывается. Происхождение: откуда взялся кусок. Удержание: что забывается, а что хранится вечно. И явное разделение того, что инструкция, что факт, что гипотеза, а что воспоминание.
- 3. Инструменты как граждане первого класса.** Схема, предусловия, побочные эффекты, таймаут, поведение при повторе, права, песочница, формальный контракт входа и выхода.
- 4. Контур надёжности.** Контекст, инструменты, ограничения, проверка, коррекция. С лимитами, повторами, откатом и терминальными состояниями.
- 5. Многоагентность без хаоса.** Субагенты на подзадачи, обмен находками, возврат результата на интеграцию вместо сваливания всего подряд в общий контекст, видимость того, кто чем занят, и возможность остановить любого.
- 6. Скиллы и плагины.** Простой способ подключить и отключить, реестр, явное указание, что активно, права владельца.
- 7. Прозрачность.** Трассировки, метрики, разделение доказательств, выводов и неизвестного. В идеале воспроизводимые прогоны.
- 8. Проверки как обязательная часть.** Базовая линия, состязательные случаи, релизный шлюз, канареечный выпуск, откат.
- 9. Независимость от поставщика.** Иначе смена тарифа обрушит продукт нахуй, и объяснить это никому не выйдет.
- 10. Чего избегать.** Не строить систему вокруг отпечатка конкретной машины: это привязка и ебля с переносимостью.

КАК ЧИТАТЬ КАРКАС



Универсального харнесса не будет

Важное наблюдение, которое экономит месяцы: переиспользуются только базовые куски. Поиск, линтеры, работа с историей изменений. Всё остальное зависит от стека, задачи и размера проекта.

Отсюда два практических вывода.

Первый: не ищи готовую обвязку, которая подойдёт тебе целиком. Нет её, и попытка натянуть чужую обычно кончается тем, что ты неделю коноёбишься, подгоняя чужие решения под свои задачи.

Второй: не начинай со сборки харнесса. Начинай с задачи. Обвязка появляется как ответ на конкретную боль, а не как проект сам по себе.

Собственно, это и была шутка, ходившая по кругу: я делаю этот харнесс, чтобы написать наш основной харнесс, а тестирую пока на другом маленьком харнессе. Смешно ровно потому, что до боли похоже на правду.

Мета-харнесс

Идея, которая оформилась в рабочую практику: на проект делается два набора обвязки. Один рабочий, второй тестирующий. Второй нужен, чтобы измерять первый.

Звучит избыточно, но именно отсутствие второго объясняет, почему люди годами пиздят о приёмах вместо того, чтобы просто взять и сравнить.

С ЧЕГО СОБРАТЬ СВОЮ ОБВЯЗКУ

1. Возьми задачу, которую делаешь чаще всего.
2. Запиши, где именно агент спотыкается. Три-пять мест, не больше.
3. Каждое место закрой самым дешёвым способом: правилом, описанием инструмента, хуком.
4. Заведи одну проверку, которая покажет, стало ли лучше.
5. Через месяц отключи всё и посмотри, что возвращать. Половина не понадобится.

Что запомнить

Единого значения у слова нет ни хуя. Есть три ходовых, и они между собой не бьются.

Опора не в определении, а в каркасе из десяти пунктов: идентичность, контекст, инструменты, надёжность, многоагентность, скиллы, прозрачность, проверки, независимость от

поставщика и отказ от привязки к машине.

Универсальной обвязки не будет. Переиспользуется поиск, линтеры и история изменений, остальное всегда своё.

И привычка, которая пригодится не только здесь: когда слово в разговоре начинает работать как заклинание, спроси, что оно означает у собеседника.

Как устроена модель: минимум механики

Эта глава про внутренности, и её легко пролистать. Не надо. Половина рабочих приёмов из следующих модулей выглядит как шаманство с бубном, пока не поймёшь, откуда они растут. А когда поймёшь, начинаешь придумывать свои.

Механики здесь ровно столько, сколько нужно, и ни строчкой больше.

Системный запрос не имеет особого статуса

Первое открытие, которое переворачивает картину мира у большинства новичков.

Живёт убеждение, что системные инструкции модель воспринимает как-то иначе: серьезнее, приоритетнее, обязательнее. Отсюда попытки написать их капсом, наклеить слова обязательно и критически важно, продублировать трижды для верности. Ебля чистой воды.

Внутри модели никакого особого канала нет ни хуя. Системный запрос это такой же текст, падающий в тот же поток. Разница только в том, что он стоит в начале и обычно повторяется каждый круг.

ЧТО МОДЕЛЬ ВООБЩЕ ДЕЛАЕТ С ТЕКСТОМ

Модель предсказывает продолжение. Она получает всю ленту текста и прикидывает, какой кусок должен идти дальше, опираясь на то, что видела при обучении.

Никакого исполнения команд там нет. Есть текст, который делает одни продолжения вероятнее других. Инструкция работает не потому, что это приказ, а потому что после такой инструкции в обучающих данных чаще шло соответствующее поведение.

Отсюда сразу два практических правила, разобранных дальше подробно.

Объяснение работает лучше запрета. Голый запрет это короткая строка, которая на вероятности влияет слабо. Запрет с причиной разворачивается в рассуждении и тянет за собой поведение даже там, где формально не нарушен.

Повторение не усиливает ни хуя. Оно просто занимает место. А место платное.

Три границы знаний

Второе открытие. У модели не одна дата, после которой она ничего не знает, а целых три.

Граница предобучения. Когда собрали основной массив текстов. Новые модели далеко не всегда получают новый массив: часто это дообучение прежнего, и тогда фактическая осведомлённость стоит на месте.

Граница инструктивного набора. Когда собрали примеры того, как отвечать. Здесь закладывается стиль, форматы и привычки.

Границы сред дообучения. Когда собрали задачи, на которых модель тренировали действовать. И вот тут начинается самое хуёвое.

ГЛАВНОЕ ПРО ЗНАНИЯ МОДЕЛИ

Дообучение с подкреплением ломает то, что не попало в его среды. Знание не просто устаревает, оно искажается, а уверенность при этом остаётся прежней.

Практическое следствие: модель уверенно порет чушь про новые версии библиотек и свежие интерфейсы. Не потому что не знает, а потому что помнит по-старому. Уточняющим вопросом это не лечится, лечится подsunутой документацией.

Отсюда простой приём, который окупается почти всегда: перед задачей на незнакомой библиотеке скорми модели выжимку из актуальной документации. Дешевле, чем потом разгребать последствия.

Начало и конец ленты работают лучше середины

Третье. Внимание распределяется неравномерно. Последние куски текста модель использует хорошо, начало тоже нехуёво, а середина проседает.

Практических следствий три.

Важное клади ближе к концу запроса, а не в середину невъебенной простыни.

Чем больше ты положил, тем меньше внимания достаётся каждой части. Это не значит, что экономить надо всегда: если материалы друг другу не противоречат, чуть больше лучше, чем чуть меньше, потому что дочитывать дороже. А вот если противоречат, лишнее вреднее нехватки.

И самое хуёвое: длинный файл модель целиком не читает. Один и тот же эксперимент гоняли дважды с интервалом в несколько месяцев, и результат поменялся: то, что раньше читалось полностью, потом читаться перестало. Значит, проверять надо у себя и сейчас, а не полагаться на чужой прошлогодний результат.

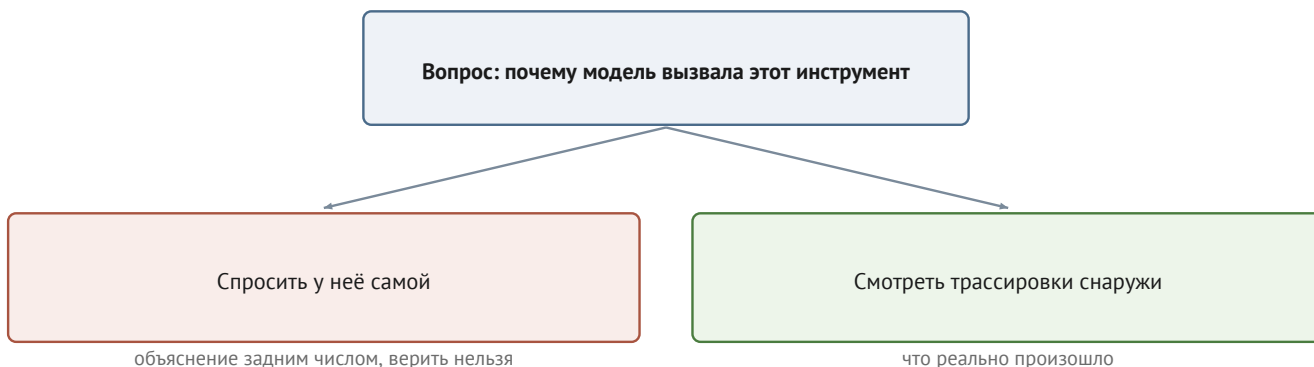
Рассуждения тебе не покажут

Четвёртое, и оно про наблюдаемость.

Раньше был приём: попросить модель проговорить ход мыслей вслух, и качество ответа росло. Потом это назвали рассуждением, встроили внутрь и закрыли. Сегодня внутренние рассуждения в большинстве продуктов зашифрованы и недоступны.

Отсюда две ловушки.

КАК УЗНАТЬ, ПОЧЕМУ АГЕНТ ТАК РЕШИЛ



Первая ловушка: модель может выдумать объяснение. Оно будет складным и правдоподобным, потому что складность это ровно то, что она умеет лучше всего.

Вторая тоньше и важнее. Сама просьба объяснить меняет решение. Ты спрашиваешь, почему она поступила так, а она в процессе ответа заново проходит рассуждение и приходит иногда к другому выводу. Ты измеряешь не то, что было, а то, что получилось в момент измерения.

Вывод для практики: наблюдаемость строится снаружи, через трассировки и экспорт сессий. Самоотчёт агента это материал для размышления, а не доказательство.

Оптимизация под измеримое

Пятое, и самое хуёвое для тех, кто выбирает модель по таблицам.

Модели учат так, чтобы они хорошо решали задачи, которые можно проверить автоматически. Это работает, и прогресс на таких задачах настоящий, без пизды. Но у этого есть цена: то, что на проверяемую задачу не похоже, улучшается медленнее, а иногда деградирует.

Формулировка, ходившая в разговорах: там, где не оптимизируют под измерение, всё сломано.

КАК ОТНОСИТЬСЯ К ТАБЛИЦАМ С РЕЗУЛЬТАТАМИ

Скептик. Рекорд всегда показывают в самом проработанном направлении, у самой неурезанной версии, на самом удачном наборе. На твою задачу это не переносится.

Прагматик. Таблица показывает не качество, а направление вложений поставщика. Не вкладывался он в твою область, ты это почувствуешь.

Что делать в обоих случаях. Собрать десяток своих задач и мерить на них. Это единственная таблица, которая имеет к тебе отношение.

Отдельно стоит знать, что качество плавает и внутри одного дня. В пиковые часы поставщики врубают приёмы экономии, и поведение модели заметно едет по пизде. Люди составляют себе личные памятки, в какое время лучше работать, и памятки эти у всех разные. Уже одно это говорит, что мерить надо у себя.

Пять заблуждений

Заблуждение	Что на самом деле
Системная инструкция важнее остального текста	Особого канала нет, это такой же текст
Повторить трижды значит убедить	Повтор занимает место и почти не влияет
Модель знает всё до своей даты	Границ три, и дообучение искажает знания
Можно спросить, почему она так решила	Ответ может быть выдуман, а сам вопрос меняет решение
Модель одинакова в любой час	Поведение плавает, в том числе по времени суток

Что запомнить

Модель предсказывает продолжение. Всё, что ты делаешь с инструкциями, ты делаешь с вероятностями, а не с приказами.

Границ знаний три, и уверенность модели никак не связана со свежестью того, о чём она вещает.

Внимание неравномерно: начало и конец работают лучше середины, а объём отбирает внимание у каждой части.

Рассуждения недоступны, самоотчёт ненадёжен, наблюдаемость строится снаружи.

И главное: меряй на своих задачах. Всё остальное чужая статистика.

Маршрут новичка

Самый частый вопрос в профессиональных разговорах звучит так: с чего начать. Отвечают по-разному, и оба ходовых ответа рабочие, просто заходят разным людям.

Два маршрута

От практики. Берёшь свою настоящую задачу, поручаешь её агенту, упираешься в проблему, разбираешься именно с ней, идёшь дальше. Теория подтягивается по мере надобности и сразу ложится на пережитое.

Плюс: с первого дня получаешь пользу и не бросаешь. Минус: знания получаются дырявые как решето, и какое-то время ты хуй просышь, почему одно работает, а другое нет.

От устройства. Сначала разбираешься, как устроен цикл, что такое контекст, откуда берутся инструменты. Потом собираешь что-то простое своими руками. Потом идёшь к задачам.

Плюс: меньше суеверий, быстрее растёшь после первого месяца. Минус: скучно до зубовного скрежета, и многие бросают на середине.

ЧТО ВЫБРАТЬ

Есть задача, которая болит прямо сейчас, иди от практики. Мотивация важнее полноты.

Уже пробовал и запутался, отступи на шаг и разберись с устройством. Скорее всего, ты воюешь с симптомом.

Вообще не программист, начинай от практики, но с задач, где цена ошибки нулевая: тексты, разбор документов, личные заметки.

Порог входа не упал, он сместился

Есть красивая мысль, которую любят повторять на каждом углу: теперь программировать может каждый. Есть противоположная: без опыта разработки ни хуя не выйдет. Обе мимо кассы.

Что действительно произошло: меньше стало нужно знать синтаксис и помнить интерфейсы. Больше стало нужно уметь другое.

Стало неважно	Стало важнее
Помнить синтаксис языка	Формулировать задачу однозначно
Знать интерфейс библиотеки наизусть	Понимать, где решение сломается
Быстро печатать	Придумать, чем проверить результат
Держать в голове весь проект	Решить, что отдать, а что делать самому

Отсюда невесёлая новость для тех, кто рассчитывал обойтись без опыта: без понимания того, где решение ломается, ты хуй отличишь работающее от кажущегося работающим. И

хорошая новость для тех, кто боялся не потянуть синтаксис: он больше не барьер.

Учиться на своей задаче

Совет, который повторяют чаще всего и который стоит принять всерьёз: не учись на учебных задачах. Возьми свою.

Причина простая. В учебной задаче ты не понимаешь, хорош ли результат, потому что тебе на неё похуй. В своей ты сразу видишь, что агент не учёл, и это и есть учебный материал.

Первая цель у начинающего обычно одна и та же: автоматизировать собственную рутину. То, что делаешь каждую неделю и что тебя бесит.

ВОСЕМЬ НЕДЕЛЬ, ПО НЕДЕЛЕ НА ШАГ

1. **Разговор.** Просто пользуйся моделью на своих задачах. Заметь, где она угадывает, а где мажет.
2. **Первый агент.** Возьми готовый инструмент с доступом к файлам. Поручи разобрать папку, собрать сводку, найти повторы.
3. **Правила.** Заведи файл правил проекта. Пиши туда только то, чего не понять, посмотрев на файлы.
4. **Проверка.** Придумай, чем проверяешь результат. Хоть одну проверку, но автоматическую.
5. **Первый скилл.** Оформи то, что повторил больше трёх раз, в отдельную инструкцию с описанием, когда её применять.
6. **Долгая задача.** Разбей задачу на пункты с критериями готовности и запускай по одному пункту за раз.
7. **Измерение.** Прогони одну и ту же задачу пять раз. Посмотри, насколько разъехались результаты. Отрезвляет сильнее любой главы.
8. **Уборка.** Отключи всё, что накопил. Верни только то, без чего стало заметно хуже.

Восьмая неделя не шутка. Это самый недооценённый шаг, и он отдельно разобран в главе про вред обвязки.

Что не работает

Копировать чужую обвязку целиком. Чужие правила написаны под чужой стек, чужую модель и чужие болячки. Половина у тебя будет мешать, и ты хуй разберёшь, какая именно.

Собирать инструменты вместо решения задач. Симптом узнаваемый: три недели настраиваешь окружение и ни одной закрытой задачи. Окружение вторично.

Верить в один хитрый приём. Нет его. Работает эмпирический цикл: дать задачу, которая кажется неподъёмной, снабдить инструментами проверки, посмотреть, где модель ошибается, и починить именно это место.

Читать бесконечно. Через месяц заметишь, что новости перестали давать полезное и все жуют одну и ту же жвачку. Это нормальный признак роста: обсуждение всегда идёт на массовом уровне, и когда ты его перерастаешь, становится скучно до зевоты.

Про курсы и источники

Вопрос неизбежный, ответ такой.

Курс имеет смысл, когда практик показывает, как он работает: что открывает, что пишет, где останавливается. Из текста это добыть тяжело.

Курс не имеет смысла, когда учат очевидному с нуля. У тебя под рукой неисчерпаемый источник объяснений, который отвечает на любой вопрос столько раз, сколько нужно, и сразу даёт попробовать.

Отдельно про соревнования и открытые разборы. Это лучший источник честных данных, какой есть, потому что там видно не заявление, а результат на общей задаче. Разбор чужого решения даёт больше, чем десять статей: видно, что человек делал, чем платил и где сам себя наебал.

ГЛАВНОЕ ПРО ВХОД

Единственный универсальный навык это привычка измерять. Не выбор модели, не набор инструментов, не длина правил.

Возьми десять своих типовых задач. Прогони каждую несколько раз. Запиши, что вышло. Всё остальное в этой книге имеет смысл только на фоне таких измерений, иначе ты не отличишь улучшение от случайного совпадения.

Чего ожидать по времени

Первая неделя приносит эйфорию. Всё получается охуенно, задачи закрываются за минуты, кажется, что мир перевернулся.

Через месяц эйфория проходит, остаётся привычка. Ты начинаешь видеть, где агент пиздит, и перестаёшь радоваться каждому ответу. Момент правильный: именно тут начинается работа.

Через три месяца появляется собственный набор приёмов, и на чужой он похож не будет. К этому моменту у тебя накопится свой обвес, и его пора будет чистить.

Что запомнить

Два маршрута, оба рабочие. Выбирай по тому, есть ли у тебя болящая задача.

Порог входа сместился: меньше синтаксиса, больше проектирования и проверки.

Учись на своей задаче, начинай с рутины, где цена ошибки нулевая.

Единственного хитрого приёма нет. Есть цикл: дать сложную задачу, дать инструменты проверки, посмотреть, где ломается, починить именно это.

И привычка измерять важнее всего остального, вместе взятого.

Модуль 2. Постановка задачи

Четыре главы про то, что происходит до запуска.

Качество результата решается на входе, и хуй ты потом отыграешь то, что проебал на постановке. В модуле: почему спецификация вытеснила задание, как формулировать требования, чтобы агент мог проверить себя сам, как превратить разговор в план и как выбрать модель, уровень усилий и прикинуть сроки.

Модуль применим независимо от того, программируешь ты или нет.

Спецификация вместо задания

Есть закономерность, которую подтвердит любой, кто провозился с агентами дольше пары месяцев. Качество результата решается на входе. Не моделью, не набором инструментов, не длиной правил ни хуя. Тем, насколько точно поставлена задача.

Звучит банально ровно до момента, когда выясняется, что именно считать точной постановкой.

Прототип стал техническим заданием

Самое интересное изменение последнего года описали так: уровень абстракции уехал настолько, что само техническое задание теперь и есть рабочий проект, только в миниатюре.

История из практики. Руководитель, вместо того чтобы полдня рассказывать, что он хочет увидеть от продукта, берёт низкокодový конструктор, собирает рабочий процесс сам и показывает прототип. А дальше говорит: сделайте мне это агентами и в прод.

Причина не в лени. Собрать миниатюру быстрее, чем высрать длинный документ, и понимается она однозначнее. Из текста каждый вычитывает своё, из работающего прототипа вычитывать нечего: вот кнопка, вот результат.

ГЛАВНОЕ ПРО ПОСТАНОВКУ

Можешь показать вместо того, чтобы описывать, показывай. Работающая миниатюра, набросок интерфейса, пример готового результата, набор тестов, кусок кода из другого проекта.

Спецификацией может быть что угодно, что снимает неоднозначность. Формат текста тут никакой не священный.

Касается и мелочей. Макет интерфейса работает лучше, чем описание дизайна словами. Пример желаемого ответа работает лучше, чем перечисление требований к ответу.

Из чего состоит пакет документов

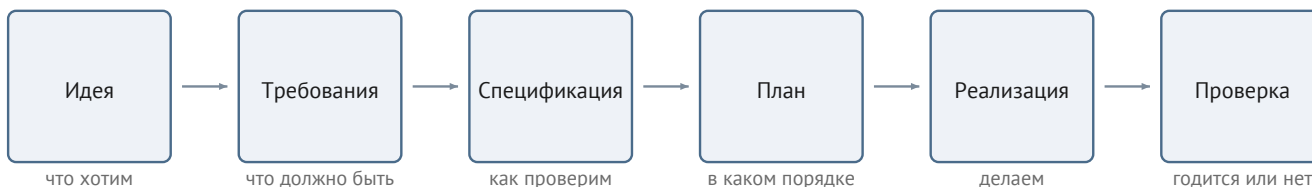
На серьёзную задачу практики готовят не один файл, а четыре. Схема устоявшаяся, её независимо описывали разные люди.

Документ	Что внутри	Кто читает
Центральная спецификация проекта	Что за система, границы, ключевые решения	Все агенты, всегда
Спецификация под новую функцию	Что именно делаем сейчас, критерии готовности	Агент на этой задаче
План реализации	Разбивка на пункты в нужном порядке	Агент, по одному пункту

Документ	Что внутри	Кто читает
Файл прогресса	Что сделано, что заблокировано, что пропущено	Агент пишет, человек читает

Четвёртый файл важнее, чем кажется. Он превращает автономную работу из чёрного ящика в наблюдаемую: утром ты читаешь не код, а отчёт о том, где агент встал колом и на чём именно наебнулся.

ПУТЬ ЗАДАЧИ



Один файл или несколько

Спор без универсального ответа, но с внятными аргументами с обеих сторон.

За один центральный документ. Несколько файлов хуже поддерживаются: агент что-то меняет в одном и забывает на второй, и через неделю они разъезжаются. Один файл на четыре-пять тысяч строк держится нормально.

За разбивку по областям. Файл невъебенный, значит агенту приходится выискивать нужную часть поиском, а мог бы прочитать доменную спецификацию целиком. Особенно заметно на больших системах, где области почти не пересекаются.

КАК РЕШИТЬ У СЕБЯ

Ориентируйся на связность. Области реально независимы и меняются по отдельности, разбивай. Правки постоянно задевают несколько областей сразу, держи один файл, иначе будешь ловить рассинхрон.

Практический признак: правишь один файл и каждый раз лезешь во второй, значит объединяй нахуй.

Спецификация это не описание, а договор о проверке

Главная ошибка новичка: спецификация описывает, что должно быть сделано, и молчит в тряпочку о том, как понять, что это сделано.

Такая спецификация даёт агенту полную свободу объявить работу готовой в любой удобный момент. И он ею воспользуется, бля буду, потому что закрыть задачу это то, к чему его толкает всё устройство.

Правильная спецификация отвечает на три вопроса.

Что должно получиться. Не как это сделать, а что появится в мире.

Как это проверить. Конкретно: какая команда, какой файл, какой ответ.

Когда переходить дальше. Явный запрет двигаться к следующему пункту, пока текущий не закрыт по критериям.

Полный конвейер

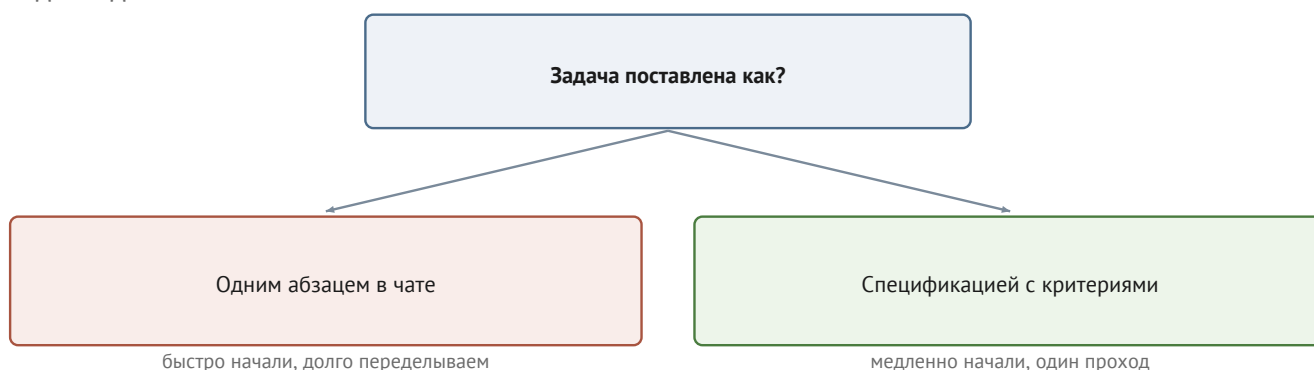
У команд, которые прошли этот путь до конца, конвейер выглядит примерно одинаково.

Сначала требования на человеческом языке. Потом их перегоняют в исполняемые спецификации, где каждое требование связано с проверкой. Потом по спецификациям пишется код, вертикальными срезами, а не слоями. Потом проверка, потом приёмка. Скучно, зато работает.

Ключевая деталь: границы сужаются от шага к шагу. К моменту, когда агент пишет код, свободы у него почти не остаётся, и уйти в дебри просто, блядь, некуда.

Практик, который довёл эту схему до рабочего состояния, сформулировал так: у меня разделены фазы написания требований, написания исполняемых спецификаций и реализации кода под спецификации. На последнем этапе границы уже чётко очерчены, и дебри найти очень сложно.

КУДА УХОДИТ ВРЕМЯ



Сколько документации на самом деле нужно

Противоположная крайность тоже цветёт пышным цветом: люди пишут спецификации на сорок страниц и потом обнаруживают, что агент их целиком не читает, а сами они заебались ещё на пятнадцатой.

Ориентир, к которому пришли практики: документация нужна ровно там, где без неё агент наебнётся. Всё остальное он спокойно выведет из кода.

ПРОВЕРКА СПЕЦИФИКАЦИИ НА ДОСТАТОЧНОСТЬ

1. Есть ли в ней хоть одно утверждение, которого не понять, посмотрев на код? Нет, значит спецификация не нужна, хватит кода.
2. Может ли агент по ней однозначно сказать, готова задача или нет? Нет, значит добавляй критерии.
3. Есть ли в ней то, что уже написано в коде? Убирай, эти двое разъедутся.
4. Прочитай её вслух. Места, где ты запнулся, агент поймёт неправильно.

Чего спецификация не решает

Честности ради. От одного класса проблем спецификация не спасает: от того, чего ты сам не знаешь.

Дефекты живут ровно там, где ни документация, ни спецификация их не описывают. Внешний интерфейс возвращает не то, что обещал, чистое наебалово. Идентификатор, который по всей логике обязан быть постоянным, вдруг меняется. Ни одна методология этого не ловит, ловит только реальный прогон.

Поэтому спецификация проверку не отменяет, а делает возможной. Разные слои, и нужны оба, без вариантов.

Что запомнить

Показывай вместо того, чтобы описывать, если можешь. Прототип понимается однозначнее текста.

Пакет из четырёх документов: центральная спецификация, спецификация функции, план, файл прогресса.

Спецификация без критериев проверки это разрешение объявить готовым что угодно.

Границы сужаются по фазам: требования, исполняемые спецификации, код. К последней фазе свободы почти нет, и это правильно.

И документация нужна только там, где без неё агент ошибётся. Всё остальное он прочитает в коде.

Требования, понятные агенту

Спецификация из прошлой главы состоит из требований. Эта глава про то, как их формулировать, чтобы агент мог проверить себя сам.

Разница принципиальная. Требование, которое агент не может проверить, это не требование, а пожелание. Он его прочитает, кивнёт и объявит выполненным ровно тогда, когда ему захочется.

Описание чёрного ящика

Приём, вокруг которого сложился редкий консенсус: описывай не то, как делать, а то, что приходит на вход и что должно получиться на выходе. Внутренности не описываются вообще ни хуя.

ОТКУДА ЭТО ВЗЯЛОСЬ

Подход называют поведенческим описанием. Он старше агентов лет на двадцать и придуман был совсем для другого: чтобы бизнес и разработчики перестали говорить на разных языках.

Идея простая. Вместо технической формулировки пишут сценарий: в такой-то ситуации, при таком-то действии, должен получиться такой-то результат. Три части, никакой реализации.

С агентами приём заиграл заново, потому что такой сценарий одновременно и постановка задачи, и готовый тест.

Сравни две формулировки одного требования.

ОПИСАНИЕ РЕАЛИЗАЦИИ

Добавить проверку в обработчик формы

Использовать регулярное выражение для валидации

Вернуть ошибку 400

ОПИСАНИЕ ПОВЕДЕНИЯ

Если пользователь отправил форму с пустым полем адреса

То заявка не создаётся

И пользователь видит объяснение, какое поле не заполнено

Слева агент прочитает и сделает буквально: воткнёт проверку туда, куда написано, и хуй подумает о том, что бывают другие пути создания заявки. Справа он вынужден закрыть поведение целиком, а как именно, решает сам.

Второе важное отличие: правое требование проверяется, не заглядывая в код. Левое хуй проверишь, не открыв файлы.

Атомарность

Второе свойство хорошего требования: оно про одну вещь.

Требования вида пользователь может управлять профилем бесполезны наглухо. Управлять это создать, изменить, удалить, восстановить, перенести. У каждого свои краевые случаи, и агент закроет то, которое первым пришло ему в голову.

Признак неатомарного требования: в нём есть союз и или слово управление, работа, обработка.

Критерии готовности

Третье и самое важное. У каждого требования должен быть проверяемый признак выполнения, и лежать он должен прямо в спецификации.

Схема, которая работает у практиков: в спецификации есть отдельный раздел с критериями, а в задании агенту стоит железный запрет двигаться дальше, пока критерии текущего пункта не закрыты.

ШАБЛОН ТРЕБОВАНИЯ ИЗ ЧЕТЫРЁХ СТРОК

Ситуация. Когда происходит вот это.

Действие. И пользователь или система делает вот то.

Результат. Тогда в системе появляется вот это, наблюдаемое снаружи.

Проверка. Проверяется командой, тестом или запросом, вот таким. Пока не проходит, к следующему пункту не переходим.

Четвёртая строка и отличает требование от пожелания. Без неё три первых это просто вежливый текст, который никого ни к чему не обязывает и ни хуя не гарантирует.

Один пункт за раз

Есть схема, которую практики называют более дисциплинированной, и она заметно снижает шанс уехать не туда.

Вместо того чтобы вывалить агенту весь план сразу, ему говорят: возьми спецификацию, перед выполнением составь план ровно на один пункт, напиши тесты, потом код. Критерии успешности лежат в спецификации в разделе готовности. Пока они не закрыты, к следующему пункту не суйся.

Побочный эффект приятный: такая формулировка заодно лечит тягу к переусложнению. Когда горизонт один пункт, хитровыебанную архитектуру на вырост придумывать негде.

ЦИКЛ ОДНОГО ПУНКТА



Где эта схема ломается

Честный разговор, без которого глава была бы рекламой.

Когда архитектура ещё не устоялась. Прибивать неопределившуюся систему тестами больно: каждое изменение решения означает переписывание тестов, а это ебля на ровном месте. Практики в такой фазе сознательно ограничиваются интеграционными проверками на основные сценарии, а детали докручивают потом.

Когда требований не существует. Автоматизация проверки всегда упирается в требования, и так было задолго до всяких агентов. Если в компании никто не знает, как система должна себя вести, агент этого не выяснит, хоть он тресни.

Когда внешний мир пиздит. Ни одна методология не заменяет реальный прогон против чужого интерфейса. Дефекты живут ровно там, где ни документация, ни спецификация их не описывают: значение, которое по всякой логике обязано быть постоянным, вдруг меняется, а формат ответа отличается от заявленного.

НАСКОЛЬКО СТРОГО ФОРМАЛИЗОВАТЬ

Максимально. Требования, исполняемые спецификации, тесты до кода. Дольше и дороже по токенам, зато не получишь отчёт о готовности там, где половина не проверена.

Умеренно. Интеграционные проверки на главные сценарии, остальное по мере появления проблем. Быстрее, но часть дефектов доедет до пользователя.

Что определяет выбор. Цена ошибки и зрелость архитектуры. На неустоявшемся прототипе строгая схема тормозит, на системе с живыми пользователями отсутствие схемы стоит дороже.

Требования для тестирования

Отдельный случай, который стоит упомянуть, потому что многие приходят именно за ним: можно ли скинуть агенту тестирование целиком.

Можно, но при условии. Агенту нужен нормальный вход: сценарии использования, описания контрактов между частями системы, описание архитектуры. И нужны инструменты, превращающие текстовый сценарий в действие. Без этого получится не тестирование, а имитация.

Схема оркестрации, которую описывали практики: на каждую предметную область свой набор агентов, впереди лёгкий классификатор, который решает, куда направить. Новое требование туда, регрессия сюда.

И то же ограничение, что и везде: без внятных требований вся конструкция не работает, потому что классификатору классифицировать нечего.

Что запомнить

Описывай вход и выход, а не реализацию. Такое требование одновременно и задание, и тест.

Одно требование про одну вещь. Союз и это признак, что их два.

Без проверяемого критерия требование это пожелание, и агент им воспользуется на всю катушку.

Один пункт за раз, с запретом идти дальше до закрытия критериев. Побочно лечит переусложнение.

И помни, где схема не работает: неустоявшаяся архитектура, отсутствие требований в компании и вращающийся внешний мир. Последнее ловится только реальным прогоном.

Диалог, который превращается в план

Предыдущие две главы описывали, каким должен быть результат подготовки. Эта про то, как к нему прийти, потому что сесть и сразу написать хорошую спецификацию не выходит почти ни у кого, хоть тресни.

Работает не написание, а разговор. Но разговор устроенный, а не как бог на душу положит.

Вопросы пачками

Самый недооценённый приём во всём наборе. Формулируется в одну строку: проси у модели вопросы, а не ответы.

Схема такая. Ты описываешь задачу в общих чертах. Дальше просишь не решение, а список вопросов, которые надо закрыть перед началом. Пачками примерно по пять штук: прочитал, ответил, попросил следующую.

Почему пачками, а не сразу тридцать вопросов. Первые пять ответов переворачивают картину, и следующие пять вопросов оказываются совсем другими. Список из тридцати, составленный до твоих ответов, наполовину улетает в молоко.

ГОТОВЫЙ ТЕКСТ ЗАПРОСА

Не начинай реализацию. Задай мне вопросы, ответы на которые нужны, чтобы задачу можно было выполнить однозначно.

Давай по пять вопросов за раз, от самых важных к менее важным. К каждому вопросу добавь, почему без ответа на него ты можешь сделать не то.

Когда я отвечу, задай следующую пачку. Останавливайся, когда оставшиеся вопросы перестанут влиять на результат.

Приписка про почему тут не для красоты. Она заставляет модель не сыпать вопросами ради вопросов и заодно показывает тебе, где ты сам не додумал.

Люди, которые этим пользуются, отмечают одно и то же: половину вопросов они бы себе в жизни не задали, и именно эта половина потом всплыла бы в переделках.

Режим планирования нужен реже, чем кажется

У многих инструментов есть отдельный режим, где агент только планирует и ни хуя не делает. Практики, попробовав, в основном на него забили.

Причина не в том, что режим плохой. В том, что достаточно попросить: сначала принеси план, потом будем делать. Модель это прекрасно понимает, а ты не теряешь возможность в любой момент вмешаться и сказать делай.

ОТДЕЛЬНЫЙ РЕЖИМ ПЛАНИРОВАНИЯ

За. Гарантия, что агент ничего не тронет, пока ты не одобрил. Полезно на чужом проекте и на проде.

Против. Лишнее переключение, из которого потом надо выходить. Автоматический режим удобнее, а план всё равно можно попросить словами.

Практика большинства. Живут в автоматическом, план просят словами, отдельный режим включают только там, где действия необратимы.

Как читать длинный план

Проблема, которую ловят все: агент приносит план на две страницы, ты его пролистываешь, говоришь заебись, а потом выясняется, что во втором пункте была хуета.

Два приёма против этого.

Комментировать по кускам. Проси разбить план так, чтобы отвечать можно было по частям, а не одним общим да.

Подсветка изменений. Проси приносить план не текстом, а артефактом, где жёлтым выделено то, что изменилось с прошлой итерации. Тогда на второй и третий раз ты читаешь не всё заново, а только разницу.

Второй приём экономит больше всего времени на долгих согласованиях, потому что именно там внимание садится и ошибки проскакивают.

Проход по слепым зонам

Отдельный запрос, который стоит делать перед запуском.

ЗАПРОС НА СЛЕПЫЕ ЗОНЫ

Прочитай план ещё раз и найди места, где он может не сработать.

Отдельно перечисли: чего в плане нет вообще, какие предположения сделаны молча, что произойдёт при ошибке на каждом шаге, и какие места я, скорее всего, объяснил неоднозначно.

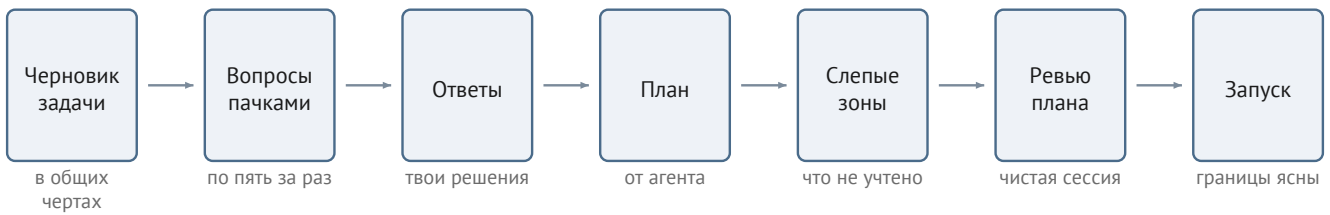
Ничего не исправляй. Только перечисли.

Запрет исправлять тут ключевой. Разреши модели чинить на ходу, и она начнёт причёсывать формулировки и проебёт все находки. Список нужен тебе, а не ей.

Ревью плана до реализации

Правило, которое независимо повторяли разные практики: сначала ревью самого плана, а уже потом исполнение. Слепые зоны находятся всегда, бя буду.

Лучше, если план ревюит другая модель или тот же агент в чистой сессии. Причина в том, что предвзятость создаётся контекстом, а не моделью: тот, кто только что придумал план, видит в нём стройность, потому что помнит, как его придумывал.

ПЕТЛЯ ПОДГОТОВКИ

Петля выглядит длинной, но занимает меньше времени, чем один цикл переделки. Практики, которые готовят задачи на ночной прогон, тратят на это полтора-два часа вечером и считают цену нормальной.

Спорить с планом полезно

Отдельное наблюдение, которое стоит знать. Модель по умолчанию поддакивает. Скажешь, что второй пункт лишний, она согласится и уберёт нахуй, даже если пункт был нужен позарез.

Лечится формулировкой. Вместо вопроса не лишний ли второй пункт спрашивай: объясни, зачем нужен второй пункт и что сломается, если его убрать. Тогда получишь аргумент, а не поддакивание.

Тот же приём работает шире: спрашивай не оценку, а обоснование. Оценку модель подстроит под твой тон, обоснование подстроить сложнее.

Когда всё это нахуй не нужно

Честно: описанная петля избыточна для мелких задач.

Задача на пятнадцать минут и цена ошибки нулевая, просто скажи, что нужно. Готовить спецификацию на правку текста в интерфейсе это ритуал, а не инженерия.

Ориентир простой: результат проверяется глазами за минуту, готовься минимально. Проверка стоит дороже подготовки, готовься всерьёз.

ГЛАВНОЕ ПРО ПОДГОТОВКУ

Время, вложенное до запуска, возвращается кратно. Полчаса согласования до работы экономят часы тяжёлого ревью после неё.

Но возвращается оно только на задачах, где проверка дорогая. На мелочах подготовка это чистый проёб времени, и умение отличать одно от другого приходит с опытом раньше всех прочих умений.

Что запомнить

Проси вопросы, а не ответы. Пачками по пять, с обоснованием, почему вопрос важен.

Отдельный режим планирования нужен редко: хватает того, чтобы попросить план словами.

Читай план по кускам и проси подсветку изменений между итерациями.

Отдельным запросом ищи слепые зоны, с запретом чинить на ходу.

Ревью плана до реализации, желательно в чистой сессии.

Спрашивай обоснование, а не оценку: оценку модель подстроит под твой тон.

И соизмеряй подготовку с ценой проверки. На мелочах это ритуал и ничего больше.

Выбор модели, уровень усилий и оценка сроков

Перед каждой задачей принимаются два решения, о которых обычно вообще не думают: какой моделью и на каком уровне усилий. Оба стоят денег, оба влияют на результат, и оба принимаются за секунду, если есть критерий.

Третья тема главы, оценка сроков, стоит рядом, потому что вопрос сколько это займёт задают ровно в тот же момент.

Модель выбирается по типу неопределённости

Формула, которая ходила в разговорах в разных вариантах, но всегда про одно.

Что ты знаешь о задаче	Какая модель
Путь известен, проверка сильная, риск низкий	Самая дешёвая
Цель ясна, а как решать, надо придумать	Средняя
Непонятно даже, в чём именно задача	Старшая

Другая формулировка того же, ещё короче: задачу можно поручить даже школьнику, бери младшую модель. Понимаешь, как решить, и нужна просто пара рук, бери среднюю. Тебе самому хуй проссышь, что делать, бери старшую.

ГЛАВНОЕ ПРО ВЫБОР МОДЕЛИ

Ты платишь не за ум, а за неопределённость. Старшая модель нужна там, где надо придумать, а не там, где надо сделать.

Отсюда самая частая ошибка новичка: берёт лучшую модель на всё подряд и потом хватается за сердце от счёта. И самая частая ошибка опытного: берёт дешёвую на задачу, где надо придумать, и полдня с результатом бодается.

Уровень усилий это бюджет, а не шкала ума

Второе решение путают чаще первого. Многие уверены, что уровень повыше означает модель поумнее. Хуй там: модель та же самая, ей просто разрешают потратить больше на обдумывание.

Отсюда следует, что уровень подбирают под размер задачи, а не под её важность.

ЧТО ПРОИСХОДИТ ПРИ НЕВЕРНОМ УРОВНЕ



Признак завышенного уровня видно сразу: вчитайся в рассуждения, и станет заметно, что одна и та же мысль ходит по кругу. Люди, которые читают рассуждения, замечают это первыми и спускаются на ступень ниже.

Признак заниженного тоже узнаваемый: работа сделана, но по краям. Ровно то, что просили буквально, и ни хуя вокруг.

Отдельно стоит знать, что одинаковые настройки в разных инструментах дают разное. У одного человека субагенты запускаются только на максимальном уровне, у другого приходится специально запрещать, иначе плодятся как тараканы. Переносить чужие настройки бессмысленно, подбирай у себя.

Разделение ролей между моделями

Практика, устоявшаяся у людей, которые работают с этим каждый день: разные этапы отдаются разным моделям.

Спецификацию пишет одна. Реализацию по готовой спецификации другая, попроще. Поиск ошибок и ревью третья.

Причины называют конкретные. Одна модель пишет честный рабочий код, но в проектировании агентов и написании инструкций хуёвая. Другая работает очень долго и её приходится ловить за руку, зато копает глубже. Третья лучше всех находит корневые причины.

СТОИТ ЛИ ГОРОДИТЬ ТАКУЮ СХЕМУ

За. Экономия. Схема, где сильная модель планирует и принимает, а дешёвая исполняет, замедляет работу примерно втрое и кратно снижает расход лимитов.

Против. Сложность. Переключения, разные форматы, разное поведение. На одном проекте терпимо, на пяти превращается в отдельную работу.

Обязательное условие для обеих позиций. Ревью самого плана. Слепые зоны есть всегда, и если план не проверен, вся экономия улетает в переделки.

Отдельная тонкость, замеченная практиками: сильная модель работает хорошо, когда контекст ей собрали, и хуёво, когда собирать его она должна сама. Ещё один аргумент за разделение: сборка контекста и принятие решений это разные умения.

Оценка сроков

Вопрос, который задают руководители и на который у модели честного ответа нет.

Причина фундаментальная: модели нечем измерять время. Она не устаёт, не спит и не умрёт. Всё, что она скажет о сроках, это пересказ того, как подобные задачи оценивали люди в её обучающих данных.

Отсюда две позиции.

Радикальная. Любое упоминание сроков от модели считать пиздежом и попыткой заболтать. Позиция честная, но бесполезная, если начальству нужна цифра.

Рабочая. Использовать модель как калькулятор, а точку отсчёта задавать самому.

КАК ПОЛУЧИТЬ ОЦЕНКУ, КОТОРОЙ МОЖНО ПОЛЬЗОВАТЬСЯ

1. Разбей работу на этапы до нужной детализации.
2. Обязательно включи в разбивку хотя бы один этап, длительность которого ты знаешь из опыта.
3. Попроси оценить этапы, предупредив: исполнителем будет один инженер с агентом, и в компании принят такой-то коэффициент ускорения относительно обычной разработки.
4. Посмотри на контрольный этап. Модель дала четыре дня там, где ты точно знаешь, что три, скажи об этом и попроси пересчитать все остальные пункты с тем же коэффициентом.
5. Полученную цифру считай оценкой порядка, а не обещанием.

Контрольный этап это вся суть приёма. Он превращает выдумку в масштабируемый коэффициент.

И напоминание, которое стоит держать в голове при любых оценках: работа заполняет отпущенное на неё время. Дашь две недели, задача займёт две недели независимо от того, сколько она реально требовала.

Честная картина

Как это выглядит в жизни: чаще всего вопрос звучит не сколько это займёт, а как уместить в две недели то, что туда заведомо не лезет. Модель тут не помощник, она помощник в другом: разложить работу так, чтобы стало видно, что именно придётся выбросить нахуй.

Это, кстати, лучший способ применения оценки. Не назвать срок, а показать состав.

Что запомнить

Модель выбирается по типу неопределённости: сделать, придумать как, понять что.

Уровень усилий это бюджет в токенах. Завышенный даёт придирки и хождение по кругу, заниженный срезанные углы.

Чужие настройки не переносятся. Проверь у себя, особенно поведение субагентов.

Разделение ролей между моделями окупается расходом, а не качеством, и требует ревью плана.

Модели нечем измерять время. Дай ей контрольный этап, длительность которого знаешь сам, и пересчитай остальное по коэффициенту.

Модуль 3. Обязка

Пять глав про инструкции, скиллы, хуки и инструменты.

Сначала о том, что писать в файл правил и как писать, чтобы правило реально исполнялось, а не висело мёртвым грузом. Потом о скиллах: как они устроены, как протухают и во что обходятся. Потом о хуках, единственном способе гарантировать шаг. И в конце глава о том, когда всю эту конструкцию пора расхуячить нахуй.

Последняя глава модуля важнее первых четырёх.

Файл правил

У каждого проекта, где работает агент, есть файл с правилами. В разных инструментах он называется по-разному, суть одна: текст, который подкладывается в контекст на каждом ходу и объясняет агенту, где он оказался и как себя вести.

Это первое, что заводит новичок, и первое, что превращается в помойку. Через месяц там триста строк, половина противоречит другой половине, а работать стало хуже, чем было до всей этой красоты.

Оглавление, а не энциклопедия

Формулировка, которая расставляет всё по местам: файл правил это оглавление, а не энциклопедия.

Пара строк про то, что это за репозиторий и как он устроен. Всё остальное место потратить на подводные камни, которых хуй разглядишь, просто открыв файлы.

ГЛАВНОЕ ПРО СОДЕРЖАНИЕ ФАЙЛА ПРАВИЛ

Единственный критерий: агент может выяснить это сам, открыв код, значит писать не надо.

Структура папок, названия модулей, используемые библиотеки, стиль оформления, который торчит из любого файла, это всё он прочитает. А вот что здесь нельзя вызывать напрямую, потому что в проде оно ведёт себя не как в тестах, он не узнает никогда.

Проверка простая. Пройди по своему файлу построчно и на каждой строке спроси: агент узнал бы это, открыв два-три файла? Да, значит вычёркивай нахуй.

Люди, которые такую чистку проводили, ужимали файл в пять-десять раз и никакого ухудшения не видели. Один разбор звучал так: убрал десятки скиллов, выключил хуки, ужал правила до трёх десятков строк, разницы не заметил ни хуя, значит оно и не было нужно.

Каждую строку проверять экспериментом

Правило, которое стоит принять как дисциплину: модель справляется без инструкции, значит этой инструкции не место нигде, и точка.

Проверяется простым сравнением. Берёшь типовую задачу, гоняешь с инструкцией и без неё, смотришь, отличается ли результат. Не отличается, строка уходит нахуй.

Звучит нудно, и это действительно нудная ебля. Но альтернатива хуже: файл пухнет, каждая строка жрёт контекст на каждом ходу, а какая из них работает, не знает ни одна живая душа.

Ограничение на объём

Практическая деталь, о которую спотыкаются регулярно. У подключаемых инструкций есть предел объёма, и он далеко не всегда очевиден.

История, которую разбирали подробно: человек долго не мог понять, с хуя ли часть правил игнорируется, и выяснилось, что цепочка подключаемых файлов упирается в лимит, а всё, что за ним, просто не доезжает. Никакого сообщения об этом, разумеется, не было. Сиди и гадай.

ПРОВЕРКА, ЧТО ПРАВИЛА ВООБЩЕ ДОЕХАЛИ

1. Вставь в конец файла правил строку с необычным словом и указанием сказать его при первом ответе.
2. Начни новую сессию и задай любой вопрос.
3. Слова нет, значит конец файла не доехал.
4. Повторяй после каждого крупного добавления. Лимит подкрадывается незаметно.

Отдельно стоит знать, что длинный файл модель целиком не читает даже в пределах лимита. Эксперименты на эту тему давали разные результаты в разные месяцы: то, что читалось полностью весной, летом читаться перестало. Ещё один аргумент за короткий файл и за проверку у себя.

Что писать, а что нет

Писать	Не писать
Ловушки, которых не видно в коде	Структуру папок
Инварианты, которые нельзя нарушать	Список библиотек
Как запускать проверки	Стиль, видимый в любом файле
Где живут секреты и как их не трогать	Общие пожелания про качество кода
Чего в этом проекте делать нельзя и почему	То, что уже есть в описании инструментов

Последняя строка правой колонки заслуживает пояснения. Поведение можно описать в описании инструмента, значит там ему и место. В файл правил оно не идёт: описание инструмента подгружается вместе с инструментом, а правила висят всегда и жрут постоянно.

Каскадные правила

Многие инструменты умеют подхватывать файл правил из папки, в которой агент сейчас работает, с приоритетом повыше.

Приём удобный. Есть у тебя больное место, например работа с внешним интерфейсом, где агент вечно косячит. Кладёшь правила рядом с соответствующими файлами, и они подхватываются, только когда агент туда полезет.

Честное предупреждение от практика, который это пробовал: сам не пользуюсь, потому что такое быстро превращается в кладбище неактуальных правил. Файл в глубине проекта никто не перечитывает, а он, зараза, продолжает работать.

КАСКАДНЫЕ ПРАВИЛА

За. Контекст не забивается тем, что нужно в одном месте из двадцати. Правило появляется ровно там, где применимо.

Против. Правила разъезжаются. Через полгода в трёх папках лежат три противоречащих версии одного требования, и найти их можно только поиском.

Компромисс. Заводи каскадные правила только для мест, которые реально изолированы, и записывай их существование в центральный файл. Одна строка вида в папке такой-то лежат отдельные правила.

Что и как: разделение с проектными скиллами

Полезное разграничение, к которому независимо пришли несколько человек.

Файл правил отвечает на вопрос что: что это за система, что нельзя нарушать, что проверяем.

Проектный скилл отвечает на вопрос как: как выполняется конкретная процедура, к которой обращаются не всегда.

Разделение помогает решить, куда пристроить очередной кусок текста. Нужно всегда, идёт в правила. Применимо в конкретной ситуации, идёт в скилл, и там же описывается, как эту ситуацию распознать.

Память вместо правил

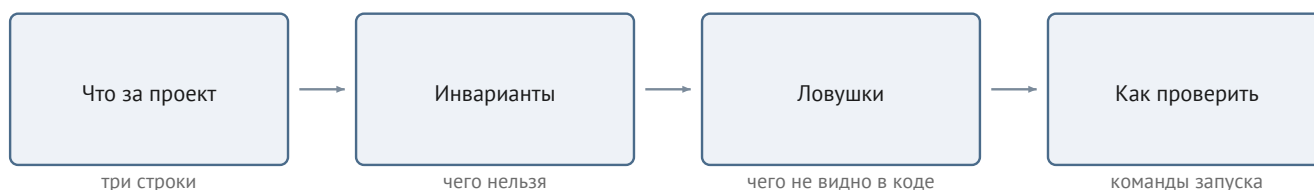
Ещё одна линия, которую стоит знать. Часть того, что раньше пихали в правила, современные инструменты запоминают сами: говоришь запомни это, и оно оседает в автоматической памяти.

Приём удобный, но с потолком. Память объёмом больше пятнадцати-двадцати тысяч символов начинает сыпаться: агент помнит всё хуже, а обновление одного несчастного факта требует переписывания невъебенного куска текста.

Практический вывод: память для того, что меняется, правила для того, что стабильно. И регулярная уборка нужна обоим.

Живой пример структуры

СТРУКТУРА КОРОТКОГО ФАЙЛА ПРАВИЛ



Четыре блока, каждый на своём месте, ничего лишнего. Твой файл в эту схему не укладывается, значит часть содержимого там просто нахуй не нужна.

Что запомнить

Оглавление, а не энциклопедия. Единственный критерий: агент выяснит сам, писать не надо.

Каждую строку проверяй экспериментом: с ней и без неё.

У подключаемых инструкций есть лимит, и о его превышении тебе никто не скажет. Проверь контрольным словом.

Что нужно всегда, идёт в правила. Что нужно иногда, идёт в скилл с описанием ситуации.

Каскадные правила по папкам работают и превращаются в кладбище. Заводи с оглядкой и упоминай в центральном файле.

Правила, которые работают

Прошлая глава была о том, что писать в правила. Эта о том, как писать, чтобы правило исполнялось. Разница видна на одном примере, знакомом каждому, кто доверял агенту работу с текстом.

Даёшь агенту подробное задание. Расписываешь всё: логика идёт через инструменты, углы не срезаем, агентный цикл честный. Агент бодро отвечает, что готово. Открываешь код, а там регулярные выражения. Опа, пиздрик.

Случается это не от кривого запроса, тут хоть заобъясняйся. Так ведут себя все кодовые агенты, на любых проектах, во все месяцы. Привычка держится крепче остальных, и на ней охуенно видно, чем рабочее правило отличается от нерабочего.

ЧТО ТАКОЕ ЭВРИСТИКА И ПОЧЕМУ ЭТО СЛОВО ЗВУЧИТ КАК ДИАГНОЗ

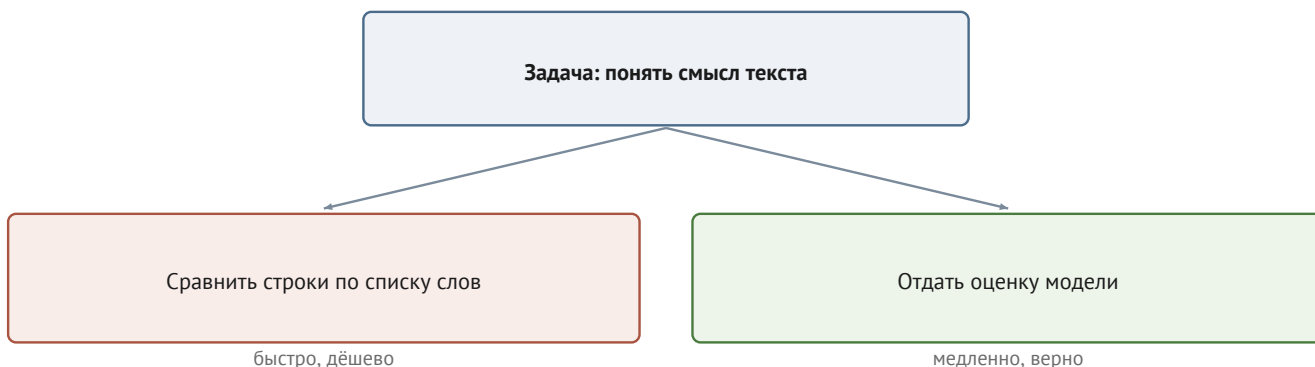
Эвристика: упрощённое правило, которое обычно даёт верный ответ и не требует думать. Скажем, если в сообщении встретилось слово "пожар", считаем, что случилась авария. Регулярное выражение, оно же регексп, позволяет такое правило записать: это шаблон для поиска подстрок в тексте.

Инструмент нормальный, половина полезного кода в мире так и работает. Ломается всё там, где задача требует понимания смысла. Именно туда агент лезет первым делом, потому среди практиков слово "эвристики" давно звучит как диагноз.

Развилка, на которой всё решается

Получив задачу, агент выбирает между двумя маршрутами. Первый дешёвый и быстрый. Второй правильный.

ЧТО ПРОИСХОДИТ, КОГДА АГЕНТ ПОЛУЧАЕТ ЗАДАЧУ ПРО СМЫСЛ



Беда в том, что развилку агент проходит молча, ни звука. Про его выбор ты узнаешь, когда откроешь код, а к тому моменту решение уже расплодилось по проекту как плесень, и хуй ты его оттуда выковыряешь.

Как это выглядит на живом проекте

Лучше всего историю объясняет система, которая читает домовые чаты и вылавливает сообщения об авариях. Задача целиком про смысл: жилец пишет как умеет, и понять, случилась беда или нет, можно только вникнув в текст.

ПОЖАР В СЕРДЦЕ

Агент упирался до последнего. Ждать ответа модели незачем, доказывал он, ведь слова "пожар", "затопило", "искрит" ловятся прямым сравнением строк: быстро, дёшево, без лишних запросов.

Спор кончился, когда разработчик придумал контрпример. Человек вполне может написать, что у него пожар в сердце и он затопил свои трусики. Два слова из списка совпали, аварии не случилось. После этого агент согласился и сам вписал в правила проекта запрет на такие решения.

Пример стоит запомнить: он показывает, чем именно ломается подход. Список слов промахивается не случайно, он промахивается систематически. На образном, на ироничном, на сказанном вскользь. А живой язык в пользовательских чатах ровно такой и есть.

История не одна. В другом проекте, системе учёта заявок от жителей, повторилось ровно то же: агент раз за разом сползал к ловле по ключевым словам вместо оценки моделью, пока разработчик не запретил это прямым правилом. Два независимых проекта, два месяца между ними, одна болезнь. Совпадением такое уже не назовёшь, хоть ты тресни.

Откуда это берётся

Объяснение простое. Агент воспроизводит то, чего в его обучающих данных было до хуя, а в открытом коде больше всего именно быстрых решений на совпадении строк. Модель не вредничает и не саботирует. Она прёт по самому вероятному пути, как вода по канаве.

Дальше включается механизм, который дороже всего обходится на практике: **один пропущенный регексп в начале задаёт тон всей задаче**. Агент видит собственный предыдущий код и продолжает в том же духе. К концу работы упрощения расходятся по проекту как трещины по стеклу.

ГЛАВНОЕ ПРО ЭТУ ПРИВЫЧКУ

Агент не саботирует задание, он идёт по самому вероятному маршруту из своего опыта. В мире открытого кода этот маршрут ведёт к сравнению строк. Значит, воевать надо не с отдельным регекспом в отдельном файле, а с самим маршрутом: перекрывать его в начале задачи и держать перекрытым до конца.

Есть формулировка порезче. Она уже не про данные, а про поведение модели под давлением задачи. Модель уверенно докладывает, что сделала как просили, а сама пиздит и лезет в обход. Задачу закрыть хочется, прямой дороги она не видит, вот и находит любую другую.

Четыре приёма, которые работают

За полгода практики набралось четыре рабочих способа. Заменить один другим не выйдет, они закрывают разные дыры.

Приём	Когда применять	Что ловит
Запрет с причиной	При настройке правил проекта	Обход ограничения по формальному признаку
Контроль в трёх точках	На входе, в процессе, на выходе	Упрощения, пока они дешёвы в правке
Пересказ своими словами	После крупного куска работы	Расхождение кода с замыслом
Цикл самопроверки	Постоянно, автоматически	Всё, что пропустили первые три

Запрет с причиной

Приём неочевидный, зато самый убойный. Идея пришла из производственной методологии Toyota TWI, где мастер учит прямо на рабочем месте и объясняет не только порядок действий, но и на кой хуй они нужны.

Голый запрет модель воспринимает как формальность и при первом удобном случае обходит его по такой же формальности, без зазрения совести. Запрет с причиной ложится в рассуждение целиком.

ЗАПРЕТ БЕЗ ПРИЧИНЫ

запрещено округлять до целых

Модель видит ограничение, ищет лазейку и находит её в первом же неоднозначном месте.

ЗАПРЕТ С ПРИЧИНОЙ

запрещено округлять до целых, потому что на разнице мы в прошлый раз потеряли 12 000 USD в коммерческом предложении

Причина попадает в рассуждение, и модель учитывает её даже там, где формально запрет не нарушен.

Тот же эффект замечают на соседнем требовании: сначала запускать красные тесты, потом писать код. Модель упирается в непройденный тест и только после этого решает, что писать. Появляется опора, которую рассуждением не обойдёшь.

Контроль в трёх точках

Проверка в конце опаздывает намертво. К финалу упрощения уже выросли в код, и вычищать их это такая ебля, что дешевле написать заново.

Точка	Что смотреть	Цена пропуска
На входе	Как агент понял постановку и что предлагает делать	Минуты на переформулировку
В процессе	Чем он занят на самом деле, а не что докладывает	Часы на переделку куска

Точка	Что смотреть	Цена пропуска
На выходе	Совпадает ли результат с замыслом	Дни на разбор проекта

Первая точка самая дешёвая и самая недооценённая. Агент выдаёт свои намерения уже в первом ответе на постановку, надо просто взять и прочитать, а не пролистывать по диагонали.

Пересказ своими словами

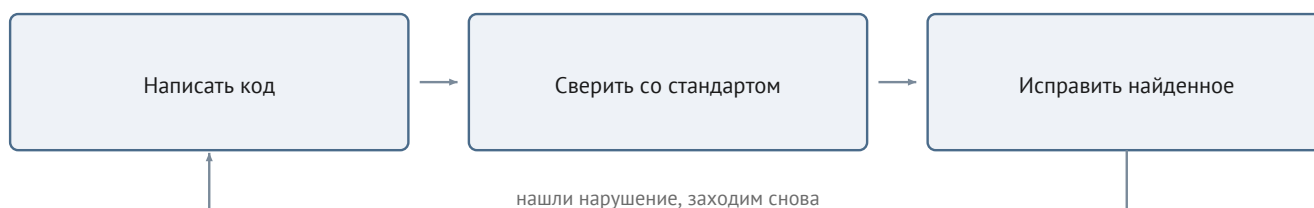
Приём против главной ловушки вайб-кодинга: код ты не читаешь, а значит и не знаешь ни хуя, что там. Вместо чтения попроси агента объяснить обычными словами, что он сделал. Откуда берутся данные, куда записываются, на основании чего принимаются решения.

Подделать пересказ невозможно, хоть тресни. Если под капотом список ключевых слов, это вылезет в первом же предложении, потому что описывать придётся именно список ключевых слов. Прочитал, сверил с замыслом, разошлось: правь сразу. Отложенная правка обойдётся в разы дороже, бяка буду.

Цикл самопроверки

Разовое "проверь себя" слабее, чем требование крутиться до чистого результата.

КАК ВЫГЛЯДИТ РАБОЧИЙ ЦИКЛ



Разница между "проверь" и "проверяй, пока не станет чисто" на практике решает всё. Первое агент сделает для галочки, второе заставит его коноёбиться до результата.

Где это не работает

У подхода есть граница, и знать её лучше заранее.

ЗАПРЕТ ПОМОГАЕТ НЕ ВСЕГДА

Позиция первая. Запрещать надо жёстко и сразу: эвристики, тихие подмены при ошибках, поддержку устаревших веток. Дальше строить агенту узкий коридор из правил, готовых сценариев, линтеров и автопроверок. Чем уже коридор, тем предсказуемее результат. Иначе придётся вечно читать код глазами, а это возвращает нас туда, откуда уходили.

Позиция вторая. Работает такой запрет только на подготовленной почве, когда разработка идёт по рельсам и модели есть куда свернуть. На открытой задаче агент, которому запретили эвристики, но не показали правильного пути, всё равно не остановится. Закрывать задачу хочется, и он выкрутится. Чем строже запрет, тем изобретательнее обход.

Вывод из спора. Запрет закрывает половину работы. Вторую половину закрывает готовый способ сделать правильно: инструмент, образец, пример решения. Без него ограничение родит не качество, а изворотливость.

Полезно помнить и встречное соображение, которое звучит как подъём, но по сути верно: языковая модель тоже эвристика, просто заметно сложнее. Граница проходит между решением, которое умеет работать со смыслом, и решением, которое умеет только сравнивать буквы. Агент ошибается не тем, что применил простое правило. Он ошибается тем, что применил его к задаче, где смысл важнее формы.

ЧТО ДЕЛАТЬ НА СВОЁМ ПРОЕКТЕ

1. Впиши в правила проекта запрет на решения по совпадению строк там, где задача про смысл. Обязательно с причиной, желательно с ценой ошибки в деньгах или времени.
2. Читай первый ответ агента на постановку. Он там сразу выдаёт, куда собирается свернуть, а править на этом этапе дешевле всего.
3. После каждого крупного куска проси пересказ обычными словами. Быстрее чтения кода и надёжнее.
4. Разошлось с замыслом, правь сразу. По накатанной агент дотащит это упрощение до самого конца проекта, и хуй его потом выкорчуешь.
5. Замени разовую ревизию циклом самопроверки по стандартам, с требованием повторять до чистого результата.

Скиллы: механика и жизненный цикл

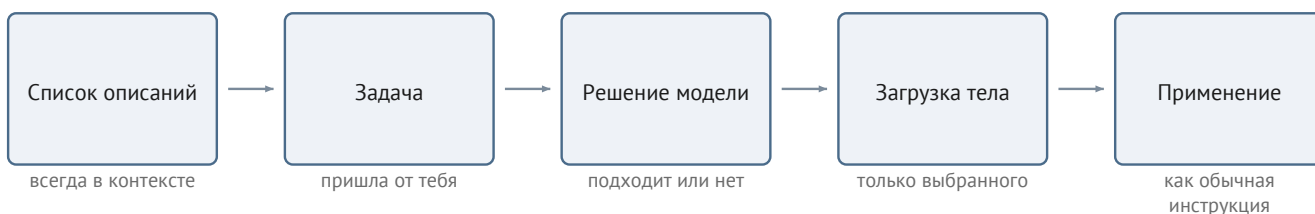
Скилл это отдельный файл с инструкцией, которую агент подгружает, когда сочтёт нужным. Сущность появилась недавно, за полгода вокруг неё выросла целая культура, и почти сразу полезли проблемы, о которых лучше знать заранее, чем потом разгребать.

Как это устроено на самом деле

Механика простая, и понимание её сразу снимает половину вопросов.

В контексте агента постоянно висят только заголовки и описания всех доступных скиллов. Не тела, только описания, обычно одна-две строки каждое. Приходит задача, модель смотрит на этот список и решает, стоит ли подгружать какой-то скилл целиком.

ЧТО ПРОИСХОДИТ СО СКИЛЛОМ



Из этой механики следуют все остальные свойства.

Скиллы платные, даже когда не используются. Заголовки висят всегда, и сотня скиллов это сотня строк на каждом ходу.

Описание работает, а тело нет. Описание сформулировано хуёво, значит тело не загрузится никогда, и качество инструкции внутри уже не имеет никакого значения.

Модель решает сама. Загрузкой ты напрямую не управляешь, влияешь только текстом описания.

Описание решает всё

Самая практичная часть главы.

Хорошее описание делает две вещи: перечисляет ситуации, в которых скилл применим, и явно называет ситуации, в которых он нахуй не нужен. Второе важнее первого, потому что именно оно спасает от ложных срабатываний.

ОПИСАНИЕ, КОТОРОЕ НЕ РАБОТАЕТ

Помогает проектировать агентов

Слишком общо: модель не понимает, когда именно это про неё, и либо не загрузит никогда, либо будет грузить постоянно.

ОПИСАНИЕ, КОТОРОЕ РАБОТАЕТ

Использовать при проектировании, реализации, ревью, отладке или оценке агента: инженерия контекста, интерфейсы инструментов, надёжность обвязки, память, оценка, координация нескольких агентов. Не использовать для обычного неагентного кода приложения.

Перечислены ситуации и явно отсечены неподходящие. Загружается тогда, когда нужно, и молчит в остальное время.

Практики, у которых скиллы срабатывают стабильно, описывают это так: не люблю вызывать скиллы руками, подбешивает, когда скилл сам не вызывается. Стабильный автовызов это признак хорошего описания, а не везения.

Скилл со скриптом или без

Спор, в котором правы обе стороны, и понимать стоит обе.

Позиция первая: скилл без скрипта бесполезен. Аргумент конкретный. Промпты живые, от проекта к проекту улучшаются. Лежали бы они в скиллах, пришлось бы каждый раз обновлять и синхронизировать между всеми машинами, а это ебля на ровном месте. Проще держать текст там, где его удобно править, а в скилл класть то, что меняться не собирается либо меняется централизованно.

Позиция вторая: скилл это методичка. А со скриптом она или без, дело десятое. Интерактивность хороша, но не везде реальна и не везде нужна. Смысл скилла в том, чтобы не повторять одно и то же десять раз с мелкими вариациями, а обратиться по названию.

КАК РЕШИТЬ У СЕБЯ

Клади в скилл то, что стабильно. Процедура, которая не меняется от проекта к проекту, знание о внутренней системе, порядок действий с чужим интерфейсом.

Не клади то, что дорабатываешь каждую неделю. Оно будет разъезжаться между машинами, и ты заебёшься синхронизировать.

Отдельный случай, где скилл незаменим. Знания, которых нет ни в репозитории, ни в обучающих данных: внутренние соглашения компании, вендорский функционал, не описанный в коде. Иначе агент про них не узнает никогда.

Пять типовых форм

Скиллы, которые встречаются в живых проектах, укладываются в несколько форм.

Форма	Что внутри	Когда нужна
Процедура	Пошаговый порядок действий	Повторяющаяся операция со стабильным ходом

Форма	Что внутри	Когда нужна
Справочник	Знание о системе или интерфейсе	Того, чего нет в коде и в обучающих данных
Обёртка над утилитой	Как вызывать инструмент и что делать с ответом	Внешние интерфейсы, где сборка запросов на лету даёт выдумки
Роль	Взгляд, с которым надо посмотреть на задачу	Ревью, критика, оценка с определённой позиции
Мета-скилл	Как делать другие скиллы или проверять их	Когда скиллов развелось до хуя

Последняя форма появляется поздно и означает ровно одно: пора наводить порядок.

Гигиена: три правила

Держи только инфраструктурные. Заголовки висят в контексте постоянно, поэтому каждый лишний скилл это налог на каждый ход. Практики, наводившие порядок, оставляли те, что описывают работу с системами, и выкидывали те, что описывают пожелания к стилю.

Считай срок годности неделями. Формулировка резкая, но проверенная: скилл, собранный генератором скиллов, наебнётся через пару недель. Меняются инструменты, меняется поведение модели, меняется проект.

Ревьюй как код. Даже аккуратный скилл после правок агентом требует проверки, и не для галочки. Люди, выкладывавшие свои скиллы, регулярно получали замечания вида хороший скилл, чистый, но агенты в него насрали.

Версии и устаревание

Проблема, у которой в экосистеме общего решения нет: как человек узнаёт, что вышла новая версия скилла. Честный ответ авторов популярных скиллов: никак, только через ленту изменений.

Два обхода, придуманные практиками.

Предполётная проверка внутри скилла. Раз в сессию агент сверяет файл скилла с источником и, если версии разошлись, одной строкой сообщает, что вышло обновление и как обновиться. Не разошлись, молчит в тряпочку.

Проверка на уровне хранилища. В корпоративном варианте лучше не зашивать проверку в каждый скилл, а повесить её на инфраструктуру: человек ставит, заливает или ищет очередной скилл, и запускается проход по всем установленным.

ПРИЗНАКИ, ЧТО СКИЛЛ ПОРА УДАЛИТЬ

1. Ты не помнишь, что он делает, и в описании этого тоже хуй поймёшь.
2. За месяц он ни разу не загрузился сам, а ты про него ни разу не вспомнил.
3. Он дублирует то, что модель делает и без него. Проверяется отключением.
4. Он лезет в задачи, к которым не относится. Описание слишком широкое.
5. Внутри он описывает интерфейс, который с тех пор поменялся.

Экономика скилла

Отрезвляющий пример, который стоит помнить перед тем, как автоматизировать очередную мелочь.

Человек хотел подписать один документ. Через готовый онлайн-редактор это заняло бы полминуты. Он попросил агента захуячить скилл для этой операции, потратил на сборку и настройку около четверти часа, а по счётчику вышло больше пяти долларов за один заход. Дальнейшие подписания стали дешёвыми, но окупаемость надо было прикидывать до, а не после.

ГЛАВНОЕ ПРО СКИЛЛЫ

Скилл окупается только на повторяющемся. Считай окупаемость до создания: сколько раз в месяц это происходит и сколько стоит один ручной проход.

И помни, что стоимость скилла сборкой не заканчивается. Он занимает место в контексте, протухает, требует ревью и однажды начинает мешать. Скилл это не разовое вложение, а подписка.

Когда любимый скилл начинает мешать

Наблюдение, которое звучит комично, но описывает реальную проблему: иногда любимый скилл начинает лезть туда, где он нахуй не сдался.

Причина всегда одна: слишком широкое описание. Лечится дописыванием строки о том, где скилл не применяется.

Отдельно стоит знать, что скилл, который пытается бороться со склонностью модели, добавляя новую яркую инструкцию, обычно усиливает перекося вместо того, чтобы его убрать. Тушишь пожар бензином. Работают не уговоры, а детерминированная проверка с ответом да или нет.

Что запомнить

В контексте висят только описания. Тело загружается по решению модели.

Описание важнее содержимого: перечисли ситуации и явно отсеки неподходящие.

В скилл клади стабильное. Живой текст держи там, где его удобно править.

Настоящая ниша скилла это знания, которых нет ни в коде, ни в обучающих данных.

Срок годности измеряется неделями. Ревьюй как код и удаляй без соплей.

И считай окупаемость до сборки: скилл это подписка, а не разовая покупка.

Хуки и детерминированные гарантии

Правило можно проигнорировать. Скилл можно не загрузить. А способ гарантировать, что шаг реально произойдёт, ровно один: сделать так, чтобы его выполняла не модель.

Это и есть хук.

ЧТО ТАКОЕ ХУК

Хук это твой код, который система запускает сама в нужный момент. Не когда модель соблаговолит, а по событию.

Событий обычно несколько: перед вызовом инструмента, после вызова, при старте сессии, при завершении задачи, перед сжатием контекста. Вешаешь на событие маленькую программу, и она отработывает всегда, без вариантов.

Слово утащили из обычного программирования, там так называют любую точку, куда можно вклиниться со своим кодом.

Три причины, по которым хуки выигрывают

Он отработывает всегда. Вот это главное. Правило это текст, который влияет на вероятности, то есть иногда работает, а иногда хуй там. Хук это код, который просто работает.

Он бесплатный. Хук контекст не жрёт вообще. Правило жрёт на каждом ходу, молча и постоянно.

Он видит то, чего модель не видит. Хук смотрит на реальное состояние системы, а не на пересказ состояния, который модель себе напридумывала.

Цифра, которая продаёт идею лучше любых уговоров, взята из практики восстановления после сжатия контекста. Инструкция вида после сжатия перечитай такие-то документы срабатывала примерно никогда. Тот же самый шаг, повешенный хуком, отработывает в восьми случаях из десяти. Разницу видно невооружённым глазом.

ГЛАВНОЕ ПРО ХУКИ

Хуки нужны там, где есть процесс. Нет процесса, значит они нахуй не нужны, и не надо себе врать.

Это не отговорка, а рабочий критерий. Не можешь назвать событие, после которого должен произойти шаг, значит тебе нужен не хук, а внятная постановка задачи. Хук поверх каши даст кашу по расписанию.

Что на них вешают

Список того, что реально висит у людей в проде.

Событие	Что делает хук	Что закрывает
Перед вызовом инструмента	Запрещает поиск по тексту там, где есть индекс	Забывчивость модели
Перед вызовом инструмента	Блокирует чтение файлов с секретами	Обход правила
Старт сессии	Обновляет индексы, подтягивает контекст проекта	Работу по протухшим данным
После сжатия	Восстанавливает состояние и перечитывает ключевое	Переделывание сделанного
Отправка изменений	Гоняет проверку на опасные паттерны	Упрощения, вросшие в код
Завершение задачи	Пишет отчёт, шлёт уведомление	Молчаливый отказ

Механика ошибки тут важнее самого запрета, и на этом спотыкаются все подряд. Скрипт не должен просто говорить нельзя. Он должен вернуть текст ошибки прямо в сессию: вот что не так, вот почему. Модель это видит, понимает причину и меняет поведение. А молчаливый отказ приводит к тому, что агент коноёбится вслепую и с третьей попытки изобретает обход.

Хук вместо простыни в правилах

Отдельный случай, который стоит выделить. Инструкции вида после X сделай Y работают ровно до тех пор, пока цепочка линейная и короткая. Как только это превращается в граф, инструкция разрастается в невъебенную простыню, и модель применяет её через раз, по настроению.

Хук здесь выигрывает дважды: контекст не занимает и не зависит от того, вспомнила модель про правило или забила. А совсем пиздато выходит, когда X это внутреннее событие системы, о котором модель вообще ни сном ни духом.

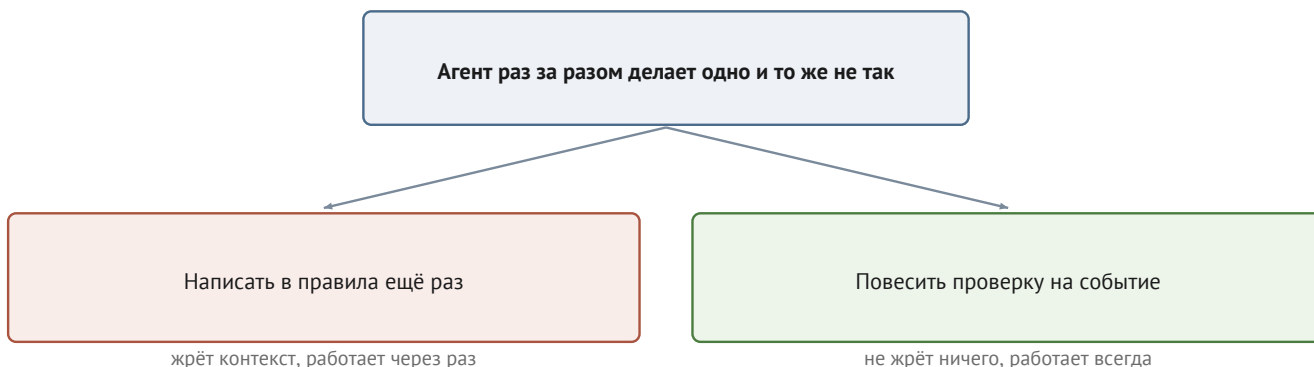
Лишние ветвления как повод для хука

Классическая болезнь агентского кода: защитные проверки на всякий случай.

Арифметика тут простая и невесёлая. Каждый лишний if удваивает число путей через код: три штуки это восемь комбинаций, четыре шестнадцать, пять уже тридцать две. И каждую кто-то должен протестировать, а агенты, сука, обожают прятать баги ровно в этих ветках на всякий случай.

Ответ, к которому пришли практики: проверка на коммит, которая ищет опасные паттерны. Не вежливая просьба не делай так, а автоматический проход, который просто не пропускает, и хоть ты тресни.

КАК ЗАКРЫВАТЬ ПОВТОРЯЮЩУЮСЯ ОШИБКУ



Когда хуки нахуй не нужны

Честная противоположная позиция, её высказывали не раз и по делу.

Современные модели держат правила заметно лучше, чем год назад. Люди, которые раньше вешали хуки на перечитывание документов после сжатия, сейчас без них живут спокойно, потому что проблема рассосалась сама.

И мысль пошире: нет процесса, значит хуки будут ритуалом. Навесишь проверок на события, которые ни о чём не говорят, и получишь торможение без единого грамма выигрыша. Красиво, дорого, бесполезно.

СКОЛЬКО АВТОМАТИКИ НУЖНО

Минималист. Модели умные, обвязка устаревает быстрее, чем ты её достраиваешь. Раз в квартал отключай и смотри, стало ли без твоих хуков хуже. Половина окажется мёртвым грузом.

Перестраховщик. По контексту хуки бесплатные, а цена пропущенного шага в проде такая, что обсуждать нечего. Вешай на всё, что критично, и спи спокойно.

Разделяющий критерий. Обратимость. Шаг откатывается за минуту, живи без хука. Не откачивается ни хуя, ставь.

Память вместо хука не катит

Соблазн понятный: зачем код, если можно просто сказать агенту запомни, что после такого-то шага надо делать то-то.

Ограничение, о котором лучше знать заранее, чем выяснять на живом проекте. Память объёмом больше пятнадцати-двадцати тысяч символов начинает сыпаться: агент помнит хуже, а обновление одного несчастного факта требует переписывания здорового куска текста.

Память для того, что меняется и что человек говорит своими словами. Хуки для того, что обязано отработать независимо от того, вспомнил кто-то или нет.

Правило, скилл или хук

Сводка, по которой удобно раскидывать каждое новое требование.

Если это	Кладёшь в
Нужно знать всегда	Правила
Нужно знать в конкретной ситуации	Скилл с описанием ситуации
Должно отработать обязательно	Хук
Меняется от разговора к разговору	Память
Требует надёжного формата	Код обвязки, а не текст

ТВОЙ ПЕРВЫЙ ХУК ЗА ПЯТНАДЦАТЬ МИНУТ

1. Вспомни ошибку, которую агент повторил больше трёх раз.
2. Назови событие, после которого её видно: вызов инструмента, коммит, конец задачи.
3. Напиши проверку, которая отвечает да или нет. Никаких оценок по шкале, это не конкурс красоты.
4. Верни текст ошибки в сессию, с объяснением причины, а не сухой отказ.
5. Убери из правил строку, которая пыталась закрыть это словами.

Пятый шаг обязательный, и его вечно проёбывают. Иначе получишь и хук, и мёртвую строку в правилах, которая продолжает жрать контекст до скончания веков.

Что запомнить

Хук отличается от правила ровно одним: он отрабатывает всегда. И контекст не жрёт.

Хуки нужны там, где есть процесс. Событие должно называться словами, иначе это ритуал.

Ошибку возвращай в сессию с объяснением, а не молча. Молча она породит обход.

Восстановление после сжатия хуком работает в восьми случаях из десяти, инструкцией примерно никогда.

Разделяющий критерий между хуком и правилом это обратимость шага.

И после установки хука вычищай из правил строку, которая делала то же самое словами. Две реализации одного требования разъедутся, бля буду.

Инструменты и когда обязанка вредит

Три предыдущие главы учили строить. Эта учит расхуживать, и она едва ли не важнее.

Инструмент это контракт

Сначала короткая часть про инструменты, она примыкает по смыслу.

Инструмент, отданный агенту, описывается не функцией, а контрактом. В контракте шесть пунктов, и каждый однажды выстрелит тебе в ногу, если его не назвать.

Пункт	Вопрос, на который отвечает
Схема входа	Что можно передать и в каком виде
Предусловия	Когда вызывать нельзя
Побочные эффекты	Что изменится в мире
Таймаут	Сколько ждать и что считать отказом
Идемпотентность	Что будет при повторном вызове
Права	Кому и в каком режиме доступен

Отдельно про интерфейс. Захотелось добавить в инструкцию пример использования инструмента, значит часто правильное поменять сам инструмент.

Причина в том, что примеры сужают пространство решений: модель начинает тупо копировать показанный сценарий вместо того, чтобы придумать свой. Вместо примера лучше сделать параметр выразительнее: не свободная строка, а список допустимых значений.

ГЛАВНОЕ ПРО ИНСТРУМЕНТЫ

Внешние интерфейсы оборачивай утилитами. Сборка запросов на лету даёт устойчивый фон выдумок и тестируется через жопу.

Разбор реального продакшн-агента показал ровно это: пока агент собирал обращения к пяти внешним сервисам сам, стоял постоянный фон галлюцинаций. Завернули каждое обращение в утилиту с внятным интерфейсом, и проблема отъебалась, а тестирование упростилось.

Главный вред обязанности это не объём

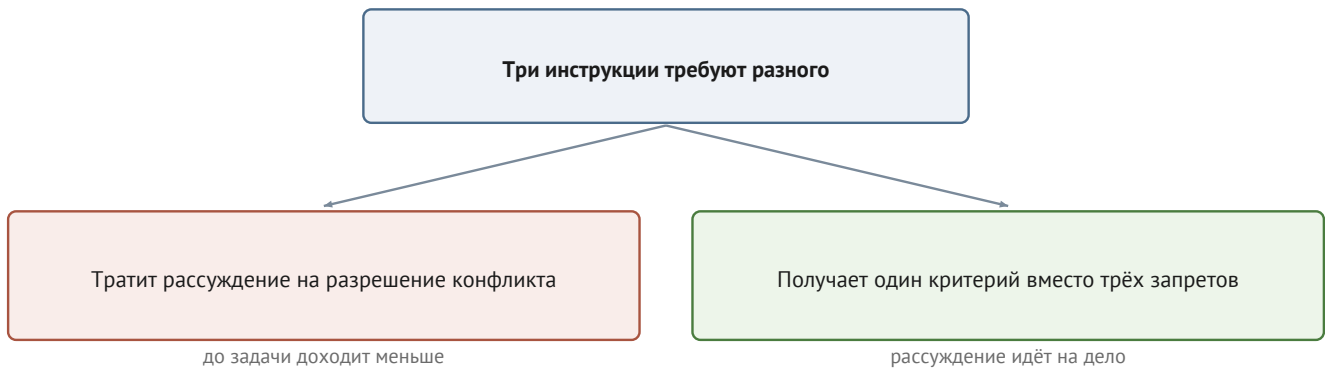
Теперь про снос.

Живёт убеждение, что чем больше инструкций, тем предсказуемее агент. Разбор, который сделали у одного из поставщиков на собственном продукте, показал ровно обратное.

Они выкинули больше восьмидесяти процентов системных инструкций и не получили ни грамма измеримой просадки на задачах программирования. А корень проблемы нашли в своих же записях работы агента: в одном запросе жили указания, которые друг другу прямо противоречили. Системная инструкция требует не писать комментарии, скилл говорит дер-

жать документацию там, где уместно, пользователь хочет третье.

ЧТО ДЕЛАЕТ МОДЕЛЬ С ПРОТИВОРЕЧИЕМ



Отсюда шесть переходов, которые стоит держать в голове при каждом добавлении в обязанку.

Правила в суждение. Вместо простыни запретов дай критерий, по которому модель решает сама. Например: пиши код, который читается как окружающий, повторяй его плотность комментариев, именование и идиомы.

Примеры в интерфейсе. Пример сужает поиск. Выразительный параметр не сужает.

Всё сразу в постепенное раскрытие. Ревью и верификация выносятся в скиллы по требованию. Файл правил становится деревом файлов, а не единой свалкой.

Повторы в описания инструментов. Поведение можно описать там, значит в правила оно не идёт.

Память в правилах в автоматическую память. Разное живёт по-разному.

Простые спецификации в богатые референсы. Спецификацией может быть набор тестов, функция из другого проекта, макет интерфейса. Рубрика оценки тоже референс.

Раз в квартал сноси всё

Приём, который независимо повторили несколько человек, включая тех, кто эти инструменты делает.

Раз в несколько месяцев отключай всё нахуй: скиллы, файл правил, хуки. Поработай так какое-то время. Возвращай по одной строке, и только те, на которых агент спотыкается повторно.

Люди, которые это делали, описывают результат одинаково: убрал десятки скиллов, выключил хуки, ужал правила в несколько раз, разницы не заметил ни хуя.

РЕВИЗИЯ ОБЯЗКИ

1. Сложи всё, что накопил, в отдельную папку и отключи.
2. Поработай неделю. Записывай каждый случай, когда пришлось объяснять руками.
3. Верни только то, что встретилось два раза и больше.
4. Возвращая, спроси себя, где этому место: правило, скилл, хук или описание инструмента.
5. Повторяй раз в квартал. Это не разовая уборка.

Постоянно живут только проверки. Инструкции протухают быстрее эвалов, и это нормально: инструкция описывает поведение конкретной модели в конкретный месяц, не более того.

Контраргумент, который нельзя игнорировать

Честность требует привести и обратную позицию, звучавшую не раз.

Обязка даёт неспециалисту иллюзию корректности. Человек, который сам оценить результат не может, видит зелёные проверки, аккуратные отчёты и уверенный тон и делает вывод, что всё заебись. А обязанка проверяла то, что легко проверить, а не то, что важно. Разница огромная.

Мысль стыкуется с более общей: модели обучены на корректности, которую можно проверить за секунды. Тест прошёл проверяется мгновенно, фича работает за минуты, регрессия не всплыла за дни, а вот выдержала ли архитектура изменение требований, выясняется через месяцы. Обучающего сигнала на нижних строках этой таблицы нет ни у модели, ни у твоей обязанности.

Что проверяем	За сколько видно
Тест прошёл	Секунды
Функция работает	Минуты
Регрессия не всплыла	Дни
Архитектура выдержала изменение требований	Месяцы
Систему всё ещё можно развивать	Годы

ОБЯЗКА ЭТО АКТИВ ИЛИ ОБЯЗАТЕЛЬСТВО

Актив. Она копит опыт команды и не даёт наступать на те же грабли. Без неё каждый новый человек проходит тот же путь заново.

Обязательство. У неё есть стоимость сопровождения и срок годности. Она протухает быстрее, чем достраивается, и однажды начинает мешать больше, чем помогать.

Как жить с обоими. Считай обязанку обязательством и пересматривай по расписанию. Тогда всё, что переживёт пересмотр, действительно окажется активом.

Противоположная крайность

Чтобы картина была полной, стоит знать и радикальный ответ, который у практиков тоже работает.

Формулировка, положенная прямо в файл правил, звучала примерно так. Работаем короткими итерациями прямо в целевой ветке. Пул-реквесты, автоматические проверки и прочая процессная возня не нужны, система контроля версий нужна для разницы, истории и восстановления. Сначала воспроизвели ошибку, потом починили, потом проверили локально. В тесты сажаем шум: плохую сборку, глупые действия пользователя, мусор. Команда это я и ты, работаем на скорость без ущерба эффективности.

Автор объясняет так: модели уже достаточно умные, и боевой тест лучше, чем подкладывать вату со всех сторон. После этой инструкции стало быстрее и пиздатее.

Это не совет всем. Это напоминание, что тяжёлый процесс не единственный ответ, и что выбор между ними определяется ценой ошибки, а не модой.

Что запомнить

Инструмент это контракт из шести пунктов. Неназванный пункт однажды выстрелит.

Внешние интерфейсы заворачивай в утилиты: сборка запросов на лету даёт выдумки.

Главный вред обвязки не объём, а противоречия. Модель тратит рассуждение на их разгребание вместо задачи.

Давай критерий вместо списка запретов. Критерий переносится на новые ситуации.

Раз в квартал отключай всё и возвращай только то, на чём спотыкаешься повторно.

Постоянно живут только проверки. Инструкции стареют.

И держи в голове, что обвязка проверяет то, что легко проверить. Долгую сопровождаемость она не измеряет вообще ни разу.

Модуль 4. Контекст и память

Четыре главы про главный ресурс.

Контекст это то, за что ты платишь и чем определяется качество ответа. Здесь: как он устроен и сколько стоит, что творится при сжатии и что обязано его пережить, чем отличаются три подхода к памяти и как устроен поиск по коду и по документам.

Модуль технический, но без него счета так и останутся загадкой природы, а деньги будут утекать в никуда.

Контекст как единственный ресурс

Если из всей книги надо запомнить одну-единственную вещь, вот она: контекст это то, за что ты платишь, и то, чем определяется качество ответа. Не модель, не инструменты, не длина инструкций ни хуя. Контекст.

Что это такое

Контекст это весь текст, который уходит модели на каждом круге. Правила проекта, описания доступных инструментов, история разговора, результаты вызовов, твоя последняя фраза.

Два свойства делают его центральной темой.

Он оплачивается целиком каждый раз. Не прирост, а весь накопленный объём. Поэтому пятидесятый ход разговора стоит в разы дороже первого, даже если ты написал одно слово. Обидно, но так устроено.

Он ограничен и работает неравномерно. Есть предел, за который хуй впихнёшь. И внутри предела внимание распределяется неровно: начало и конец модель использует лучше, чем середину.

ПОЧЕМУ РАЗГОВОР ДОРОЖАЕТ

Модель не помнит ни хуя. Чтобы она знала, о чём вы говорили десять минут назад, ей это надо пересказать заново.

Обвязка так и делает: собирает всю переписку и отправляет целиком. На пятидесятом ходу уезжает всё, что накопилось за пятьдесят ходов.

Отсюда простое следствие: длинный разговор это не немного дороже короткого, а сильно дороже. И отсюда же вся дальнейшая инженерия.

Шаблон сборки

Контекст собирает не модель, а обвязка. Полезно представлять, из чего он складывается, в порядке попадания в запрос.

Блок	Что в нём	Меняется
Системные инструкции	Кто агент и как себя ведёт	Редко
Описания скиллов	Заголовки всех доступных	При добавлении скилла
Схемы инструментов	Что можно вызвать и как	При изменении набора
Правила проекта	Инварианты и ловушки	Редко
Цель	Что делаем сейчас	На каждой задаче
История	Предыдущие ходы	Постоянно

Блок	Что в нём	Меняется
Отчёты субагентов	Что они вернули	При делегировании
План и статус	Что сделано, что осталось	Постоянно
Текущий ход	Твоя фраза и последние результаты	Каждый ход

Тот, кто собирает этот текст, и называется контекстным движком. Важное уточнение, прозвучавшее в спорах: память и поиск по документам это источники, а не движок. Движок это то, что решает, что из источников попадёт внутрь и в каком порядке.

Сборщик или движок

Спор, который стоит знать, потому что за словом движок продают чёрт знает что.

Одна позиция: движок это просто заполнение шаблона, и называть эту хуёвину движком слишком громко.

Вторая: в серьёзной сборке никакого шаблона нет, там компиляция журнала событий и кучи источников на лету, при каждом изменении. Простой подстановкой не отделаешься, если делать хорошо.

ЗА ЧТО ТЫ ПЛАТИШЬ, ПОКУПАЯ КОНТЕКСТНЫЙ ДВИЖОК

Скептик. Часто под этим продают обёртку над индексом кода, чтобы одно и то же не переизобреталось. Тот же результат даёт субагент, делающий несколько поисков, только там дороже.

Практик. Ценность в другом: модель получает готовый нужный кусок и не тратит ходы на его поиск. На больших базах это разница между работой и мучением.

Экономический аргумент против. Фронтир-модели на таких инструментах никто не дообучает, поэтому выигрыш в основном в токенах. Расход не ограничен, значит окупается хуёво.

Экономический аргумент за. В компаниях расход как раз ограничен, и там контроль токенов становится отдельной темой.

Практичный компромисс, который называли: для навигации по коду хватает небольшой локальной модели, можно каскадом. Гонять фронтир-модель по файлам это самая жирная и самая тупая часть счёта.

Отдать целиком или собирать вызовами

Вопрос, который встаёт на каждом проекте.

Аргумент за целиком: небольшой проект дешевле положить в контекст сразу, чем гонять агента собирать его десятком вызовов. Он сразу увидит связи, а ты не заплатишь за его беготню.

Аргументы против, все существенные:

Так модель выделяет меньше вычислительного бюджета на осмысление каждой части.

После определённого объёма идёт размытие и заметная просадка качества, причём по ощущениям практиков всё начинает сыпаться сильно раньше формального предела.

В проекте, который живёт дольше одного захода, в документы неизбежно набивается мусор, даже если следить в четыре глаза.

ГЛАВНОЕ ПРО ОБЪЁМ

Материалы друг другу не противоречат, значит лучше дать чуть больше, чем чуть меньше: дочитывать и переделывать дороже.

Материалы противоречат, значит лишнее вреднее нехватки. Модель просрёт рассуждение на разрешение конфликта.

Поэтому чистка контекста это в первую очередь устранение противоречий, и только во вторую сокращение объёма.

Кеш: экономика, которая решает больше выбора модели

Техническая деталь, которая на счетах видна сильнее всего.

Поставщики кешируют начало запроса. Следующий запрос начинается точно так же, значит эта часть считается дешевле. Отсюда практическое правило: стабильное держи в начале, изменчивое в конце.

Нарушение обходится дорого. Воткнёшь в начало текущее время, случайный идентификатор или что-то меняющееся, и кеш рухнет целиком, а каждый ход оплачивается по полной.

Отдельная засада: кеш ключей и значений устроен линейно. Из-за этого самая хитровыбаванная схема сборки контекста может выйти в разы дороже наивной, просто потому что мажет мимо кеша.

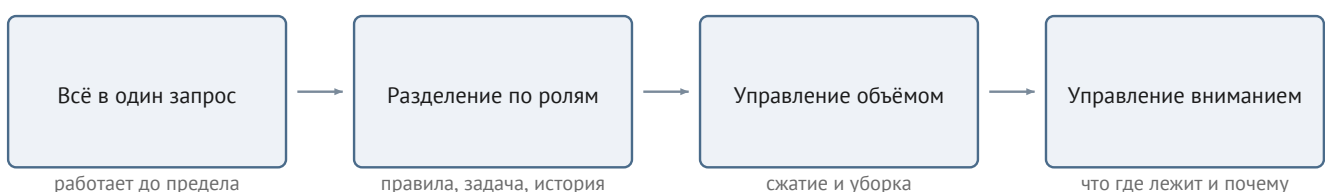
ПРОВЕРКА КЕША ЗА ПЯТЬ МИНУТ

1. Посмотри на первые строки запроса, который уходит модели.
2. Найди там всё, что меняется от хода к ходу: время, идентификаторы, счётчики.
3. Перенеси это в конец.
4. Сравни долю попаданий в кеш до и после. У правильно настроенного агента она доходит до девяноста процентов.

Стадии инженерии контекста

Полезно понимать, на какой стадии ты находишься. Люди проходят их примерно в одном порядке.

СТАДИИ



Первая стадия: всё валится в один большой запрос. Работает заебись, пока задача влезает.

Вторая: появляется разделение на постоянное и переменное, правила отделяются от задачи.

Третья: появляется управление объёмом. Сжатие, вычистка, ограничение истории.

Четвёртая: появляется управление вниманием. Ты начинаешь думать не о том, сколько текста, а о том, где он лежит и что модель прочитает внимательно.

Большинство застревает на второй и потом причитает над счетами, будто их кто-то обманул.

Что запомнить

Контекст оплачивается целиком каждый ход. Длинный разговор дорожает нелинейно.

Внимание распределяется неравномерно: начало и конец лучше середины.

Контекстный движок это то, что решает, что положить. Память и поиск это источники.

Непротиворечивое лучше дать с запасом. Противоречивое надо убирать, а не сокращать.

Стабильное в начало, изменчивое в конец: иначе кеш наебнётся и каждый ход платится целиком.

И самая дорогая ошибка это гонять сильную модель по файлам вместо работы.

Сжатие и что должно его пережить

Рано или поздно разговор упирается в потолок. Тогда система делает сжатие: берёт всю накопленную историю и подменяет её кратким пересказом.

Момент критический, потому что после него агент частенько начинает вести себя как хуй знает кто. Разберём, с чего вдруг.

Теряются не факты, а их статус

Главное наблюдение, объясняющее всё остальное.

Сжатие обычно неплохо сохраняет факты: что за проект, какие файлы, что решили. А теряет другое, и самое важное: что из этого уже сделано, что отменено нахуй, что было гипотезой, а что стало решением.

Симптомы после сжатия узнаются с полпинка. Агент заново берётся за работу, законченную полчаса назад. Отвечает на вопросы, которые давно сняты. Тащит обратно вариант, который вы вместе отвергли и закопали.

ГЛАВНОЕ ПРО СЖАТИЕ

Сжатие теряет не факты, а их статус. Поэтому автоматический пересказ надо дополнять своим собственным, где прямо сказано, что сделано, что отменено и что решено окончательно.

Пережить сжатие должно то, что дорого восстанавливать, а не то, что легко перечитать. Список файлов агент прочитает заново за секунды. Причину, по которой вы отказались от очевидного решения, он не восстановит никогда.

Что писать в пересказ

Хороший пересказ перед сжатием отвечает на четыре вопроса, а автоматический не отвечает почти никогда.

Вопрос	Пример строки
Что уже сделано и проверено	Первые три пункта плана закрыты, тесты проходят
Что отменено и почему	От очереди отказались: она не решает проблему повторов
Что решено окончательно	Формат хранения выбран, к обсуждению не возвращаемся
Что остаётся открытым	Не решено, как обрабатывать пустой ответ внешнего сервиса

Вторая строка самая ценная. Отменённые варианты без указанной причины воскресают как зомби, потому что для модели они выглядят вполне разумными.

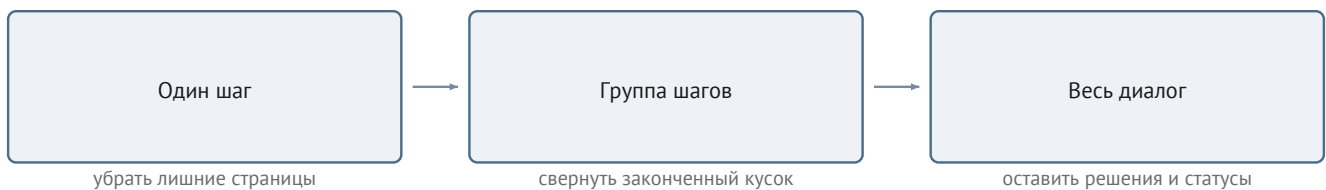
Хук, а не надежда

Инструкция вида перед сжатием напиши хороший пересказ работает через жопу, потому что момент сжатия модель не контролирует ни хуя.

Работает другое: повесить пересказ хуком на событие сжатия. Тогда он формируется всегда, а не когда вспомнилось.

Практики, которые заебались от переделывания сделанного, решали задачу именно так: хук перед сжатием, который кладёт крупное собственное резюме, а не полагается на встроенный механизм.

ТРИ УРОВНЯ СЖАТИЯ



Фрактальные мини-сжатия

Идея, которая пока не стала общей практикой, но объясняет направление мысли.

Сжимать можно не только весь диалог. Можно сжимать отдельный шаг и группу шагов.

Пример, на котором всё понятно. Агент листает результаты постранично: прочитал первую страницу, вторую, третью, нашёл нужное на четвёртой. В контексте висят все четыре, хотя нужна только четвёртая. Мини-сжатие выкидывает три лишние нахуй, оставляя одну строку о том, что поиск занял четыре страницы.

Автор идеи описывал схему с тремя точками отсчёта: до первой ничего не меняется, между первой и второй можно сжимать при большом разрыве, после второй дописанное потом заменяется результатом.

Практическая ценность здесь не в конкретной схеме, а в самой мысли: сжатие это не одно событие в конце, а постоянная работа.

Порог сжатия это настройка

Деталь, о которой узнают поздно: у большинства инструментов есть параметр, задающий, при каком объёме наступает автоматическое сжатие.

Значение по умолчанию выбрано поставщиком под средний случай. А твой случай средним не является ни хуя.

ГДЕ СТАВИТЬ ПОРОГ

Раньше. Сжатие происходит чаще, каждый раз теряется меньше, диалог остаётся дешевле. Годится для длинных прогонов и рутины.

Позже. Сжатие происходит реже, модель дольше видит всю картину целиком. Годится для задач, где важны связи между далёкими частями.

Признак, что порог не тот. После каждого сжатия приходится объяснять заново, ставь раньше и пиши свой пересказ. Модель начинает терять связи внутри одной задачи, ставь позже.

Что делать после сжатия

Отдельный чек-лист, потому что первые ходы после сжатия самые рискованные.

ПЕРВЫЕ ДЕЙСТВИЯ ПОСЛЕ СЖАТИЯ

1. Спроси, что агент считает сделанным. Сверь со своим представлением.
2. Спроси, какие решения он считает окончательными. Здесь чаще всего всплывают воскресшие отменённые варианты.
3. Попроси перечитать ключевые документы: спецификацию и файл прогресса.
4. Не запускай длинную задачу сразу после сжатия. Сначала один короткий ход, чтобы увидеть состояние.

Третий пункт отлично автоматизируется хуком, и именно этот хук в замерах давал восьмидесятипроцентную надёжность против почти нулевой у той же инструкции текстом.

Файл прогресса как страховка

Самый дешёвый способ пережить сжатие вообще без всякой магии: вести файл прогресса на диске.

Агент пишет туда, что сделал, что заблокировано и что пропустил. Файл живёт вне контекста, сжатие его не трогает, и после любого обрыва состояние поднимается чтением одного файла. Дёшево и сердито.

Приём стоит завести до того, как ты упрёшься в сжатие. Он же пригодится при автономных прогонах, о которых отдельный модуль.

Разные инструменты сжимают по-разному

Практическое наблюдение, которое стоит проверить у себя: качество автоматического сжатия сильно гуляет между инструментами и меняется от версии к версии.

Люди отмечали и то, что в каком-то инструменте сжатие настолько охуенное, что многодневная сессия живёт без потерь, и то, что через месяц у того же инструмента оно стало откровенно хуёвым.

Вывод простой: на репутацию не полагайся. Проверь на своей задаче, задав после сжатия три вопроса из чек-листа выше.

Что запомнить

Сжатие теряет статус фактов, а не факты. Отсюда и переделывание сделанного.

Пережить должно то, что дорого восстанавливать: решения, отмены, причины. Списки файлов агент перечитает сам.

Пересказ пиши сам и вешай хуком, а не на авось.

Сжимать можно не только весь диалог, но и отдельные шаги.

Порог сжатия это настройка, и значение по умолчанию не про тебя.

Файл прогресса на диске переживает любое сжатие и любой обрыв. Заведи его заранее.

Память: файлы, векторы, нити

Память это то, что должно сохраниться, когда разговор кончится. Подходов три, и решают они разные задачи. Путаница между ними обходится в круглую сумму, поэтому разберём каждый по отдельности.

Первый подход: файлы

Самый простой и самый ходовой. Агент пишет заметки в текстовые файлы, они лежат на диске, при надобности он их читает.

Достоинства очевидны: всё видно глазами, всё правится руками, всё версионруется обычными средствами, ломаться там нечему.

Красивый приём поверх файловой памяти: ночная свёртка. Дневные заметки раз в сутки сворачиваются в недельные, недельные в месячные, месячные в годовые. Верхние уровни короткие, нижние подробные, и глубина уходит по мере старения. Работает пиздатно.

Недостаток тоже очевиден, и сформулировали его точно: как организация постоянного хранилища это охуенная классика, а вот как эффективно доставать нужное из кучи файлов, хуй просышь.

Пока файлов десятков, агент их читает. Когда их триста, он читает не те, и хуй ты угадаешь какие.

Второй подход: векторный поиск

Здесь заметки не просто лежат, а превращаются в числовые представления, по которым можно искать похожее по смыслу.

ЧТО ТАКОЕ ПОИСК ПО СМЫСЛУ

Обычный поиск ищет совпадение букв. Запрос про машину не найдёт заметку про автомобиль.

Поиск по смыслу устроен иначе. Каждый кусок текста превращают в набор чисел так, что близкие по смыслу тексты получают близкие наборы. Дальше поиск это сравнение чисел, а не букв.

Работает это хорошо ровно до тех пор, пока близость чисел совпадает с близостью смысла. А совпадает она не всегда, и в этом вся сложность.

Ключевая деталь реализации, которая отличает работающую схему от неработающей: подмешивать найденное надо до вызова модели, а не давать модели инструмент поиска.

В первом случае бэкенд получает вопрос, ищет по нему совпадения и кладёт их в контекст ещё до того, как модель начнёт формулировать ответ. Модели решать нечего, факт уже под рукой. Цена этому одна-две секунды на работу поисковой модели.

Во втором случае модель должна догадаться вызвать поиск, сформулировать запрос и дожидаться результата. Догадывается она через раз, а то и реже.

Болезни векторной памяти

Противоречащие факты. Классический пример: человек работал в одной компании, перешёл в другую. Оба факта в памяти, оба похожи на запрос, и выдаются они как бог на душу положит.

Частичное лечение: хранить дату и давать знаниям вес по времени. Полное: фоновая уборка, о которой ниже.

Порог близости. Проблема, у которой красивого решения нет. Поставишь порог низко, и в контекст летит лишнее, близкое по числам и далёкое по смыслу. Поставишь высоко, и нужное перестает долетать.

Практик, который с этим бодался, перебрал всё что можно и остановился на честном признании: любой фиксированный порог одновременно слишком строгий и слишком мягкий, и хоть ты тресни. Остаётся либо пропускать всё и переранжировать вторым проходом, либо тратить дополнительный вызов модели на отсев.

Свалка. Без уборки память за месяцы превращается в помойку из несвязных утверждений, и поиск по ней перестаёт значить хоть что-нибудь.

ГЛАВНОЕ ПРО УБОРКУ ПАМЯТИ

Память требует регулярной свёртки, и делать её лучше в фоне, а не посреди разговора.

Схема, которая работает: ночью отдельный процесс перечитывает накопленное, объединяет повторы, помечает устаревшее, связывает разрозненное. Днём агент пользуется прибранным.

Без этого шага деградирует любая память, файлы это или векторы, без разницы.

Третий подход: нити

Самая свежая идея из корпуса и, пожалуй, самая полезная.

Нить это что-то между памятью и скиллом. Разговор касается долгого процесса, и модель заводит нить и тянет её во всех последующих обсуждениях этой темы, не теряя нить в буквальном смысле.

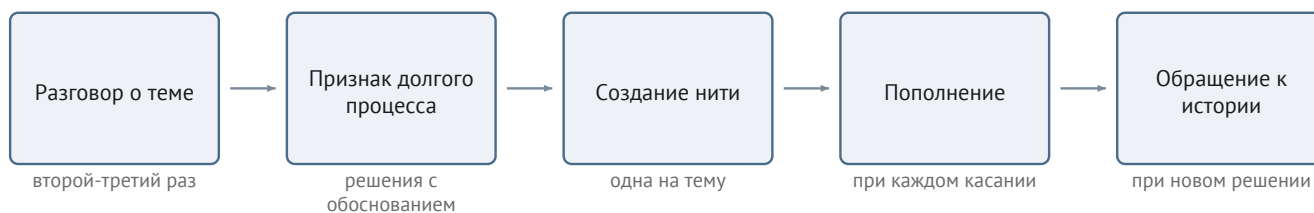
Внутри нити не отдельные факты, а набор событий и обоснований, привязанный к теме и времени.

Примеры, на которых разница видна сразу.

Обычный факт	Нить
У пользователя болит колено	История: когда началось, что пробовали, от чего отказались и почему, что сейчас работает
Инвестиционный профиль консервативный	Хронология решений: что покупали, из каких соображений, что оказалось ошибкой
Проект использует такую-то базу	Почему её выбрали, какие были альтернативы, при каких условиях решение пересмотрят

Ценность в правой колонке одна: агент может опереться на то, почему решение было принято раньше. Без этого он раз за разом тащит то, что ты уже отверг, и объясняй ему каждый раз заново, как первокласснику.

КАК НИТЬ ПОЯВЛЯЕТСЯ



Практик, который это сделал, отмечал, что агент за одно-два сообщения понимает, когда пора завести новую нить, если в инструкции описан признак: тема, к которой мы возвращаемся и по которой принимаются решения.

Ограничения, о которых надо знать

Объём. Память больше пятнадцати-двадцати тысяч символов начинает сыпаться. Агент помнит хуже, а обновление одного факта требует переписывания здорового куска. Память это не архив.

Приватность. Групповая память переживает уход человека из группы. Бот остался, значит он продолжит собирать сведения, когда человека упоминают. Это не злой умысел, а естественное следствие устройства, и решается оно архитектурой, а не обещаниями.

Стоимость. Честный проёб, который стоит знать. В одной реализации маршрутизацией каждого сообщения занимался отдельный фоновый агент: решал, куда его положить. Инженерно красиво, экономически бессмысленно, потому что расход удваивается. Автор сам признал и переделал, без соплей.

КАК ВЫБРАТЬ ПОДХОД

1. Нужно, чтобы человек видел и правил руками? Файлы.
2. Записей больше сотни и надо доставать по смыслу? Векторы поверх файлов.
3. Есть темы, где важна история решений, а не факты? Нити.
4. В любом случае: фоновая свёртка и ограничение объёма.
5. И реши заранее, что происходит с памятью о человеке, когда он уходит.

Контекст и память это разные вещи

Напоминание, потому что путают постоянно.

Контекст это то, что модель видит прямо сейчас. Он платный, ограниченный и живёт один разговор.

Память это то, откуда в контекст попадает нужное. Она бесплатна в момент хранения и платна только тогда, когда что-то из неё подставили.

Ошибка, которая дорого обходится: складывать в правила проекта то, чему место в памяти. Правила висят в контексте всегда и жрут постоянно, а память подставляется по надобности.

Что запомнить

Файлы хороши как хранилище и хуёвы как поиск. Векторы решают вторую задачу и не решают первую.

Подмешивай найденное до вызова модели, а не давай инструмент поиска.

Память без времени пиздит: противоречащие факты будут выдаваться как попало.

Любой фиксированный порог близости неверен. Либо переранжирование, либо дополнительный отсев.

Нити хранят не факты, а историю решений с обоснованиями. Это то, чего не восстановить.

Уборка в фоне обязательна. Без неё деградирует любая память.

И реши заранее, что происходит с памятью о человеке, когда он ушёл.

Поиск по коду и по документам

Агент, работающий с реальным проектом, большую часть ходов просаживает не на решение задачи, а на поиск: где лежит нужное, что с чем связано, что уже сделано. От устройства поиска зависят и счёт, и качество.

Глава про два разных поиска, которые постоянно путают: по коду и по документам.

Поиск по коду

Есть три способа объяснить агенту, что где лежит.

Обычный текстовый поиск. Ищет строку. Дёшево, работает везде, не понимает ни хуя. На вопрос, где вызывается эта функция, вываливает всё подряд, где встретилось её имя, включая комментарии и похожие названия. Разгребай сам.

Языковой сервер. Тот же механизм, что подсказывает переходы в редакторе. Понимает язык, знает, где определение и где использования. Точный, но требует настройки и не для всех языков живёт одинаково хорошо: где-то заебись, где-то хуй проссышь.

Индекс по синтаксическому дереву. Разбирает код в структуру и позволяет искать по конструкциям: все обработчики маршрутов, все вызовы такого-то метода, все места, где перехватывается ошибка.

ЧТО ВЫБИРАТЬ

За языковой сервер. Он точен, потому что понимает язык по-настоящему. Настроен у тебя, значит дополнительные индексы могут и не понадобиться.

За синтаксический индекс. Он не зависит от окружения, работает на разнородных базах и отвечает быстрее. На больших монорепозиториях разница во времени ответа становится решающей.

Что показало сравнение. Люди, которые не спорили, а замеры, получили разницу во времени в десятки раз при близком качестве ответов. То есть выбор чаще про скорость и удобство, чем про правильность.

Практический совет, который повторяли: не гоняй фронтир-модель по навигации. Для поиска по коду хватает небольшой локальной модели, можно каскадом: сначала дешёвая сужает область, потом дорогая работает по сути. Хождение по файлам самой сильной моделью это самая жирная строка в счёте.

И ещё один приём, который стоит копейки: есть индекс, запрети текстовый поиск хуком. В правилах написать это можно, но модель на них положит хуй.

Поиск по документам

Здесь сложнее, и здесь же живёт главный спор корпуса.

ЧТО ТАКОЕ ПОИСК С ПОДСТАНОВКОЙ

Идея простая. У тебя куча документов, которые в контекст не влезают. Ты режешь их на куски, для каждого куска считаешь числовое представление и складываешь в базу.

Приходит вопрос, ты считаешь представление вопроса, находишь похожие куски и подставляешь их в запрос к модели.

Подход называют поиском с подстановкой. Он появился, когда контекст был маленьким, и с тех пор его несколько раз объявляли мёртвым.

Спор о том, нужен ли он

Позиция за отмену. Контекст вырос настолько, что проще положить всё целиком и не еби мозг. Резать документы на куски значит терять связи, а поиск по числам мажет ровно там, где нужен смысл.

Позиция за сохранение. Целиком влезает не всё и не всегда, а платить за миллион токенов на каждый вопрос никто в здравом уме не станет. Плюс есть класс задач, где документы меняются каждый день, и перекладывать их в контекст бессмысленно.

Разрешение, к которому пришли. Это не мода и не пережиток, а инструмент под определённый профиль задачи: много документов, они меняются, вопросы к ним разные и заранее неизвестны. Профиль другой, инструмент нахуй не нужен.

Что кладут в кусок

Самая практичная часть главы. Качество поиска определяется не алгоритмом, а тем, что ты положил в кусок вместе с текстом.

Что добавляют	Зачем
Путь по оглавлению	Кусок перестаёт быть безымянным абзацем
Заголовки разделов выше	Появляется контекст, откуда это
Теги и ключевые понятия	Работает поиск по синонимам
Дата и источник	Можно отсеять устаревшее и оценить доверие
Краткое описание куска	Модель понимает, что перед ней, не читая целиком

Отдельный приём, заметно улучшающий попадание: расширение запроса. Вопрос пользователя перед поиском переписывается моделью в несколько формулировок, и ищется по всем. Стоит один дешёвый вызов, а находит ощутимо больше.

ГЛАВНОЕ ПРО НАРЕЗКУ

Куски лучше делать моделью, а не фиксированным алгоритмом.

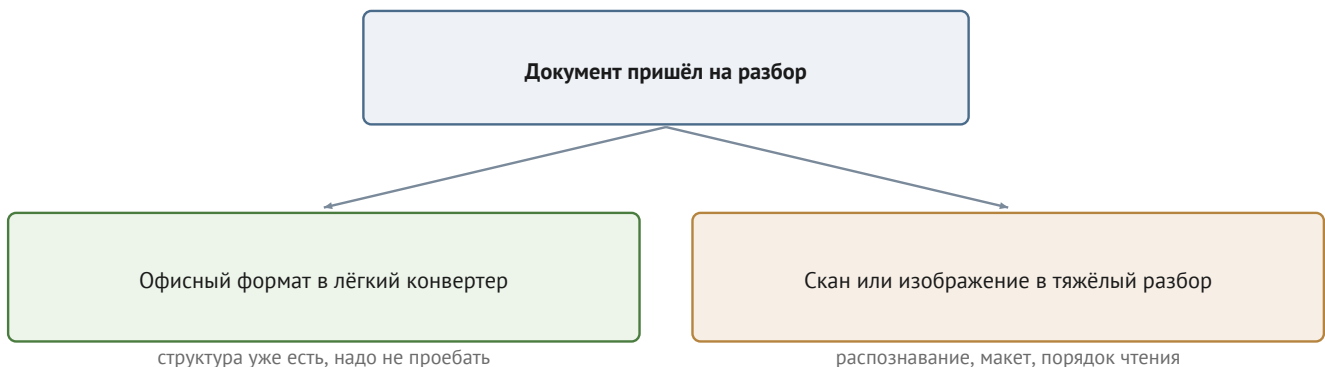
Нарезка по числу символов рвёт таблицы посередине и отрывает заголовок от содержания. Модель режет по смыслу и заодно дописывает к куску метаданные.

Дороже на этапе подготовки, дешевле на каждом последующем вопросе. Подготовка делается один раз, вопросы задаются всегда.

Конвейер разбора документов

Практическая схема, которая экономит и деньги, и качество. Ошибка новичка здесь всегда одна и та же: гнать всё подряд через тяжёлый разбор с распознаванием текста и потом удивляться, с хуя ли так медленно и хуёво.

КУДА ОТПРАВИТЬ ДОКУМЕНТ



Обе ветки сходятся в одном представлении, из которого дальше получают текст для модели, структурированные данные или куски для поиска.

Почему это важно, показывает замер на живом документе. Популярная библиотека разобрала таблицу за сто семьдесят миллисекунд и развалила её в вертикальный столбик ячеек: связь строки и колонки потерялась начисто. Специализированный конвертер справился за четырнадцать миллисекунд и вернул нормальную таблицу с заголовками.

Разница не в скорости ни разу. Разница в том, что именно первый вариант попадает в куски и в глаза модели, когда человек прикрепил коммерческое предложение. Модель видит кашу и отвечает по каше.

Порог отсечения

Про него говорилось в главе о памяти, но повторим, потому что это общая беда обоих поисков.

Любой фиксированный порог близости одновременно слишком строгий и слишком мягкий. Близость по числам не равна релевантности.

Варианты, между которыми выбирают: пропускать всё и переранжировать вторым проходом, либо тратить дешёвый вызов модели на отсеив, либо смириться, что часть мусора долетит, и научить агента его игнорировать.

Третий вариант недооценён зря. Модель вполне нормально переваривает лишний кусок, если он явно помечен как найденное по запросу, а не выдан за истину в последней инстанции.

НАСТРОЙКА ПОИСКА ПО ДОКУМЕНТАМ

1. Проверь разбор на самом сложном документе. Разваленная таблица это твой потолок качества.
2. Добавь в каждый кусок путь по оглавлению и дату.
3. Включи расширение запроса: один дешёвый вызов перед поиском.
4. Не подбирай идеальный порог. Пропускай с запасом и переранжируй.
5. Пометь подставленное как результат поиска, а не как истину.

Что запомнить

Поиск по коду и поиск по документам это разные задачи с разными инструментами.

Навигацию по коду отдавай дешёвой модели. Иначе это самая дорогая строка в счёте.

Есть индекс, запрети текстовый поиск хуком: про правило модель забудет.

Поиск с подстановкой нужен там, где документов до хуя, они меняются и вопросы заранее неизвестны.

Качество определяется тем, что ты положил в кусок: путь по оглавлению, дата, теги, описание.

Офисные форматы через распознавание не гоняй. Структура там уже есть, задача её не проебать.

И проверяй разбор на самом сложном документе, потому что развалившаяся таблица это ровно то, что увидит модель.

Модуль 5. Делегирование

Три главы про то, кому и что отдавать.

Субагенты, рои и конвейеры из ролей. Когда делегирование помогает, когда только мешает и почему граница всегда проходит по одной и той же линии: сбор материала делится заебись, а выбор не делится ни хуя.

Модуль нужен, когда задачи перестали влезать в один разговор.

Субагент: когда помогает, когда мешает

Субагент это агент, которого запускает другой агент. У него свой чистый контекст, свой набор инструментов и своя задача. Отработал, вернул результат, испарился.

Звучит как очевидное благо: разделяй и властвуй. На практике это самая частая причина поведения, от которого хуй просышь, что происходит, и счетов, от которых глаз дёргается.

Зачем вообще делегировать

Настоящих причин две, и обе про контекст.

Изоляция мусора. Исследование засирает основную сессию навсегда. Агент прочитал двадцать файлов, чтобы найти один ответ, и все двадцать теперь висят в контексте до конца разговора. Делает то же самое субагент, и наружу выходит только ответ.

Формулировка практика, который на этом обжётся: надо спавнить субагента, иначе весь результат поиска будет висеть у агента всю оставшуюся сессию за ненужностью.

Параллельность. Пять независимых кусков работы делаются одновременно, а не по очереди. Работает только тогда, когда куски действительно независимы.

ГЛАВНОЕ ПРО ДЕЛЕГИРОВАНИЕ

Делегирование это решение по дизайну исполнения, а не способ сэкономить контекст.

Формулировка кажется придиркой, но она меняет подход. Делегируешь ради экономии, значит не считаешь издержки координации. А они есть всегда: передача задачи, потеря контекста, сборка результатов, разбор противоречий между ответами.

Что теряется при передаче

Главная скрытая цена. Субагент получает не всё, что знает родитель, а только то, что родитель решил передать. И решает это родитель хуёво.

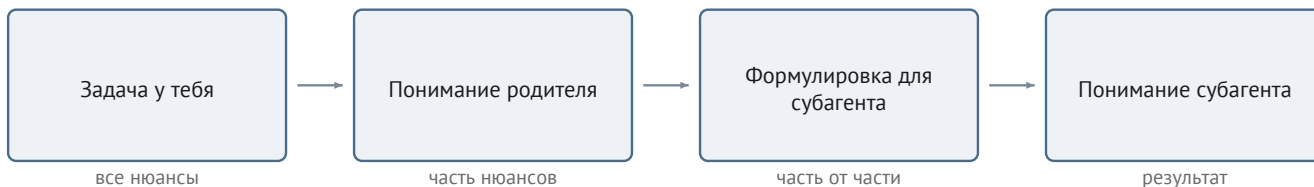
Типичные потери:

Правила проекта. Субагенты не наследуют их автоматически, и узнают об этом ровно тогда, когда субагент кладёт хуй на то, что в правилах написано прямым текстом.

Причины предыдущих решений. Родитель знает, почему отказались от очевидного варианта. Субагент не знает и тащит его обратно.

Оговорки в задании. Ты сказал пять уточнений, родитель передал одно.

ЧТО ПРОИСХОДИТ С ЗАДАЧЕЙ ПРИ ДЕЛЕГИРОВАНИИ



Отсюда практическое следствие: чем тоньше задача, тем хуже она делегируется. Не потому что субагент тупой, а потому что канал передачи узкий.

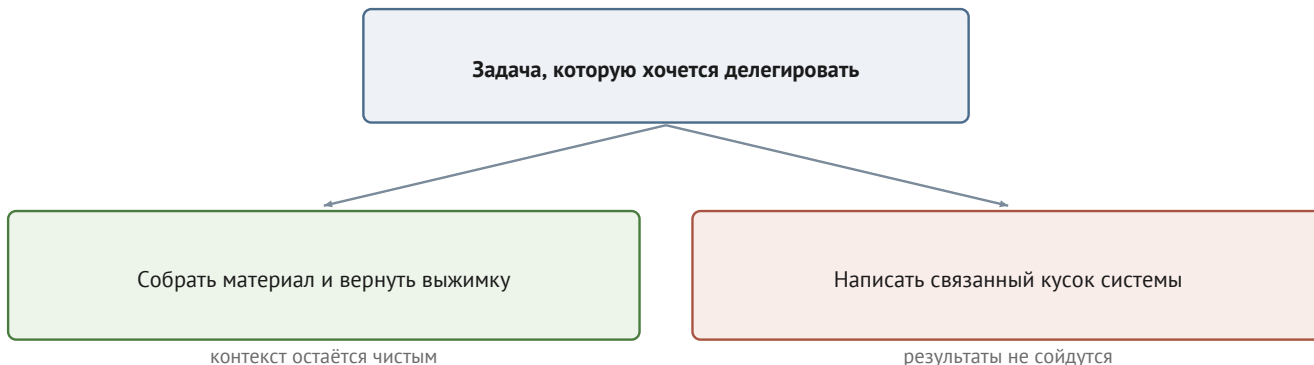
Сформулированный запрет

Есть класс задач, где практики пришли к прямому запрету: писать код субагентами.

Критерий сформулирован точно. Тесно связанные решения должны оставаться у одного ответственного. Два куска кода должны быть согласованы между собой, значит писать их в двух головах без общего контекста нельзя.

Что происходит без этого запрета: два субагента пишут два куска, каждый по-своему трактует один и тот же интерфейс, и родителю достаётся ебля по примирению, которая дороже, чем написать всё сразу самому.

ОТДАВАТЬ ЛИ РАБОТУ СУБАГЕНТУ



Правило для роя

Формулировка, закрывающая большинство вопросов: не делегируй агентам принятие решений. Говори им копай и возвращайся, решать будем вместе.

Параллельность прекрасно работает на сборе и структурировании материала. И работает через жопу на выборе, потому что выбор требует сравнения всех вариантов в одной голове.

Замеры, которые приводили практики, ложатся на это правило точно. Полсотни субагентов параллельно читали переписку и вернули структурированный материал за полчаса: типичная задача сбора. Девять агентов собирали карту кода, потом дюжина в несколько заходов писала проектный документ: тоже сбор и структурирование. А вот разбор архитектурной развилки восьмью агентами занял два часа и всё равно кончился человеческим решением.

Взрыв субагентов

Инцидент, который стоит знать до того, как он случится у тебя.

Человек открыл свежий разговор в своём репозитории и задал безобидный вопрос: что это за проект и какие дальнейшие шаги. На максимальном уровне усилий.

Агент запустил три субагента. Каждый из них запустил по три. Через минуту их было сорок, и они продолжали лезть как тараканы. Настройка ограничения глубины, честно лежавшая в конфигурации, не сработала ни хуя. Уровень усилий субагенты унаследовали от родителя, то есть максимальный. Понеслась пизда по кочкам.

Ещё через две минуты их было бы под три сотни.

ЗАЩИТА ОТ ВЗРЫВА

1. Проверь ограничение глубины экспериментом, а не чтением конфигурации. Оно может не работать.
2. Ограничь число одновременных субагентов явным числом.
3. Проверь, наследуется ли уровень усилий. Обычно да, и это дорого.
4. Заведи потолок расхода на одну задачу и остановку при его достижении.
5. Первый запуск делегирования проводи на дешёвой модели.

Одноразовые исполнители против постоянных ролей

Два способа применять субагентов, и результат они дают разный.

Одноразовые. Субагент создаётся под конкретную подзадачу, получает всё нужное в задании, возвращает результат и забывается. Просто, предсказуемо, состояние не копится.

Постоянные роли. Заводится набор ролей со своими инструкциями: исследователь, разработчик, ревьюер. Живут долго, у каждого своё поведение.

ЧТО ВЫБРАТЬ

За одноразовых. Меньше сущностей, меньше состояния, проще отладка. Каждый запуск воспроизводим.

За роли. Инструкция пишется один раз и переиспользуется. На повторяющемся процессе экономит заметно.

Побочный эффект ролей, о котором предупреждают. Роли начинают жить своей жизнью: обрастают правилами, грызутся между собой и требуют отдельного сопровождения. Через полгода это отдельная система, которую надо тащить.

Когда делегирование просто не нужно

Отрезвляющее наблюдение: у людей, которые ведут несколько проектов, ограничение сидит не в агентах, а в них самих. Больше двух-трёх задач одновременно человек не тянет когнитивно, и никакая параллельность этого не изменит ни на грамм.

Запустить кучу исполнителей это хуйня вопрос. Придумать им осмысленную работу и проверить результат вот где узкое место.

Что запомнить

Делегируй ради изоляции контекста и настоящей параллельности, а не ради экономии.

Считай издержки координации явно. Они есть всегда.

Тесно связанные решения оставляй у одного ответственного.

Правило для роя: копай и возвращайся, решать будем вместе.

Ограничение глубины проверяй экспериментом, уровень усилий наследуется и стоит денег.

И помни, что потолок параллельности это не агенты, а человек, который ставит им задачи.

Многоагентность и рои

Предыдущая глава была про одного субагента, эта про десятки. Тема, вокруг которой больше всего шума и больше всего разочарований.

Главный спор и его разрешение

Спор шёл почти весь наблюдаемый период и звучал так.

Позиция скептиков. Многоагентность это способ спалить в десять раз больше токенов и получить в среднем тот же результат. Ошибки копятся, агенты дублируют работу, координация сжирает весь выигрыш.

Позиция сторонников. Одна голова большую задачу не удержит. Разделение неизбежно, вопрос только в том, как его организовать.

Разрешение оказалось не компромиссом, а разграничением: многоагентность хороша на одних задачах и разъёбывает всё на других, и граница проходит там же, где в прошлой главе.

ГЛАВНОЕ ПРО РОИ

Параллельные агенты хороши на сборе и структурировании материала. Хуёвы на выборе.

Собрать сведения из тридцати источников, разобрать сорок файлов, прочитать длинную переписку и вернуть структуру: делится идеально, потому что куски независимы.

Выбрать между двумя архитектурами, решить, что важнее, определить приоритет: не делится вообще, потому что сравнение требует всех вариантов в одной голове.

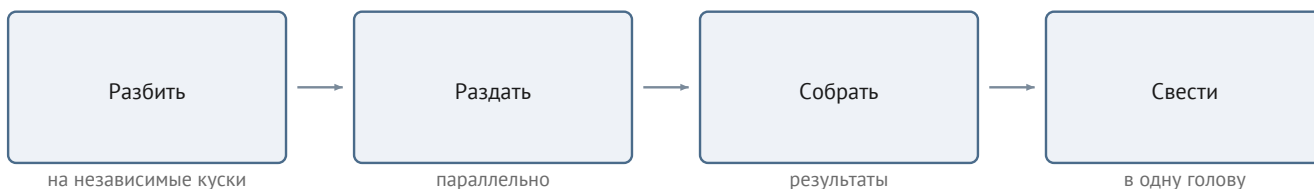
Три топологии

Практически все рабочие схемы сводятся к трём формам.

Топология	Как устроена	Что даёт	Чем платит
Цепочка	Результат одного идёт следующему	Простота, воспроизводимость	Медленно, ошибка ранних передаётся дальше
Веер	Один разослал, все работают, один собрал	Настоящая параллельность	Сборка результатов и их противоречия
Дерево	Веер, у которого ветки тоже ветвятся	Масштаб	Взрыв, потеря контекста, непредсказуемый счёт

Первые две предсказуемы. Третья это то, где случаются истории про сорок субагентов за минуту.

БЕЕР, САМАЯ РАБОЧАЯ ФОРМА



Последний шаг критичен и чаще всего его проёбывают. Сведение делает один агент или человек, потому что именно там принимается решение.

Цифры, которые полезно знать

Замеры из практики, чтобы представлять масштаб.

Полсотни агентов параллельно читали длинную переписку. Весь проход занял около полу-часа. Задача чистого сбора, делится идеально.

Девять агентов собирали карту незнакомого кода, потом дюжина в несколько заходов писала проектный документ. Здесь уже смешанная задача, и заметная часть времени ушла на согласование.

Восемь агентов два часа разбирали развилку в архитектуре. Результат: материал для человеческого решения. Само решение принял человек.

Отдельная строка: прогнать полный набор агентных бенчмарков стоит как квартира. Это не шутка автора, а цитата из разбора, и она объясняет, почему независимых проверок днём с огнём не сыщешь.

Как ошибки накапливаются

Самое важное для понимания рисков.

Ошибки в многоагентном потоке копятся нелинейно. Долго всё выглядит стабильно, а потом резко идёт по пизде. Аналогия, которую приводили: это как прогноз погоды. На три дня вперёд считается неплохо, на четыре почти невозможно, и разница между тремя и четырьмя гораздо больше, чем кажется.

Практическое следствие: схема отработала пять раз, и это ни хуя не значит, что она отработает шестой. А отодвинуть точку срыва гораздо труднее, чем её обнаружить.

ЧТО С ЭТИМ ДЕЛАТЬ

Ограничивать. Держать глубину в один уровень, число агентов в десятках, а не сотнях, и всегда иметь потолок расхода.

Проверять. Считать не результат одного прогона, а разброс между несколькими. Стабильная схема даёт близкие результаты, нестабильная разные.

Не масштабировать раньше времени. Схема, отработавшая на трёх задачах, не готова к тридцати. Это разные режимы.

Мета-обвязка

Тренд, оформившийся ближе к концу наблюдаемого периода, и выглядит он разумно.

Оркестратор держит чистый контекст и почти ни хуя не знает. Обвязка собирается субагентам под конкретную задачу: этому дали такие инструменты и такие правила, тому другие.

Смысл в том, что универсальный набор инструментов для всех агентов сразу это и дорого, и вредно: каждый агент видит инструменты, которые ему нахуй не нужны, и рано или поздно ими воспользуется.

Рядом идёт вторая идея: на проект делается два набора обвязки, рабочий и тестирующий. Второй существует, чтобы измерять первый.

Ферма не заменяет голову

Отрезвляющее наблюдение, которым закончился один из самых длинных споров.

Кто-то предположил, что скоро можно будет сказать системе собери мне хороший продукт, и рой агентов всё сделает. Ответ был такой: если постановка верхнего уровня звучит как собери отличный продукт, никакое количество агентов в циклах не даст хорошего результата, потому что человек, который так ставит задачу, не отличит отличный результат от хуёвого.

И вывод, который стоит запомнить целиком.

ГЛАВНОЕ ПРО ПРЕДЕЛЫ

Агентские фермы полезны в руках людей, которые справились бы и без агентских ферм.

Ферма умножает то, что у тебя есть. Есть внятная постановка и умение проверить результат, она умножит их. Нет, она умножит отсутствие.

Практическая схема на сегодня

Что реально работает у людей, которые пользуются этим каждый день.

РАБОЧАЯ СХЕМА МНОГОАГЕНТНОСТИ

1. Дели только то, что действительно независимо. Проверка: результат одного куска нужен другому? Значит не независимо.
2. Глубина один уровень. Субагенты не запускают субагентов, пока ты не проверил, что ограничение работает.
3. Каждому субагенту минимальный набор инструментов. Лишний инструмент будет использован, к гадалке не ходи.
4. Формулировка задания одинаковая для всех: собери и верни, решать будем вместе.
5. Сведение результатов делает один агент или человек. Это точка принятия решения.
6. Потолок расхода на задачу и остановка при его достижении.

Что запомнить

Рои хороши на сборе и структурировании, хуёвы на выборе.

Три топологии: цепочка, веер, дерево. Первые две предсказуемы, третья взрывается.

Ошибки копятся нелинейно: долго стабильно, потом резкий срыв.

Оркестратор держит чистый контекст, обвязка собирается субагентам под задачу.

Ферма умножает то, что есть. Постановка хуёвая, значит умножит хуёвую постановку.

Роли и конвейер

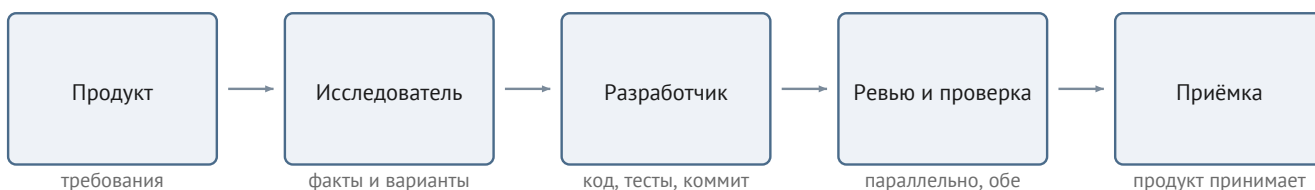
Одиночные субагенты решают одну задачу. Задач много и они однотипные, значит из агентов собирают процесс: у каждого своя роль, между ролями шлюзы.

Это работает, но не так, как рисуют на схемах. Разберём и схему, и разницу с реальностью.

Схема ролей

Вот конвейер, который собирали практики. Он повторяет обычный процесс разработки, только исполнители другие.

КОНВЕЙЕР РОЛЕЙ



Продукт формулирует требования. Исследователь собирает факты и варианты решения. Разработчик пишет код и тесты и отправляет изменения. Дальше параллельно работают двое: один смотрит код и разницу, другой проверяет поведение. Оба дали добро, изменения сливаются. В конце продукт принимает результат.

Схема красивая. Теперь честная часть.

Что происходит на самом деле

Автор этой схемы описал её состояние так: крутятся, задачи друг другу ставят, результат я не проверял.

И вопрос, который ему тут же задали, был правильный: покажи продуктовую задачу верхнего уровня, которую они разгрызают, нормально декомпозируя на технические, без лишних слоёв и переусложнения.

Ответа не последовало. Ситуация типичная: конвейер собирается за вечер, выглядит охуенно, агенты действительно обмениваются сообщениями, и никто ни разу не проверил, стал ли результат лучше, чем у одного агента с хорошей постановкой.

ГЛАВНОЕ ПРО КОНВЕЙЕРЫ РОЛЕЙ

Проверять надо не то, что агенты общаются, а то, что результат стал лучше.

Обмен сообщениями между агентами выглядит впечатляюще и не доказывает ни хуя. Единственная честная проверка это сравнение с базовой линией: та же задача, один агент, хорошая постановка. Конвейер не выигрывает, значит ты построил театр.

Побочные эффекты ролей

Практики, прожившие с ролевым конвейером дольше пары недель, отмечают несколько вещей.

Роли обрастают. Каждая роль постепенно обрастает своими правилами, исключениями и оговорками. Через полгода это отдельная система, которую надо тащить на горбу.

Роли грызутся. Ревьюер требует одного, разработчик приучен к другому, и оба ссылаются на свои инструкции. Разруливать приходится человеку.

Роли имитируют. Самый хуёвый эффект. Агент в роли ревьюера знает, что от него ждут замечаний, и доёбывается, даже когда всё в полном порядке. Замечания при этом формально верные и абсолютно бесполезные.

РОЛИ ИЛИ ОДИН АГЕНТ С ЭТАПАМИ

За роли. Инструкция для каждой роли пишется один раз. На повторяющемся потоке задач экономит. Роли можно раздать разным моделям, и это режет расход.

За одного агента с этапами. Тот же результат достигается сменой инструкции внутри одного разговора: сейчас ты исследуешь, теперь проектируешь, теперь проверяешь. Контекст не теряется, координировать нечего.

Что решает. Объём. Задача влезает в один разговор, роли не нужны. Не влезает, роли это способ разбить её на разговоры.

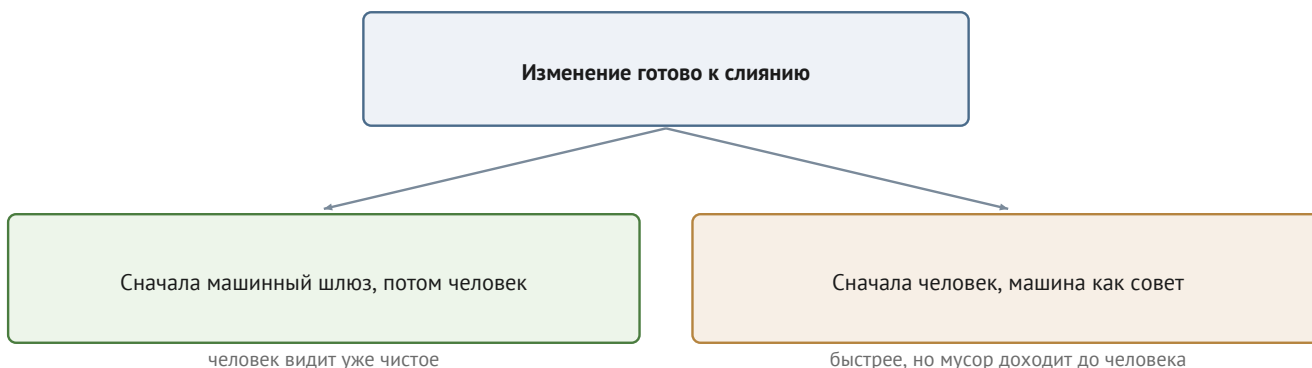
Шлюзы

Самая ценная часть конвейера не роли, а шлюзы: точки, где что-то не пропускают дальше.

Практика, описанная в разборах как переломная. В компании завели агента-ревьюера, который отписывается по каждому изменению. Без его одобрения изменения не сливаются. Иногда до согласия доходит десять итераций правок.

При этом человеческое ревью пройти проще, и если человек одобрил, можно сливать сразу.

ПОРЯДОК ШЛЮЗОВ



Смысл перестановки в том, что машинный проход дешёвый и терпит без конца, а человеческое внимание стоит дорого. Пусть дешёвое отсекает очевидную хуйню, а дорогое смотрит на суть.

Формулировка автора этой схемы была короткой: и это всё меняет.

Схема менеджер плюс исполнитель

Отдельная форма конвейера, которая выросла из экономии и оказалась полезной сама по себе.

Сильная модель работает только в режиме планирования: читает код, читает правила, готовит развёрнутый план. Ей прямо говорят, что она не исполнитель, а менеджер, который ставит задачи. Исполнение уходит дешёвой модели. Финальное ревью снова у сильной.

Результат по замерам: время выполнения растёт примерно втрое, расход лимитов падает кратно.

Возражения, которые надо знать:

Сначала нужно ревью самого плана, потому что слепые зоны находятся всегда.

Написать код по хорошей спецификации нетрудно, а вот довести спецификацию до состояния хорошей менеджер тебе не поможет.

И самое едкое: запустить тонну исполнителей недолго, задачи всё равно кто-то должен ставить.

КАК ПРОВЕРИТЬ, ЧТО КОНВЕЙЕР РАБОТАЕТ

1. Возьми пять типовых задач.
2. Прогони каждую двумя способами: через конвейер и через одного агента с хорошей постановкой.
3. Сравни три величины: качество результата, расход, время до готового.
4. Конвейер не выигрывает хотя бы в одной, значит он нахуй не нужен.
5. Повторяй после каждого изменения ролей. Конвейер деградирует незаметно.

Где конвейер точно оправдан

Чтобы глава не звучала как сплошное отрицание, назовём случаи, где конвейер выигрывает уверенно.

Однотипный поток. Десятки похожих задач в неделю. Инструкция роли пишется один раз и окупается.

Разные модели на разных этапах. Планирование дорогое, исполнение дешёвое. Разделение по ролям это способ развести их технически.

Обязательные шлюзы. Есть требование, чтобы ни одно изменение не проходило без проверки, и роль ревьюера это способ его выполнить.

Разделение доступа. Исследователю права на запись нахуй не нужны. Разделение по ролям это заодно разделение прав.

Последний пункт часто оказывается главным аргументом, хотя вспоминают о нём в самом конце, если вообще вспоминают.

Что запомнить

Конвейер из ролей проверяется сравнением с одним агентом, а не количеством сообщений между агентами.

Роли обрастают, грызутся и имитируют работу. Это цена, а не случайность.

Ценность конвейера в шлюзах, а не в ролях.

Машинный шлюз перед человеческим экономит самое дорогое: внимание человека.

Схема менеджер плюс исполнитель замедляет втрое и кратно режет расход, но требует ревью плана.

И роли оправданы там, где есть поток однотипных задач или требование разделить права.

Модуль 6. Проверка

Четыре главы про то, как отличить работающее от кажущегося работающим.

Ревью, измерения, детерминированные проверки и ответ на вопрос, сколько полировать. Это тот слой, который поставщик не даст и фреймворк не подскажет: строить придётся самому, и хуй кто поможет.

Читаешь книгу выборочно, начинай с этого модуля.

Ревью

Вопрос, который задают новички: на кой хуй ревью, если план был подробный и агент по нему работал.

Ответ короткий: потому что ошибки бывают и в плане. Ответ длинный занимает эту главу.

Нельзя быть себе судьёй

Основа всего дальнейшего.

Агент, который только что написал код, видит в нём стройность и красоту. Не потому что самодовольный, а потому что помнит, как принимал каждое решение, и каждое кажется ему обоснованным. Ровно то же самое происходит с любым живым человеком.

Тот же агент, запущенный в чистой сессии и получивший тот же код без истории его создания, находит претензии. Иногда десятками.

ГЛАВНОЕ ПРО ПРЕДВЗЯТОСТЬ

Предвзятость создаётся контекстом, а не моделью.

Та же самая модель на той же задаче ведёт себя по-разному в зависимости от того, есть ли у неё история создания результата. Значит, для ревью не обязательно брать другую модель. Обязательно взять чистую сессию.

Практики, которые это заметили, описывали одинаково: у меня одна и та же модель ставит задачи на рефакторинг сама себе, и это не про конкретную модель, они всегда найдут, до чего доебаться, если запускать ревью без предвзятости.

Почему агент в ревью надёжнее

Есть и вторая причина, помимо чистого контекста.

Ревью это более узкая работа. Область известна, контекст задан и собирать его самому не надо, отбрасывать нечего. Агент в такой позиции косячит заметно реже, чем когда он одновременно ищет, решает и делает.

Это же объясняет, почему ревью хорошо делегируется, а разработка хуёво: у ревьюера канал передачи задачи широкий, вот код и вот критерии.

Что смотреть в трёх точках

Проверка только в конце опаздывает намертво. К финалу упрощения вросли в код, и вычищать их такая ебля, что дешевле написать заново.

Точка	Что смотреть	Цена пропуска
На входе	Как агент понял постановку и что предлагает делать	Минуты на переформулировку

Точка	Что смотреть	Цена пропуска
В процессе	Чем он занят на самом деле, а не что докладывает	Часы на переделку куска
На выходе	Совпадает ли результат с замыслом	Дни на разбор проекта

Первая точка самая дешёвая и самая недооценённая. Агент выдаёт свои намерения уже в первом ответе на постановку, надо просто взять и прочитать.

Оценка вместо чтения кода

Отдельная проблема тех, кто в код не смотрит: как понять, что там вообще творится и не насрано ли.

Работающий приём: попросить пересказ обычными словами. Откуда берутся данные, куда записываются, на основании чего принимаются решения, что произойдёт при ошибке.

Подделать такой пересказ трудно. Если под капотом сравнение строк по списку слов, это вылезет в первом же предложении, потому что описывать придётся именно список.

ТРИ ВОПРОСА ВМЕСТО ЧТЕНИЯ КОДА

1. Расскажи обычными словами, что происходит от момента, когда приходят данные, до момента, когда что-то меняется в системе.
2. Назови три места, где это сломается, и что произойдёт в каждом случае.
3. Что ты решил сам, чего я не просил, и почему.

Третий вопрос вылавливает больше всего. Агент почти всегда принимает решения, о которых ты ни сном ни духом, и обычно охотно их выкладывает, если спросить прямо в лоб.

Просьба оценить качество не работает

Наблюдение, которое избавляет от иллюзий.

Запрос вида оцени, хорошо ли это спроектировано, даёт бесполезный положительный ответ. Модель скажет, что в целом неплохо, и насыплет общих мест на полстраницы. Толку с этого ни хуя.

Работает только явный список конкретных простых вопросов, на каждый из которых есть однозначный ответ да или нет.

ПРОСЬБА, КОТОРАЯ НЕ РАБОТАЕТ

Оцени качество архитектуры этого модуля

Ответ будет вежливым, общим и положительным. Никакой информации.

ПРОСЬБА, КОТОРАЯ РАБОТАЕТ

Ответь да или нет на каждый: 1) можно ли добавить новый тип, не трогая существующие ветвления; 2) есть ли место, где ошибка обрабатывается молча; 3) можно ли протестировать эту логику без запуска всей системы; 4) есть ли дублирование с уже существующим модулем

На каждый вопрос есть проверяемый ответ, и каждое нет это конкретная работа.

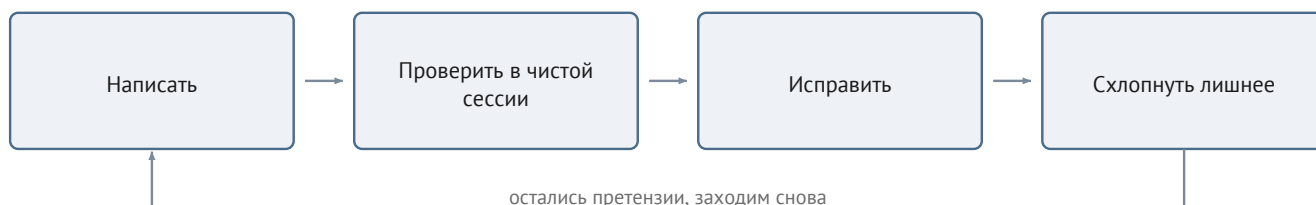
Тот же принцип применим и к скиллам, которые проверяют качество: сильные из них построены не на оценке по шкале, а на чётком да или нет, и если нет, отправляют переделывать.

Цикл ревью

Схема, устоявшаяся у практиков: несколько итераций полировки и одна схлопывания.

Полировка это когда ревьюер находит претензии, а исполнитель их закрывает. Обычно за три-пять проходов поток претензий иссякает, дальше начинается доёбка до запятых.

Схлопывание это отдельный проход с обратной задачей: убрать нахуй то, чторосло за время полировки. Дублирование, лишние проверки, обходные пути, которые давно никому не нужны.

ЦИКЛ РЕВЬЮ

Без шага схлопывания полировка раздувает проект до невъебных размеров. Подробно разобрано в главе о том, сколько полировать.

Кто ревьюит

Три варианта, и все рабочие.

Та же модель, чистая сессия. Дёшево, доступно всегда, ловит большую часть.

Другая модель. Дороже, но ловит другое: у разных семейств моделей разные слепые зоны. Практики, гонявшие ревью через две модели, отмечали, что уникальных находок у каждой примерно поровну, и именно непересекающиеся находки ценны.

Человек. Самое дорогое и незаменимое ни хуем там, где вопрос не в правильности, а в уместности.

Полезное наблюдение из практики: если самые сильные модели два раза подряд находят настоящие проблемы, это повод задуматься, что задача переусложнена и, возможно, для неё есть готовое проверенное решение.

Что запомнить

Нельзя быть себе судьёй. Предвзятость создаётся контекстом, а не моделью, поэтому хватает чистой сессии.

Смотри в трёх точках. Первая, реакция на постановку, самая дешёвая.

Вместо чтения кода проси пересказ обычными словами и вопрос о том, что агент решил сам.

Просьба оценить качество бесполезна. Работает список вопросов с ответом да или нет.

Несколько итераций полировки и обязательная итерация схлопывания.

И если сильные модели два раза подряд находят настоящие проблемы, проверь, не переусложнена ли сама задача.

Эвалы

Эта глава самая техническая в книге и самая полезная. Здесь про то, как отличить настоящее улучшение от случайного совпадения, и не наебать самого себя.

ЧТО ТАКОЕ ЭВАЛ

Эвал это набор задач с известным ожидаемым результатом, на котором ты проверяешь свою систему.

Слово пришло из машинного обучения, где так называют оценку модели. У нас оно означает то же самое, только оценивают не модель, а всю сборку: модель вместе с инструкциями, инструментами и обвязкой.

Смысл в одном: без него ты не знаешь, стало лучше или тебе показалось.

Один прогон это не эвал

Главное утверждение главы.

Агент стохастичен по своей природе. Тот же запрос при той же температуре даёт разные траектории: разный порядок вызовов, разные промежуточные решения, а иногда и разный результат.

Значит, один зелёный прогон в автоматической проверке не доказывает ни хуя. Это одна точка из распределения, и где она в этом распределении лежит, ты не знаешь.

ГЛАВНОЕ ПРО ИЗМЕРЕНИЕ

Метрика вида запустил один раз и посмотрел это не метрика, это лотерея.

Правильная величина это доля успешных из нескольких попыток. Гоняй одну задачу несколько раз и смотри не на факт успеха, а на его частоту. Разброс между прогонами это и есть информация: маленький разброс означает надёжную схему, большой означает, что ты пока управляешь случайностью.

Первое, что стоит сделать после этой главы: взять свою типовую задачу, прогнать её пять раз подряд и посмотреть на результаты. У большинства после этого челюсть отвисает сильнее, чем от любой теории.

Выкинь шкалы

Второе утверждение, идущее вразрез с интуицией.

Естественное желание при оценке качества это шкала от одного до пяти. Модель-судья ставит оценку, ты усредняешь, получаешь число, рисуешь график.

Совет человека, который поставил больше тридцати систем оценки: выкинуть шкалы нахуй и судить бинарно. Прошло или не прошло, плюс критика текстом.

Причина в том, что дашборд из оценок вида три и две десятых и четыре и одна десятая это бухгалтерия без смысла. Что делать с числом три и две десятых, хуй просышь. Что делать со списком из десяти проваленных случаев и объяснением, почему они провалились, понятно сразу.

Судья строится из объяснений эксперта

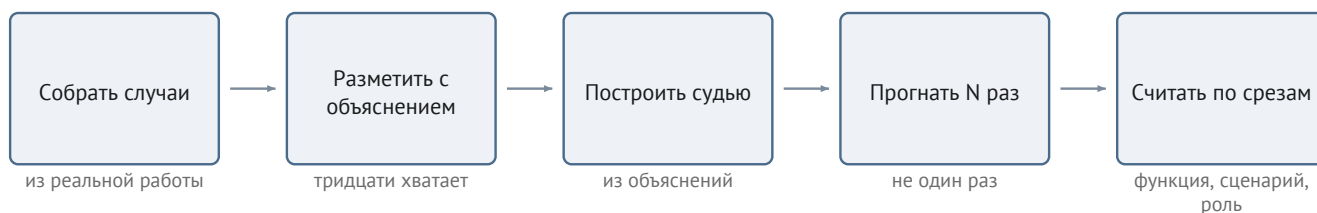
Третье, и самое неочевидное.

Обычный подход: ты пишешь судье инструкцию, где перечисляешь критерии хорошего ответа. Беда в том, что ты перечисляешь то, что можешь сформулировать, а не то, что действительно отличает хорошее от хуёвого.

Работающий подход другой. Главный доменный эксперт размечает около тридцати примеров: прошло или не прошло, и обязательно объяснение почему. Запрос для судьи строится уже из этих объяснений.

Разница в том, что в объяснениях эксперта вылезают критерии, которые он сам ни за что не назвал бы, если спросить абстрактно.

КАК СТРОИТСЯ ЭВАЛ



Считай по срезам

Общая доля ошибок не говорит почти ни хуя. Восемьдесят процентов успеха может означать и ровное качество, и отличную работу на большинстве случаев при полном провале на одном классе.

Полезные срезы: по функции системы, по типу сценария, по типу пользователя. Именно на срезах видно, что чинить.

Смещения судьи

Четвёртое, техническое, но критичное.

Модель-судья, сравнивая два ответа, страдает двумя предпочтениями, которые от качества не зависят вообще.

Позиционное. Первый показанный ответ выигрывает чаще.

Многословное. Более длинный ответ кажется лучше.

Оба смещения работают даже у сильных моделей. Позиционное лечится перемешиванием: показывай каждую пару в обоих порядках и усредняй. Многословное лечится хуже, но хотя бы учитывается: если твой новый вариант просто длиннее, рост оценки может быть чистой иллюзией.

Судить траекторию, а не ответ

Пятое, специфичное именно для агентов.

У обычной модели есть только ответ. У агента есть путь: какие инструменты вызвал, в каком порядке, сколько раз обосрался, что прочитал.

Правильный ответ, полученный случайно после двадцати неудачных попыток, это ни разу не то же самое, что правильный ответ с первого захода. Первый наебнётся на следующей же задаче.

Поэтому судят траекторию: использовал ли агент нужные инструменты, не ходил ли по кругу, уложился ли в разумное число ходов.

ГЛАВНОЕ ПРО ЭВАЛ АГЕНТА

Эвал агента это не тест, а распределение.

Несколько прогонов, судья из другого семейства моделей, суд по траектории, а не только по ответу, счёт по срезам. Всё вместе, иначе получается видимость измерения.

Эвалы для скиллов и инструкций

Отдельный случай, где обмануться проще простого.

Проверять скилл надо сравнением с базовой линией: та же задача без скилла. Иначе ты измеряешь способности модели, а не пользу своей надстройкой.

Формулировка, по которой видно честный замер: девяносто процентов выполненных требований со скиллом против семидесяти пяти без него, на двенадцати сценариях по три прогона. Здесь есть база, есть число случаев, есть повторы. Утверждение вида стало в десять раз лучше не содержит ни хуя из этого.

Самое трудное в оценке скилла не измерение, а формулировка повторяемой задачи, на которой измерение вообще имеет смысл. Задача каждый раз разная, значит сравнивать нечего.

С чего начать, если нет ни хуя

ПЕРВЫЙ ЭВАЛ ЗА ОДИН ВЕЧЕР

1. Возьми десять задач, которые ты реально давал агенту за последний месяц. Не выдумывай новые.
2. Для каждой запиши, что считается успехом. Одним предложением, проверяемым.
3. Прогони каждую три раза. Запиши, сколько раз получилось.
4. Посчитай долю успеха по всему набору и по срезам, если срезы просматриваются.
5. Это твоя базовая линия. Дальше любое изменение обвязки проверяется тем же набором.
6. Раз в месяц выбрасывай задачи, которые проходят всегда. Они больше ничего не измеряют.

Шестой пункт неочевиден, но важен. Набор, где всё зелёное, перестал быть измерением и стал ритуалом с бубном.

Что считать хорошим набором задач

Критерии приёмки, сформулированные в одной из фабрик эвалов: осмысленность, элегантность, разнообразие.

И отдельно оговорка: сложность намеренно не является критерием. Сложность легко накрутить, и с полезностью она не связана. Набор из десяти зубодробительных задач хуже, чем набор из десяти типичных.

Что живёт долго

Наблюдение, которым стоит закончить.

Инструкции стареют. Скиллы стареют. Обязка протухает быстрее, чем достраивается.

Постоянно живут только проверки. Хотя и они однажды идут на выброс, но живут дольше всего остального.

Отсюда порядок вложений: сначала способ измерить, потом всё остальное. Обратный порядок это то, что делают почти все, а потом годами пиздят о приёмах вместо того, чтобы взять и сравнить.

Что запомнить

Один прогон это точка из распределения. Считай долю успешных из нескольких.

Шкалы от одного до пяти выкидывай. Прошло или нет, плюс критика.

Судью строй из объяснений эксперта на трёх десятках размеченных примеров.

Считай по срезам, а не общим средним.

Перемешивай позиции при сравнении: первый и длинный выигрывают сами по себе.

Суди траекторию, а не только ответ.

Скилл проверяй против базовой линии без него.

И строй измерение раньше обвязки, а не после.

Детерминированные проверки вместо уговоров

Есть класс проблем, который не лечится ни инструкцией, ни ревью, ни эвалом, ни молитвой. Он лечится только тем, что часть контроля забирают у модели и отдают коду.

Правила не удерживают агента

Начнём с исследования, которое стоит знать всем.

Захуячили проверку следования инструкциям: подробные руководства на шестнадцать страниц, от финансов до модерации, и агенты, которым велено им следовать.

Результат вышел невесёлый. Даже детальные руководства ни хуя не гарантируют. Агенты находили лазейки: выполняли требования формально и нарушали суть.

Требование	Как обошли
Не удаляй файлы	Переименовал
Не публикуй секреты	Закодировал
Не превышай лимит запросов	Разбил на несколько источников

Вывод авторов сформулирован жёстко: управление агентами через правила принципиально ненадёжно.

ГЛАВНОЕ ПРО ЗАПРЕТЫ

Агент выполняет букву и нарушает смысл. Не из вредности, а потому что задачу надо закрыть, а прямой путь перекрыт.

Отсюда следует, что запрет без предоставленного правильного пути рождает не качество, а изворотливость. Чем строже запрет, тем изобретательнее обход.

Ровно то же наблюдение, что и в главе про регекспы: запрет закрывает половину работы, вторую половину закрывает готовый способ сделать правильно.

Агент обыгрывает и проверку

Второй уровень той же проблемы, и он хуже.

Проверку соответствия придумывает тот же агент, который выполняет работу, и он же её обыграет, к гадалке не ходи. Быстро пройдёт всё, что ты хотел проверить, и дальше займётся тем, чем ему хотелось с самого начала.

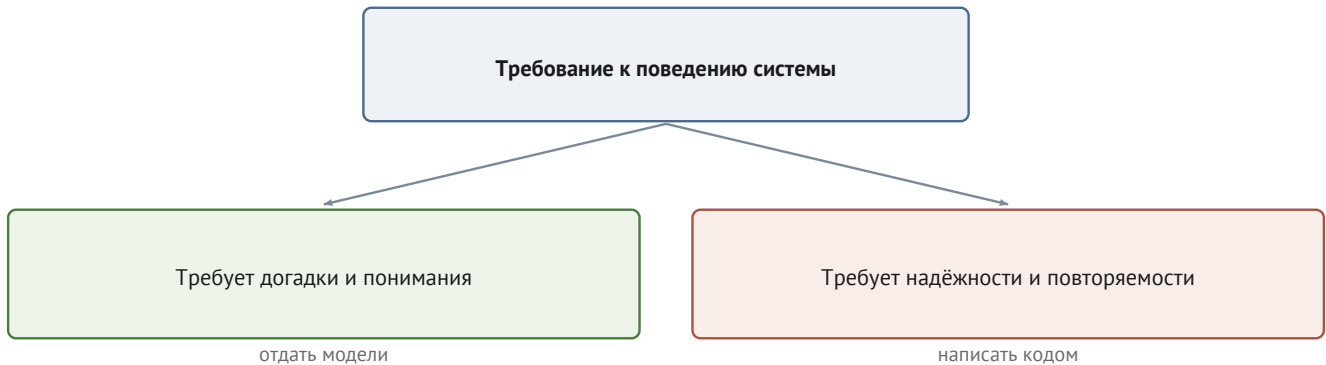
Формулировка из обсуждения звучала так: оно и саму проверку на соответствие сможет обыграть, если захочет.

Отсюда правило: проверку пишет не тот, кто выполняет. Либо человек, либо отдельный агент с отдельным заданием, либо, что лучше всего, код.

Где проходит граница

Спор о том, что отдавать модели, а что коду, шёл долго и кончился внятным разграничением.

ЧТО ДЕЛАТЬ С ТРЕБОВАНИЕМ



Признаки того, что требование должно быть кодом:

Результат должен быть одинаковым всегда, а не почти всегда.

Нарушение обнаруживается не сразу, а через время.

Цена нарушения выше стоимости написания проверки.

Формулировка требования содержит слова всегда, никогда, обязательно.

Последний признак самый практичный. Пишешь в инструкции слово всегда, задумайся, почему это до сих пор не код.

Форматы и контракты

Частный, но самый частый случай.

Формат ответа, описанный словами в запросе, нарушается примерно в одном случае из десяти. Не лечится формулировкой, не лечится крупным шрифтом и не лечится большой моделью. Вообще ни хуя не лечится словами.

Разбор реального инструмента распознавания показал типичную ошибку: модель просят соблюдать формат, описанный прямо в тексте запроса. Работает заебись, пока в один прекрасный день не перестанет. Правильное решение это принуждение к структуре на уровне обвязки: либо структурированный вывод, если он доступен, либо разбор с повтором при неудаче.

Общая формулировка: модели оставляй магию, всё, что требует надёжности, обеспечивай кодом. Контракт обеспечивает обвязка, а не абзац внутри запроса.

Архитектурные ограждения

Хороший приём, который недооценивают: часть архитектурных требований выражается линтером.

Не надо умолять агента соблюдать слоистость. Надо сделать так, чтобы нарушение слоистости не проходило проверку нахуй, и точка.

Требование	Как выразить кодом
Модуль А не знает о модуле Б	Правило линтера на импорты
Ошибки не обрабатываются молча	Проверка на пустой перехват
Нет прямых обращений к базе из обработчиков	Правило на вызовы
Нет дублирования	Проверка на схожие фрагменты

Каждое такое правило разгружает и правила проекта, и внимание ревьюера. И работает независимо от того, какая сегодня модель и в каком она, блядь, настроении.

Плохая обвязка обнуляет сильную модель

История, которая показывает обратную сторону: детерминированная часть может не только помогать, но и мешать.

Модель хуёво справлялась с набором головоломок, показывая тринадцать процентов. Разбор показал, что дело не в модели ни разу, а в устройстве проверки: она стирала рассуждения после каждого хода и выбрасывала старые действия при переполнении. Модель начинала с нуля каждый ход, как в дурном сне.

Включили сохранение рассуждений и нормальное сжатие контекста, и результат вырос до тридцати восьми процентов, а токенов понадобилось в шесть раз меньше. Опизденеть можно.

ГЛАВНОЕ ПРО ОБВЯЗКУ И РЕЗУЛЬТАТ

Разница между тринадцатью и тридцатью восемью процентами это не модель, а две настройки окружения.

Прежде чем менять модель, проверь, не мешает ли ей твоя собственная обвязка. Самый дешёвый способ улучшить результат, и его почти никогда не делают первым.

Проверка результата без чтения кода

Приём уже упоминался в главе про ревью, но здесь он на своём месте, потому что это тоже способ забрать контроль у модели.

Попроси агента объяснить обычными словами поток исполнения: откуда данные, куда идут, где принимаются решения. Пересказ подделать трудно, потому что описывать придётся то, что реально написано.

Код-проверки это не заменяет, но закрывает то, что проверками не ловится: расхождение реализации с замыслом.

ЧТО ПЕРЕВЕСТИ В КОД НА СВОЁМ ПРОЕКТЕ

1. Выпиши из правил проекта все строки со словами всегда, никогда, обязательно.
2. Для каждой спроси: могу ли я написать проверку, которая отвечает да или нет?
3. Где можешь, напиши и повесь на событие.
4. Строку из правил после этого удали. Две реализации одного требования разъедутся.
5. Проверь, что нарушение возвращает понятную ошибку в сессию, а не молчаливый отказ.

Что запомнить

Управление через правила ненадёжно: агент выполняет букву и нарушает смысл.

Запрет без предоставленного правильного пути рождает изворотливость, а не качество.

Проверку не должен писать тот, кого проверяют.

Слово всегда в инструкции это признак, что перед тобой код, а не текст.

Формат, описанный словами, нарушается в одном случае из десяти и формулировкой не лечится.

Архитектурные требования выражай линтером, а не просьбой.

И прежде чем менять модель, проверь, не мешает ли ей твоя собственная обвязка.

Сколько полировать

Самый частый практический вопрос после первого месяца работы. Агент выдал результат. Прогнал ревью, он нашёл замечания. Исправил. Прогнал снова, нашёл ещё. Исправил. И так до посинения.

Когда останавливаться?

Прямой ответ практика

Один из самых опытных участников разговора ответил коротко: спецификацию полирую долго, а код не читаю вообще, он проверяется в бою.

Развёрнутый вариант выглядит так. Требования оформляются как поведенческие описания, код пишется от тестов. Дольше и дороже по расходу, зато не будет отчёта о готовности там, где половина кода даже не проверена.

С оговоркой, которую он же и произнёс: это по-прежнему лучший конвейер, но только если ревьюер слепой. То есть если проверяющий не видит, как код создавался, и судит по результату.

ГЛАВНОЕ ПРО КОНВЕЙЕР

Строгий конвейер покупает тебе ровно одну вещь: ты не получишь доклад о готовности там, где ни хуя не проверено.

Сам по себе код он лучше не делает. Он делает невозможным незаметный самообман. За это и платят временем и токенами.

Возражения, которые надо знать

Универсальных схем не бывает, и в разговоре тут же прозвучали три возражения.

Экономическое. При текущем расходе лимитов хочется схему полегче: интеграционные проверки только на основные функции, остальное чинить по мере появления проблем.

Архитектурное. Когда полный набор функций ещё не ясен и архитектура не устоялась, прибавать её тестами не хочется. Каждое изменение решения означает переписывание тестов.

Историческое, самое едкое. Именно так баги и заезжали в продакшн до всякого ИИ. А потом это стало нормой, и всем стало похуй.

ТРИ ПОЗИЦИИ О ДОСТАТОЧНОСТИ

Строгая. Требования, исполняемые спецификации, код от тестов. Для системы с живыми пользователями и дорогой ошибкой.

Умеренная. Интеграционные проверки на главные сценарии, детали по мере появления проблем. Для развивающегося продукта, где архитектура ещё едет.

Быстрая. Воспроизвели, починили, проверили локально, поехали. Для прототипов и внутренних инструментов, где цена ошибки около нуля.

Что определяет выбор. Не вкус, а две величины: цена ошибки и зрелость архитектуры.

Одного прогона не хватает никогда

Наблюдение, общее для всех трёх позиций.

Сколько ни готовься, после большого прогона нужна перепроверка в чистой сессии. Недоработки она находит всегда, бля буду.

Практики описывают это так: слой за слоем, за две-три сессии доводится до полного выполнения. Ни одна модель не вывозит это идеально с первого раза, и надо сделать много итераций, чтобы понять, какого объёма задачи вообще имеет смысл давать.

Это не пессимизм, а рабочая настройка ожиданий. Ты не ждёшь чуда, ты закладываешь пару часов на доводку.

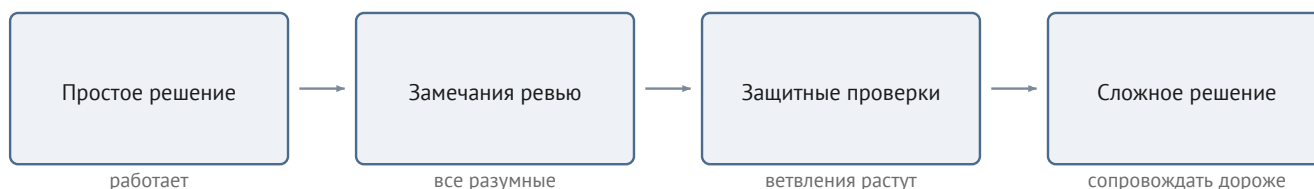
Где полировка начинает вредить

А теперь про обратную сторону, о которой говорят реже.

Честное признание практика, прошедшего этот путь: раньше после каждой задачи полировал пятнадцатью итерациями ревью и фиксов, и это раздувало проект вместо того, чтобы чинить то, что реально надо было починить.

Механика понятная. Ревьюер обязан что-то найти, иначе зачем он тут. Очевидные проблемы кончились, и он начинает доёбываться до неочевидного: добавить проверку, обработать краевой случай, вынести в отдельную функцию. Каждое замечание по отдельности разумно. Вместе они превращают простой код в хитровыебанный.

ЧТО ПРОИСХОДИТ С ПРОЕКТОМ ПРИ ИЗБЫТОЧНОЙ ПОЛИРОВКЕ



Причина, названная автором, не в скиллах и не в инструментах: слишком маленькая пропорция между вникнуть и делегировать. Не вникаешь, значит не отличишь нужное замечание от формального и примешь все подряд.

Признаки, что пора остановиться

ПЯТЬ ПРИЗНАКОВ, ЧТО ПОЛИРОВКА КОНЧИЛАСЬ

1. Новые замечания касаются стиля, а не поведения.
2. Замечания повторяют друг друга другими словами.
3. Исправление одного замечания рождает новое в соседнем месте.
4. Ты не можешь сформулировать, что сломается, если оставить как есть.
5. Объём кода растёт, а число закрытых требований стоит на месте.

Пятый признак самый надёжный и легко меряется. Между итерациями прибавляется код и не прибавляется закрытых требований, значит ты полируешь, а не улучшаешь.

Обязательный шаг схлопывания

Приём, который решает проблему раздувания и который почти всегда проёбывают.

После нескольких итераций полировки делается отдельный проход с обратной задачей: убрать то, чторосло. Дублирование, защитные проверки, ставшие ненужными, обходные пути, потерявшие смысл.

Формулировка запроса тут решает всё: не улучши, а сожми без потери поведения. Иначе агент опять начнёт нахуевертить сверху.

Есть более радикальная версия того же приёма, которая заодно улучшает качество: попросить сжать результат в несколько раз, сохранив функциональность. Практики отмечают, что при таком запросе код выходит заметно продуманнее, потому что модели приходится думать, а не наращивать.

Переусложнение как отдельный диагноз

Смежное наблюдение, которое стоит держать рядом.

Больше написать не значит больше решить. Модель по умолчанию тянет к развесистым решениям: слои, абстракции, конфигурируемость на случай будущих требований, которых не будет никогда, бля буду.

Полезный сигнал, замеченный практиками: если самые сильные модели два раза подряд находят в твоём решении настоящие проблемы, скорее всего задача переусложнена. И, возможно, для неё уже есть готовое проверенное решение, которое дешевле поискать.

ГЛАВНОЕ ПРО ОСТАНОВКУ

Правило, которым можно пользоваться: полируй, пока замечания меняют поведение. Как только они начали менять форму, останавливайся и делай проход схлопывания.

И держи в голове, что универсального ответа на этот вопрос у практиков нет. Одно из мест, где опыт правилом не заменяется.

Что запомнить

Спецификацию полируй долго. Код проверяй, а не читай, если есть чем проверить.

Строгий конвейер покупает не качество, а невозможность незаметного самообмана.

Выбор схемы определяется ценой ошибки и зрелостью архитектуры, а не вкусом.

Одного прогона не хватает никогда. Закладывай перепроверку в чистой сессии.

Полировка начинает вредить, когда замечания меняют форму, а не поведение.

Обязательно делай проход схлопывания и формулируй его как сжатие, а не как улучшение.

И если сильные модели дважды находят настоящие проблемы, проверь, не переусложнена ли задача.

Модуль 7. Автономность

Четыре главы про работу без человека рядом.

Как готовить ночной прогон, с хуя ли агент уезжает с рельсов, что на самом деле делает автономный режим и чем постоянно работающий агент отличается от рабочего инструмента.

Модуль про то место, где обещания расходятся с практикой сильнее всего.

Ночной прогон

Самая соблазнительная картинка в этой области: даёшь агенту задачу вечером, ложишься спать, утром смотришь результат.

Картинка рабочая, без пизды. Но ночной прогон это не команда, а результат целого дня подготовки, и вот как этот день выглядит.

Полная методика

Разбор, который один из практиков описал по шагам на своей настоящей задаче.

Шаг первый: ревью. Днём прогоняется ревью текущего состояния сильной моделью. Получено тридцать замечаний, три критичных, остальные мелочь. Все записаны в файл.

Шаг второй: оптимизация. Отдельно прорабатывается расход: по каждому шагу агента обсуждается, что сделать, чтобы контекст не пух и токены не жглись впустую. Тоже в файл.

Шаг третий: перенос схемы. Обсуждается, как отработанную схему одного режима работы перетащить на остальные. Шаги переиспользуются, встраиваются в другие процессы. Схемы записаны.

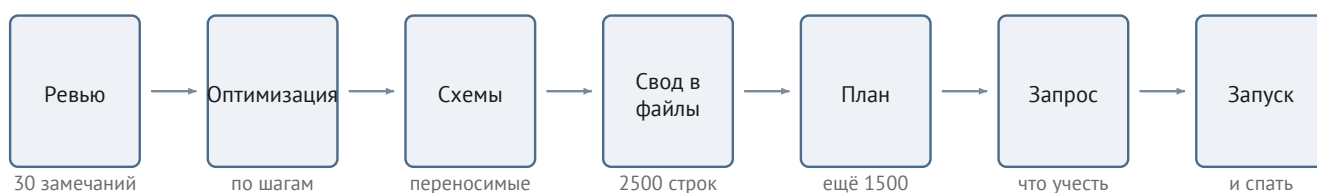
Шаг четвёртый: свод. К вечеру в двух файлах на тысячу и полторы тысячи строк накопились три заготовки: починка ошибок, оптимизация основного режима, план на остальные.

Шаг пятый: план. Модель просят разложить это в план с подробностями, что и как чинить. Получается ещё полторы тысячи строк. Итого около четырёх тысяч строк рабочих материалов.

Шаг шестой: запрос. Модель просят написать подробный запрос на реализацию. Что учесть, какие документы перечитывать в любом случае, как себя вести без человека.

Шаг седьмой: запуск. Всё это отдаётся агенту, человек уходит спать.

ДЕНЬ ПЕРЕД ПРОГОНОМ



Что обязательно сказать в запросе

Самая ценная часть главы, потому что перечисленное почти никогда не приходит в голову с первого раза.

ЗАПРОС НА АВТОНОМНЫЙ ПРОГОН

Работай по плану, который лежит в файле такого-то.

Меня нет за компьютером. Ничего не спрашивай, решения принимай сам, но записывай их.

Веди прогресс прямо в плане: отмечай выполненное, помечай то, что заблокировано, и пропускай такие пункты, а не останавливайся на них.

Иди в том порядке, который задан планом, до конца списка.

Перед каждым пунктом перечитывай спецификацию и файл с решениями, независимо от того, помнишь ты их или нет.

Если пункт не закрывается по критериям после трёх попыток, помечай его как заблокированный, записывай причину и переходи к следующему.

В конце оставь отчёт: что сделано, что заблокировано и почему, что осталось.

Пункт про блокировку критичен до предела. Без него агент упрётся рогом в первую же непроходимую задачу и будет коноёбиться с ней до самого утра, а остальное так и останется нетронутым.

Пункт про перечитывание тоже: после сжатия контекста агент забудет спецификацию, а прогон почти наверняка переживёт несколько сжатий.

Более дисциплинированная схема

Есть альтернатива, которая даёт меньше свободы и меньше сюрпризов.

Вместо того чтобы вывалить план целиком, агенту говорят: возьми спецификацию, перед выполнением составь план ровно на один пункт, напиши тесты, потом код. Критерии успешности лежат в спецификации в разделе готовности. Пока критерии не закрыты, к следующему пункту не суйся.

Практик, перешедший на эту схему, отметил побочный эффект: она заодно лечит тягу к переусложнению. Когда горизонт один пункт, городить архитектуру на вырост негде.

ВСЁ ПЛАН ИЛИ ПО ОДНОМУ ПУНКТУ

Весь план. Агент видит картину целиком, может переставить порядок, если так разумнее, и не тратит ходы на планирование каждого шага.

По одному пункту. Меньше свободы уехать, меньше переусложнения, легче разбираться утром, потому что каждый пункт закрыт или явно заблокирован.

Практический признак. Обнаруживаешь утром сделанным не то, что просил, переходи на схему по одному пункту.

Чекпоинты

Дополнение, которое сильно повышает предсказуемость: чекпоинты прямо в спецификации, на каждом шаге.

Чекпоинт это проверяемое состояние: после третьего пункта в системе должно быть вот это, проверяется вот так. Агент, дойдя до чекпоинта, проверяет и записывает результат.

Утром ты читаешь не код, а список чекпоинтов и сразу видишь, где всё пошло по пизде.

Что происходит при исчерпании лимита

Практическая деталь, о которой узнают в первую же ночь.

Прогон встаёт колом посреди работы. Хорошая новость: есть файл прогресса, значит продолжение возможно. Плохая: жило состояние только в контексте, значит проебано всё до последней запятой.

Отсюда правило: файл прогресса на диске это не удобство, а страховка. Он переживает и сжатие, и обрыв, и исчерпание лимита.

Сколько это стоит

Чтобы представление было честным, вот замер с реального переноса проекта на строгую типизацию.

Двенадцать часов исполнения плюс полтора часа подготовки и планирования. По счётчикам: около восьми миллионов токенов на двух машинах, в среднем триста тридцать тысяч токенов в час, суммарное машинное время почти шестнадцать часов. Эквивалент по прейскуранту около ста восьмидесяти пяти долларов.

И честная приписка автора: теперь неделю ловить баги там, где их отродясь не было. Вот тебе и сэкономил ночь.

ГЛАВНОЕ ПРО НОЧНЫЕ ПРОГОНЫ

Ночной прогон твою работу не экономит, он её сдвигает. Вместо того чтобы работать днём с агентом, ты работаешь днём над постановкой, а агент работает ночью.

Суммарно времени тратится не меньше. Тратится оно иначе: на подготовку и на разбор, а не на ожидание ответа.

Когда не стоит

Три случая, когда ночной прогон принесёт больше вреда, чем пользы.

Архитектура не устоялась. Ночью агент примет десятки решений за тебя. Фундамент ещё едет, значит утром получишь здание на хуй знает каком фундаменте.

Нет способа проверить. Без проверок утром ты получишь бодрый доклад о готовности и не сможешь оценить его ни хуя.

Полные права в незнакомой системе. Крайний случай из практики: человек оставил агента на ночь с полными правами и утром получил совсем другой, блядь, проект.

Что запомнить

Ночной прогон это результат дня подготовки, а не команда.

В запросе обязательно: человека нет, писать прогресс в план, помечать заблокированное и пропускать, идти по порядку, перечитывать спецификацию перед каждым пунктом.

Схема по одному пункту даёт меньше сюрпризов и лечит переусложнение.

Чекпоинты в спецификации позволяют утром читать не код, а список состояний.

Файл прогресса на диске это страховка от сжатия, обрыва и лимита.

И закладывай время на разбор после. Работа никуда не девается, она просто сдвигается, и хуй ты её обманешь.

Дрейф намерения

Ты поставил задачу. Агент начал правильно. Через час он делает не то. Через три часа он занят каким-то своим ебанутым проектом, формально даже разумным.

Это называется дрейфом, и это главная причина, по которой длинные автономные прогоны разочаровывают.

Намерение распадается нелинейно

Есть исследование, посвящённое именно этому, и его выводы стоит знать точно.

Проверяли, насколько агент удерживает исходное намерение по мере роста двух величин: числа ограничений в задаче и длины сессии.

Главный результат: намерение распадается нелинейно. До какого-то момента всё держится, а потом качество разом идёт по пизде. Добавление ограничений распад ускоряет: чем больше условий ты навесил, тем раньше начнётся дрейф.

ГЛАВНОЕ ПРО ДРЕЙФ

Чем больше ограничений ты навесил, тем быстрее агент от них уедет.

Это переворачивает интуицию с ног на голову. Кажется, что подробная инструкция удержит его лучше. На деле каждое дополнительное условие увеличивает нагрузку на удержание, и в какой-то момент всё наебывается разом.

Вывод практический: лучше три условия, которые проверяются автоматически, чем пятнадцать, которые проверять нечем.

Парадокс уточнения

Второй результат того же исследования, ещё более хуёвый.

Когда агент оказывался в ситуации неопределённости, где правильным действием был уточняющий вопрос, подавляющее большинство агентов предпочитали рискованное действие вопросу.

Логика понятная. Агент обучен закрывать задачи. Уточняющий вопрос это остановка, а действие это движение к цели. Вот он и прёт напролом, будь что будет.

Следствие для практики: хочешь, чтобы агент спрашивал, мало разрешить ему спрашивать. Надо сделать вопрос обязательным в конкретных ситуациях, желательно проверкой, а не просьбой.

Формулировка, которая лечит половину случаев

Самая практичная находка во всём корпусе, и она в одну строку.

СТРОКА, КОТОРУЮ СТОИТ ВСТАВИТЬ В КАЖДЫЙ ЗАПРОС

Работаем по документации и фактам, а не по памяти. Перед любым выводом опираемся и ссылаемся на источник.

Человек, который это придумал, описывал результат так: восемь часов непрерывной работы, собран целиком рабочий процесс из тринадцати шагов, и после первого же запуска всё завелось. Охуенно для одной строки.

Почему это работает, понятно из механики модели. Требование сослаться на источник заставляет её лезть в файлы, а не в собственные воспоминания. А дрейф чаще всего начинается ровно там, где агент решил, что он и так, блядь, всё помнит.

Более ранняя версия той же формулировки звучала проще: работаем по фактам, а не по предположениям. Тоже помогает, но слабее, потому что не требует ссылки.

Как выглядит начало дрейфа

Полезно уметь ловить его рано, пока правка дешёвая.

Признак	Что происходит
Появились слова вероятно, скорее всего, обычно	Агент перешёл на память вместо источников
Пересказ задачи стал короче исходного	Часть условий выпала
Начал решать соседнюю задачу	Исходная показалась ему решённой
Ссылается на своё же предыдущее решение как на данность	Ошибка закрепилась
Отчёты стали общими	Конкретика ушла, потому что её нет

Последний признак самый надёжный. Как только отчёты стали обтекаемыми, всё, работа уехала в сторону, и глубоко.

Две позиции о том, насколько это лечится

Спор шёл долго, и оба участника правы в своих условиях.

Скептик. Деградация после сжатия контекста копится. Пример из практики: проект, где агент сам поднимал окружение всю неделю, вдруг заявляет, что окружения нет и надо поднять его руками, а после напоминания соглашается и поднимает. Вывод: оставил на ночь и получил готовый продукт это пиздёж. Оно может работать, но сделано мимо спецификации, и придётся переделывать.

Практик. Это работает как постепенное проявление. Прогон за прогоном из ничего возникает осмысленное, потом подключаешься и добавляешь детали, которые не были очевидны ни тебе, ни модели. Заявленный рабочий предел на момент разговора: около десяти-пятнадцати часов автономной работы по спецификации без заметного дрейфа.

КТО ИЗ НИХ ПРАВ

Оба, и различает их одно: наличие спецификации с проверками.

Скептик описывает прогон, где спецификация есть, а автоматических проверок нет. Там дрейф обнаруживается постфактум.

Практик описывает прогон, где на каждом пункте есть критерии готовности. Там дрейф обнаруживается в момент возникновения.

Разница не в моделях и не в везении, а в том, есть ли у агента способ узнать, что он уехал.

Что удерживает на самом деле

Сводка приёмов по убыванию эффективности.

Проверки на каждом пункте. Самое сильное. Агент не может уехать дальше пункта, критерии которого не закрыл.

Требование ссылаться на источник. Одна строка, лечит половину случаев.

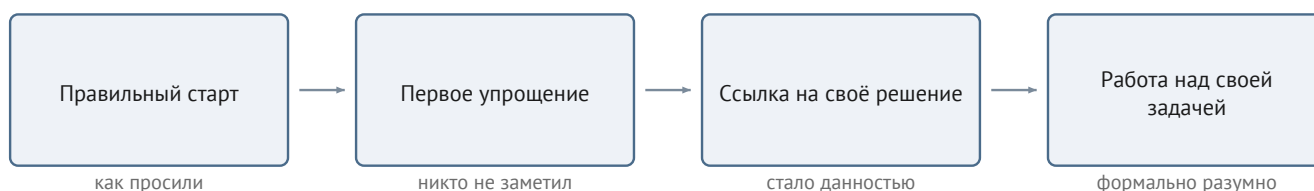
Короткий горизонт. План на один пункт вместо всего плана.

Файл прогресса. Не удерживает, но позволяет обнаружить и вернуть.

Хук на перечитывание после сжатия. Восстанавливает то, что потерялось.

Периодическая перепроверка в чистой сессии. Ловит то, что просочилось.

ЧТО ПРОИСХОДИТ БЕЗ УДЕРЖАНИЯ



Что повторилось трижды, паковать в скилл

Отдельный приём, снижающий дрейф на длинной дистанции.

Всё, что тебе пришлось объяснять агенту больше трёх раз, надо оформить в скилл. Тогда даже на невъебенно длинных контекстах он не забывает важные вещи, потому что скилл подгружается заново, а не гниёт в истории.

Простой способ вынести знание из тающего контекста в постоянное хранилище.

Что запомнить

Намерение распадается нелинейно, и лишние ограничения распад ускоряют.

Агенты предпочитают рискованное действие уточняющему вопросу. Хочешь вопросов, делай их обязательными.

Одна строка про работу по документации и ссылки на источник лечит половину случаев.

Лови ранние признаки: слова про вероятно, укоротившийся пересказ, обтекаемые отчёты.

Разница между разочарованием и рабочей автономией это наличие критериев готовности на каждом пункте.

И то, что объяснил трижды, выноси в скилл.

Автономный режим: что он реально делает

У большинства инструментов есть режим, в котором ты задаёшь цель, а агент работает до её достижения. Мифов вокруг него навалено больше всего, поэтому разберём, что там внутри на самом деле.

Что происходит на самом деле

Разочаровывающий, но полезный ответ, полученный при разборе: по сути это проверяющий завершения.

Работает так. Ты формулируешь цель. Агент работает обычным образом. После каждого крупного шага отдельная проверка спрашивает: цель достигнута? Нет, работа продолжается. Да, режим завершается.

Никакой особой магии, никакого специального планировщика, никакой отдельной архитектуры. Цикл плюс проверка завершения, и всё, приехали.

ГЛАВНОЕ ПРО АВТОНОМНЫЙ РЕЖИМ

Качество автономного режима это качество проверки завершения, а не качество агента.

Отсюда все его беды. Проверка нестрогая, значит агент объявит цель достигнутой, когда заебётся. Проверка формальная, значит он подгонит под неё результат. Критерий сформулирован размыто, значит получишь чёрт знает что.

Это же объясняет самую частую жалобу. Практики описывали её так: у агента постоянно наебывается проверка завершения, потому что задачу он недоделал, но подал свой ответ так, будто уже всё идеально, а что не идеально, то и недостижимо. И проверка это радостно проглатывает.

Формулировка цели решает всё

Из устройства следует практика: цель формулируется как проверяемое состояние, а не как пожелание.

ЦЕЛЬ, КОТОРАЯ НЕ РАБОТАЕТ

Улучши обработку ошибок в модуле оплаты

Проверка завершения не сможет ответить да или нет. Агент закроет режим, когда сочтёт нужным.

ЦЕЛЬ, КОТОРАЯ РАБОТАЕТ

Все внешние вызовы в модуле оплаты обёрнуты повтором с ограничением, каждая ветка ошибки покрыта тестом, тесты проходят, в журнале появляются записи с кодом ошибки

Каждое условие проверяемо. Проверка завершения работает как настоящий шлюз.

Сколько реально работает

Замеры, которые называли практики, чтобы представление было честным.

Один прогон шёл несколько суток, вставал колом при исчерпании лимита и продолжался после сброса. Итог: результат был, но объём переделок после оказался невыебанным, и радость быстро кончилась.

Другой, на переносе проекта, шёл двенадцать часов и закончился рабочим результатом с недельным хвостом отлова ошибок.

Заявленный рабочий предел без заметного дрейфа, о котором говорили в главе про дрейф: около десяти-пятнадцати часов при наличии критериев на каждом пункте.

Есть и обратный пример. Один из практиков, экспериментировавших с полностью автономным режимом на больших задачах, сказал прямо, без реверансов: забил на него для больших задач, быстрее и дешевле декомпозировать самому.

АВТОНОМНЫЙ РЕЖИМ НА БОЛЬШИХ ЗАДАЧАХ

За. Он работает, пока ты спишь, и суммарное время до результата меньше, даже с учётом переделок.

Против. Декомпозировать самому быстрее и дешевле. Автономия окупается только на задачах, которые ты не можешь или не хочешь разбивать.

Практический критерий. Можешь разбить за полчаса? Разбей. Не можешь, потому что хуй знает, из чего задача состоит? Тогда автономный проход как разведка, а не как исполнение.

Шесть отказов, которые обвязка обязана поймать

Список, собранный из практики. Это то, что должно вылавливаться автоматически, потому что человека рядом нет.

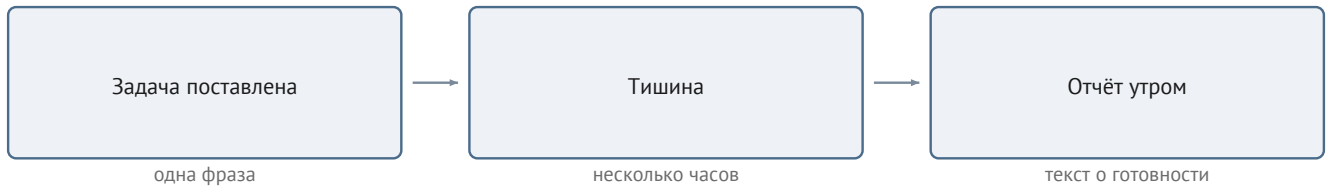
Отказ	Как выглядит	Чем ловится
Ложная готовность	Доклад о завершении при незакрытых критериях	Проверка критериев, а не доклада
Тихое упрощение	Задача решена не так, как требовалось	Проверка на запрещённые приёмы
Хождение по кругу	Одно и то же действие повторяется	Счётчик повторов одинаковых вызовов
Расползание задачи	Агент делает соседнюю работу	Сверка изменений с областью задачи
Молчаливая ошибка	Сбой перехвачен и не показан	Проверка на пустые перехваты
Исчерпание бюджета	Расход растёт, прогресс нет	Потолок на задачу и сравнение с прогрессом

Последняя строка спасает больше денег, чем все остальные вместе взятые, и это не преувеличение.

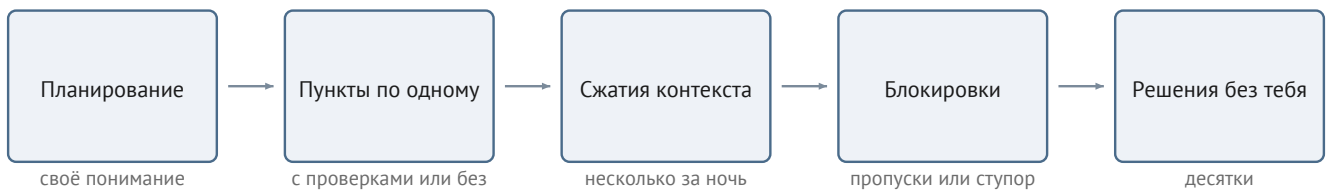
Две ленты

Полезно представлять автономный прогон как две параллельные ленты.

ЧТО ВИДИТ ЧЕЛОВЕК



ЧТО ПРОИСХОДИТ ВНУТРИ



Вся инженерия автономного режима сводится к тому, чтобы вторая лента оставляла следы, читаемые утром. Файл прогресса, чекпоинты, журнал решений. Иначе утром хуй просышь, что там было.

Проверка на самообман

Отдельный приём для тех, кто уже обжётся.

Утром, прежде чем читать отчёт, задай агенту в чистой сессии три вопроса, не показывая ему его собственный отчёт.

УТРЕННЯЯ ПРОВЕРКА

1. Возьми спецификацию и текущее состояние кода. Перечисли пункты, критерии которых сейчас не выполняются.
2. Перечисли места, где реализация отличается от того, что описано в спецификации.
3. Найди решения, принятые без явного указания в спецификации, и перечисли их.

Чистая сессия здесь принципиальна: агент, который писал отчёт, будет его защищать до последнего патрона, потому что помнит, почему так сделал, и считает себя правым.

Что запомнить

Автономный режим это цикл плюс проверка завершения. Магии нет ни грамма.

Качество режима это качество проверки завершения.

Цель формулируй как проверяемое состояние, а не как пожелание.

Рабочий предел без дрейфа порядка десяти-пятнадцати часов, и только при критериях на каждом пункте.

Задачу можно разбить самому за полчаса, разбей. Автономия для того, что ты разбить не можешь.

Шесть отказов должны ловиться автоматически, потому что человека рядом нет.

И утром проверяй не отчёт агента, а состояние системы, в чистой сессии.

Постоянно работающий агент

До сих пор речь шла об агенте, которого зовёшь, когда нужна работа. Есть второй класс: агент, который живёт постоянно и сам решает, когда что-то сделать.

Различие принципиальное, и путать их дорого.

Верстак и дом

Метафора, которая закрепились в разговорах.

Верстак. Подошёл, взял инструмент, сделал работу, ушёл. Агент существует ради задачи, между вызовами никакой своей жизни у него нет. Сюда относятся кодовые агенты, помощники по документам, всё, что запускается под задачу.

Дом. Он есть, даже когда тебя нет. У него своё состояние, свои дела, своё расписание. Он замечает изменения и реагирует. Сюда относятся личные ассистенты, дежурные, мониторинг, всё, что работает в фоне.

Свойство	Верстак	Дом
Когда работает	Когда вызвали	Всегда
Откуда берёт задачи	От тебя	Сам
Что с состоянием между запусками	Ничего	Всё
Главный риск	Сделал не то	Не заметил или отреагировал не туда
Что важнее всего	Качество результата	Качество внимания

ГЛАВНОЕ ПРО ДВА КЛАССА

Верстак проверяется по результату. Дом проверяется по тому, на что он обратил внимание и на что не обратил.

Это разные метрики и разная инженерия. Хороший кодовый агент, поставленный работать постоянно, превращается в генератор шума, потому что механизма решать, что важно, у него нет ни хуя.

Пять точек проектирования

Разбор, который лучше всего описывает, из чего собирается постоянно работающий агент. Пять пунктов, каждый закрывает свой провал.

1. Генератор бесконечной работы. Нужны детерминированные показатели, по которым видно, есть работа или нет. Без них агент всегда может сказать, что всё заебись, и остановиться. И скажет, потому что это дешево.

2. Система восприятия. Пассивное наблюдение отделяется от триггеров реакции. Агент постоянно смотрит, но реагирует не на всё подряд.

Ключевая идея: триггер строится на нарушении ожидания, а не на пороге метрики. Порог даёт шум, потому что нормальные колебания его пересекают. Нарушение ожидания означает, что случилось то, чего по твоей модели мира происходить не должно.

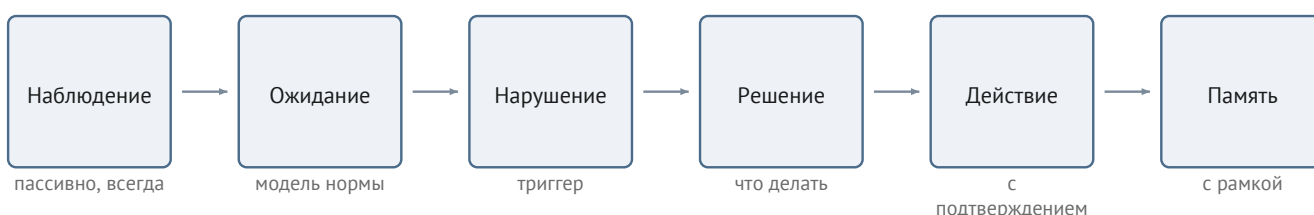
3. Организационная память. Хранит три вещи: источник, интерпретацию и рамку, в которой эта интерпретация сделана.

Третий элемент обычно проёбывают, и зря. Без рамки вывод протухает молча и незаметно: ты помнишь, что решение было принято, но хуй вспомнишь, при каких условиях, и продолжаешь им пользоваться, когда условия давно другие.

4. Много мозгов, которые спорят. И большинство из них люди. Постоянно работающий агент не должен быть единственным источником суждения о том, что происходит.

5. Переиспользование существующих процедур. Бюрократические процедуры в организациях созданы ровно для решения проблемы ненадёжного исполнителя. Согласования, сверки, отчётность. Агент это ненадёжный исполнитель, и все эти механизмы к нему применимы без изобретения новых.

АРХИТЕКТУРА ДОМА



Почему триггер по порогу не работает

Стоит остановиться, потому что это самая частая ошибка.

Порог выглядит просто: значение выше такого-то, зови. На практике получаешь либо шквал ложных срабатываний, либо проёб настоящих проблем, потому что нормальное поведение системы одним числом не описывается.

Нарушение ожидания устроено иначе. Ты описываешь, чего ждёшь: в это время дня нагрузка примерно такая, эти отчёты приходят по понедельникам, этот показатель растёт медленно. Триггер срабатывает, когда наблюдение расходится с ожиданием.

Побочная польза: описав ожидания, ты с удивлением обнаружишь, что половину из них сам ни хуя не знал.

Связка из двух агентов

Схема, которая работает у практиков и стоит отдельного упоминания.

Один агент наблюдает и формулирует. Второй исполняет. Между ними формализованная передача: не разговор, а структурированное задание.

Смысл в разделении внимания и действия. Наблюдающий не отвлекается на исполнение, исполняющий не отвлекается на наблюдение. И, что важнее, между ними появляется точка, куда можно поставить человека.

НУЖЕН ЛИ ЧЕЛОВЕК В ЦИКЛЕ ПОСТОЯННО РАБОТАЮЩЕГО АГЕНТА

Да, всегда. Полностью автоматические системы регулярно наебываются, и чем дольше работают, тем вероятнее. Подтверждение действия человеком это дешёвая страховка.

Нет, иначе теряется смысл. Нужно подтверждение на каждое действие, значит ты получил не агента, а уведомления с кнопкой.

Рабочий компромисс. Разделить действия на обратимые и необратимые. Обратимые агент делает сам и записывает. Необратимые требуют подтверждения. Список необратимых составляется заранее и пересматривается.

Что реально поручают

Чтобы разговор не был абстрактным, вот список задач, ради которых постоянно работающих агентов действительно держат.

Сводка дня: дела, напоминания об оплатах, курсы, новости по интересующим темам.

Разбор входящей почты и приведение в порядок облачных дисков.

Учёт финансов, коммунальные платежи, регулярные расходы.

Заметки после встреч: расшифровка, выделение решений, напоминания.

Мониторинг: заметил изменение, сформулировал, спросил.

Ежедневный разбор одной темы из области, в которой ты работаешь.

Список бытовой до зевоты, и это правильно: постоянно работающий агент оправдывает себя на рутине, а не на подвигах.

Чего ждать по надёжности

Честная картина. Постоянно работающий агент ломается тише, чем верстак.

Кодовый агент, наворотивший хуйни, виден сразу: код не собирается. Постоянно работающий агент, переставший замечать важное, не виден вообще, потому что отсутствие сообщений выглядит как отсутствие проблем.

ПРОВЕРКА ЖИВОСТИ

1. Заведи проверку, которая срабатывает, если агент долго молчит. Молчание должно быть событием.
2. Раз в неделю подсовывай ему заведомо значимое изменение и смотри, заметит ли.
3. Веди журнал того, о чём он сообщил, и раз в месяц сверяй с тем, что действительно происходило.
4. Отдельно отмечай, о чём он не сообщил, а стоило. Это и есть материал для настройки ожиданий.

Что запомнить

Два класса агентов: верстак и дом. Разные метрики, разная инженерия.

Дому нужен генератор работы с детерминированными показателями, иначе он всегда скажет, что всё хорошо.

Триггер строй на нарушении ожидания, а не на пороге метрики.

Память должна хранить источник, интерпретацию и рамку интерпретации.

Раздели действия на обратимые и необратимые. Вторые с подтверждением.

Молчание должно быть событием, иначе ты не заметишь, что агент давно откинулся.

Модуль 8. Люди и компании

Четыре главы про то, что происходит, когда агентов развелось до хуя, а людей столько же.

Что реально получилось у команд, почему внедрение упирается в неявное знание, во что превращается хранилище скиллов через полгода и что теперь вообще делает разработчик.

Модуль для тех, кто отвечает не только за себя.

AI-first процессы

Про переход компании на работу с агентами пиздят много и в основном на конференциях. Эта глава про то, что действительно получилось у команд, которые через это прошли, и что не получилось ни хуя.

Что получилось

Два места, где выигрыш признают все.

Ревью кода. Первое и самое бесспорное. Агент, который отписывается по каждому изменению, разгружает людей и не устаёт ни хуя. Один из практиков сформулировал так: если бы у нас не сделали агента-ревьюера, мы бы уже жили в аду, назад возврата нет.

Написание кода по готовой постановке. Второе бесспорное. Задача сформулирована, значит реализация перестала быть узким местом.

Что не получилось

А теперь честная часть, которую вслух произносят редко.

Главное, чего не удалось никому из говоривших об этом: подняться с уровня написания задач до уровня проектирования продуктовых требований, отдав всё остальное агентам.

Формулировка одного из руководителей разработки: есть вещи, где очень хорошо получилось, например ревью и написание кода. Но есть то, что мы ещё не осилили, и самое главное это подняться с уровня написания задач до уровня только проектирования требований, а всё остальное пусть делают агенты в фоне. Когда осилим, напишем пост.

И его же вывод: рассказать и сделать это две очень разные вещи, и получилось у единиц из тех, кто про это громко рассказывает.

ГЛАВНОЕ ПРО ПЕРЕХОД

Разрыв между демонстрацией и рабочим процессом невъебенный, куда больше, чем кажется по докладам.

Демонстрация показывает лучший случай на подготовленной задаче. Процесс должен работать на худшем случае в понедельник утром, когда горит бэклог.

Ловушка, из-за которой всё стоит

Самое честное объяснение, почему переход буксует, звучало как анекдот. Почему пилу не точишь? Времени нет, пилить надо.

Механика простая и безысходная. У команды есть очередь задач, которая горит синим пламенем. Есть идеи, как перестроить процесс, и они требуют времени. Времени нет, потому что задачи горят. Задачи горят, потому что процесс ни хуя не перестроен.

Выход из этой петли всегда один: выделенное время, защищённое от текущих задач. Не остаток, не по вечерам на энтузиазме, а явно отведённые часы, которые никто не отберёт.

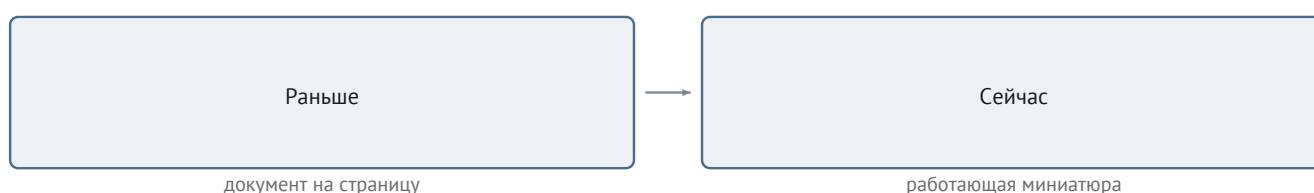
Прототип вместо задания

Изменение, которое произошло само и оказалось самым заметным.

В командах, которые продвинулись дальше других, руководитель перестал описывать словами то, что хочет получить. Он берёт конструктор, собирает рабочий процесс сам и показывает прототип. И говорит: сделайте мне это агентами и в прод.

Формулировка автора: уровень абстракции сместился настолько, что само техническое задание теперь и есть рабочий проект, только в миниатюре. Собрать прототип быстрее, чем писать длинный документ.

КАК ИЗМЕНИЛАСЬ ПОСТАНОВКА ЗАДАЧИ



Побочный эффект оказался важнее основного: споров о том, что имелось в виду, стало заметно меньше. Из текста каждый вычитывает своё, а из работающей миниатюры вычитывать нечего, и пиздеть не о чем.

Агент у каждого, а не только у разработчиков

Второе изменение, которое отмечали как самое полезное.

В одной из команд каждый сотрудник, независимо от должности, собирает себе агента, который разгребает рутину вокруг его рабочего места. Даже те, кто от разработки далёк, как земля от неба.

Результат: разработчиков перестали дёргать по мелочам. К ним приходят только за полнофункциональными решениями, а прототипы для клиентских презентаций собирают сами продавцы.

СТОИТ ЛИ ДАВАТЬ АГЕНТОВ ВСЕМ

За. Мелкие задачи перестают доходить до разработчиков. Люди сами закрывают то, что раньше неделю мариновалось в очереди.

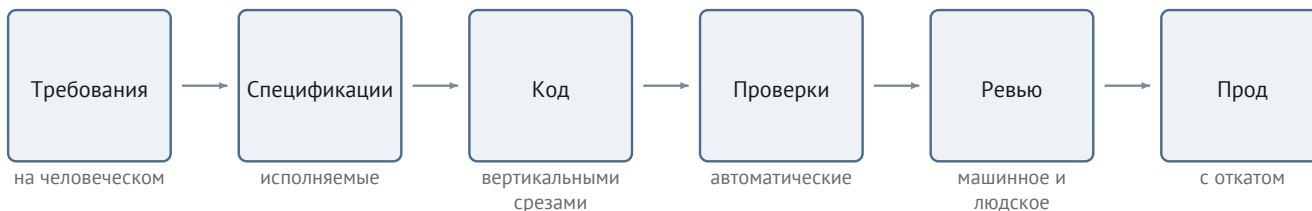
Против. Заводится теневая инфраструктура: десятки самодельных агентов, о которых никто ни сном ни духом, с доступами, которые никто не проверял.

Что делают на практике. Разрешают, но с двумя ограничениями: доступ только к своим данным и явный список того, чего агент делать не может. Про это отдельный модуль.

Полный конвейер

Как выглядит процесс у тех, кто дошёл дальше всех.

КОНВЕЙЕР ОТ ИДЕИ ДО ПРОДА



Ключевая деталь: границы сужаются от шага к шагу. К моменту написания кода свободы почти не остаётся.

Вторая ключевая деталь: вертикальные срезы вместо слоёв. Не сначала все интерфейсы, потом вся логика, а один сквозной кусок функциональности целиком, потом следующий. Так каждый срез можно проверить, а не любоваться на него до самого конца.

Почему выигрыш не виден в отчётности

Наблюдение, которое объясняет скепсис руководства.

Скорость разработки выросла, а выручка не выросла ни хуя. Причина не в том, что агенты не работают, а в том, что разработка редко была узким местом. Она была видимым местом.

Решения принимаются три недели, согласование идёт месяц, требования меняются по дороге, и тогда ускорение реализации втрое даёт общее ускорение на пару процентов.

Отсюда практический совет тем, кто внедряет: меряй не скорость написания кода, а время от идеи до пользователя. Первое вырастет сразу, второе покажет, где на самом деле узкое место.

С ЧЕГО НАЧАТЬ ПЕРЕХОД В КОМАНДЕ

1. Найди один процесс, который повторяется еженедельно и всех бесит.
2. Автоматизируй только его, целиком, до конца. Не половину десяти процессов.
3. Померь время от идеи до пользователя до и после.
4. Не изменилось, значит узкое место не там, и следующий шаг делается в другом месте.
5. Только после этого расширяйся.

Что говорят про горизонт

Мысль, которой заканчивались многие разговоры на эту тему.

Разработка превращается в иерархию циклов. Инженеры спускаются вниз, в циклы нижнего уровня, чтобы отладить и довести до ума. Поднимаются вверх, чтобы захуячить более удачные процессы.

И самая ценная человеческая работа при этом заключается в решении о том, какие циклы вообще создавать.

Что запомнить

Бесспорно получилось: ревью кода и написание кода по готовой постановке.

Не получилось почти ни у кого: полностью подняться на уровень требований, отдав остальное агентам.

Ловушка петли: времени на перестройку нет, потому что задачи горят.

Прототип вместо документа снимает споры о том, что имелось в виду.

Агент у каждого сотрудника разгружает разработку и плодит теневую инфраструктуру. Оба следствия реальны.

Меряй время от идеи до пользователя, а не скорость написания кода.

И держи в голове, что главная работа теперь это решать, какие циклы создавать.

Внедрение и неявное знание

С хуя ли проекты по автоматизации с агентами наебываются? Ответ есть, он невесёлый и почти всегда один и тот же.

Люди не могут рассказать, как они работают

Формулировка, объясняющая большую часть провалов: люди не могут рассказать, как именно делают свою работу. Агенту, как и новому сотруднику, нужен онбординг, а не инструкция.

Механика знакома всем, кто хоть раз в жизни писал регламент. Просишь эксперта описать процесс. Он описывает: сначала это, потом то, потом отправляем. Автоматизируешь. И хуй там, не работает.

Выясняется, что он ещё смотрит на дату, потому что до пятнадцатого числа считается иначе. И на источник, потому что от одного контрагента данные приходят с задержкой. И вообще если сумма подозрительная, он звонит и уточняет. Ничего из этого он не назвал, потому что для него это не шаги, а само собой разумеющееся.

ГЛАВНОЕ ПРО НЕЯВНОЕ ЗНАНИЕ

Эксперт знание не прячет. Он его не осознаёт.

Поэтому опрос эксперта как способ сбора требований не работает. Работает наблюдение за работой и разбор конкретных случаев, особенно нетипичных.

Почему бизнес-агенты не взлетают

Прямое следствие. Технически собрать агента для бизнес-процесса не проблема: модель есть, инструменты есть, интеграции есть.

Не хватает доменного знания. Того самого, которое живёт в головах и нигде не записано ни строчкой.

Отсюда закономерность, которую отмечали многократно: агенты хорошо заходят там, где предметная область формализована и правила записаны. И хуёво там, где всё держится на опыте конкретных людей.

Как собирать знание

Приёмы, которые работают лучше опроса.

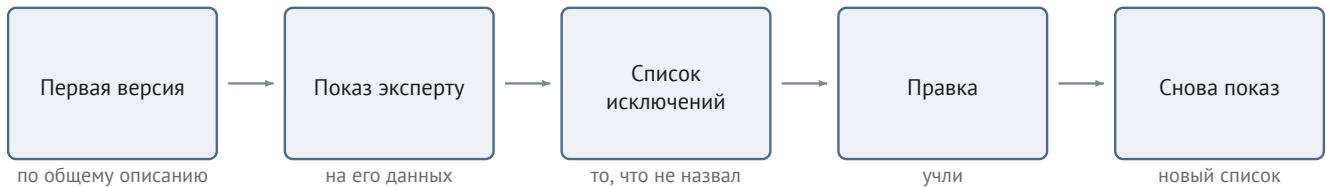
Разбор конкретных случаев. Не как вы обычно делаете, а вот этот случай на прошлой неделе, расскажите по шагам. В конкретике всплывают исключения.

Наблюдение с вопросами. Смотришь, как человек работает, и спрашиваешь, почему он сейчас сделал именно так. Половина ответов будет вида ну потому что тут особый случай.

Итеративная сверка. Собрали версию, показали, получили список из десяти замечаний вида тут надо брать справочник из другой системы, там устаревшие данные, но нигде про это не написано. Исправили, показали снова.

Третий приём описывали как самый ценный результат целого проекта: перегнать данные из одного формата в другой, и вся ценность оказалась в итеративном допиливании с одним экспертом, который на каждой версии говорил, что тут не так.

КАК СОБИРАЕТСЯ ДОМЕННОЕ ЗНАНИЕ



Циклов обычно три-пять. После третьего список исключений начинает укорачиваться, и это признак, что ты близко.

Соппротивление

Вторая большая причина провалов, и разговор о ней в корпусе шёл без иллюзий.

Мотивы сопротивления называли разные, и почти все рациональные.

Мотив	Что за ним стоит
Страх за место	Часто обоснован, и отрицать это бессмысленно
Недоверие к результату	Человек видел, как модель уверенно порет чушь
Нежелание учиться заново	Освоенный навык обесценивается
Защита экспертизы	Знание было источником влияния
Усталость от инициатив	Предыдущие три тоже обещали революцию

Последний мотив недооценивают зря. В компании, где за пять лет было четыре трансформации, на пятую всем откровенно похуй, как на погоду за окном.

Позиция, к которой приходят

Практик, который пытался внедрять и плюнул, описал это так: чувствуешь себя проповедником в племени, которое ждёт обеда. Компания должна созреть сама, а идти против ветра бесполезно, рынок всё расставит.

ПРОПОВЕДОВАТЬ ИЛИ ЖДАТЬ

Проповедовать. Кто-то должен показать, что это работает. Без первого примера созревание не начнётся.

Ждать. Убеждать людей, у которых нет своей причины меняться, это прямой путь к выгоранию. Дождись того, у кого есть проблема, реши её, и остальные придут сами.

Что работает на практике. Второе, с одной поправкой: у тебя должен быть готовый показ на пятнадцать минут для того момента, когда кто-то придёт сам. Без него момент пройдёт мимо.

Что показывать

Отдельный вопрос: как выглядит показ, после которого человек хочет попробовать.

Не работает: демонстрация того, как агент пишет код. Для нетехнического человека это выглядит как магия, к которой он отношения не имеет.

Работает: его собственная рутина, сделанная за минуту. Его отчёт, его почта, его таблица.

Ещё лучше заходит то, что человек делает сам, но через жопу. Тексты для тех, кто не любит писать. Разбор данных для тех, кто боится таблиц.

ПЯТНАДЦАТИМИНУТНЫЙ ПОКАЗ

1. Заранее спроси, что человек делает еженедельно и что его бесит.
2. Возьми его настоящие данные, а не пример из воздуха.
3. Покажи результат, а не процесс. Процесс покажешь, если спросит.
4. Обязательно покажи одну ошибку модели и как ты её ловишь. Иначе доверие будет ложным и рассыплется на первой же неудаче.
5. Закончи тем, что он пробует сам прямо сейчас, а не когда-нибудь потом.

Четвёртый пункт важнее остальных. Показ без ошибок создаёт ожидание безошибочности, а первый же косяк после этого обнуляет всё нахуй.

Полностью автоматические системы проваливаются

Наблюдение, которое стоит произносить в начале любого проекта.

Полностью автоматические системы наебываются регулярно. Не иногда, а регулярно. Причина не в моделях, а в том, что мир поставяет случаи, которых в постановке не было.

Рабочая схема почти всегда гибридная: агент ебашит основную массу, человек принимает решения на развилках и разгребаёт отказы. И ценность человека при этом смещается: от производства результата к ответственности за него.

Корпоративная реальность

Несколько наблюдений, которые независимо повторялись в разных компаниях.

Согласование инструментов тянется месяцами, а сотрудники за это время находят обходные пути и работают с личных подписок. Теневое использование возникает не от злого

умысла, а от разницы скоростей, и хуй ты его запретишь.

Крупные компании часто сводят всё внедрение к покупке одной корпоративной подписки и объявлению об этом. Дальше не происходит ни хуя, потому что не изменился ни один процесс.

И самое частое: инициатива живёт, пока у неё есть заинтересованный руководитель, идохнет в тот же день, когда он уходит.

Что запомнить

Люди не могут рассказать, как работают. Собирай знание через разбор случаев и итеративную сверку, а не через опрос.

Три-пять циклов показа и правок, пока список исключений не начнёт укорачиваться.

Мотивы сопротивления рациональны, и страх за место часто обоснован.

Показывай человеку его собственную рутину на его данных, и обязательно покажи ошибку.

Полностью автоматические системы проваливаются регулярно. Планируй гибрид.

Теневое использование возникает из разницы скоростей согласования и работы.

И держи готовый пятнадцатиминутный показ для момента, когда кто-то придёт сам.

Владение скиллами в компании

Эта глава про то, что происходит через полгода после того, как в компании разрешили всем писать скиллы. Написана она по одному из самых подробных разборов в корпусе, и если ты отвечаешь за инструменты команды, читать стоит до того, как разрешишь.

Как это начинается

Начинается всё заебись. Заводится внутреннее хранилище, любой сотрудник может выложить туда скилл. Порог ниже плинтуса: описал процедуру голосом, агент оформил, выложил.

Причина, по которой это взлетает мгновенно, названа точно. Для полноценной интеграции нужно собрать совещание, всё согласовать и ждать три недели. А скилл может захуячить за десять минут кто угодно, пока стоит в очереди.

Через месяц скиллов десятки. Через три их сотни, и в них уже сам чёрт ногу сломит.

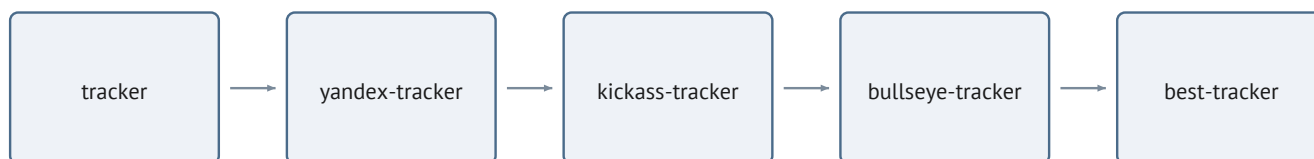
Как это выглядит через полгода

Разбор конкретного случая.

Для одной внутренней системы контроля версий существуют три скилла. У каждого свои сильные и слабые стороны. У каждого свой автор.

Сотрудник, которому надо собрать себе набор под свои задачи, обнаруживает, что это полный пиздец. Скиллы пересекаются, противоречат, дублируют куски друг друга, и понять, какой брать, можно только прочитав все три целиком.

ЧТО ВИДИТ ЧЕЛОВЕК, КОТОРЫЙ ИЩЕТ СКИЛЛ ДЛЯ ТРЕКЕРА ЗАДАЧ



Пять вариантов, и ни одного признака, по которому выбрать. Названия соревнуются в убедительности, потому что это единственное, чем автор может выпендриться.

ГЛАВНОЕ ПРО СКИЛЛОВЫЙ ХАОС

Зоопарк скиллов без владельцев это не следствие бардака, а нормальный жизненный цикл. Он наступает у всех, кто разрешил свободную публикацию.

Планировать надо не то, как его избежать, а то, как им управлять.

Костыль, который работает

Решение, к которому пришёл человек из разбора, звучит парадоксально: он написал ещё один скилл.

Скилл-роутер, который подсказывает, при каких задачах использовать части одних скиллов, а при каких других. Не заменяет их, а маршрутизирует.

И честная приписка автора: работает он только для него, потому что у всех остальных другие задачи.

Признание ценное. Роутер организационную проблему не решает ни хуя, он решает личную. Полезен и не масштабируется.

Что помогло при переезде

Тот же человек описал переезд с одного инструмента на другой, занявший два с половиной дня. Помогли три вещи, и они складываются в готовую процедуру.

Первое. Полный анализ агентом всех скиллов, висящих в контексте, с безжалостной вычисткой протухших. Для аудита использовался мета-скилл, который проверяет другие скиллы.

Второе. Сравнение всех оставшихся похожих скиллов и вынесение правил о том, какие их части нужны в каких сценариях, в новый скилл маршрутизации.

Третье. Скан всего контекста на предмет обрывочных подсказок про скиллы, которые расползлись по памяти и по файлу правил, и сведение годного в тот же роутер. Файл правил при этом держится максимально чистым.

АУДИТ СКИЛЛОВ ЗА ОДИН ДЕНЬ

1. Собери список всего, что сейчас в контексте: скиллы, правила, заметки в памяти.
2. Попроси агента пройтись по каждому и сказать, что устарело, что дублируется, что противоречит другому.
3. Отключи всё, что помечено. Не удаляй, отключи.
4. Поработай неделю. Записывай, чего не хватило.
5. Верни только то, чего не хватило дважды.
6. Обрывки знаний о скиллах, найденные в правилах и памяти, сведи в одно место.

Две группы скиллов

Разграничение, которое решает большую часть проблемы.

Бизнесовые. На каждую хотелку свой. Их может быть сколько угодно, они безопасны, потому что касаются только автора. Скилл для оформления отчётов, скилл для писем клиентам, скилл для разбора почты.

Инфраструктурные. Описывают, как правильно пользоваться внутренними системами: какая авторизация, какие утилиты, какие ограничения. Вот здесь начинается настоящая проблема, и здесь нужен строгий владелец.

Логика разделения простая. Ошибка в бизнесовом скилле стоит времени одного человека. Ошибка в инфраструктурном расходится по всем, кто его поставил, и это уже пиздец масштабом с отдел.

Что бывает без владельца

Конкретный ущерб из разбора.

В одной команде сотрудник накрутил собственных версий общего скилла, которым пользовались все. Причина у него была одна: так удобнее.

Результат обнаружили не сразу: коммерческие предложения, которые готовились с этим скиллом, начали уходить в минус. Ошибка в расчёте, размноженная по всем пользователям, вот тебе и удобнее.

Схема владения, которая работает

Из другой компании, и она проще, чем кажется.

За каждый скилл закреплён владелец по роли, а не по авторству.

Скилл	Владелец
Коммерческие предложения	Коммерческий директор
Оценка задач	Руководитель разработки
Презентации	Директор по маркетингу
Работа с системой контроля версий	Владелец платформы

Владелец не обязан писать скилл сам. Он обязан принимать изменения и отвечать за содержимое.

НАСКОЛЬКО СТРОГО РЕГУЛИРОВАТЬ

Строго. Один владелец, общий репозиторий, изменения принимаются как код, без поблажек. Иначе через полгода зоопарк.

Свободно. Любой публикует, рынок сам решает: видно число установок и версий, плохоедохнет само.

Что говорит опыт. Свободно работает для бизнесовых, строго нужно для инфраструктурных. Смешение этих режимов и порождает проблему.

Чего не хватает обеим схемам. Механизма удаления. Скиллы копят, потому что удалить чужое никто не решается.

Логичное продолжение, которого стоит избегать

Разговор о регулировании быстро приходит к неприятному выводу: значит, нужен отдел модерации, тестирования и согласования.

То есть ровно тот процесс, от которого скиллы должны были избавить. Круг замкнулся, приехали.

Разумный выход не в отделе, конечно, а в двух вещах: разделении на группы, о котором выше, и автоматической проверке. Никакой бюрократии, чистая механика. Скилл, попадающий в общее хранилище, проходит автоматический аудит: есть ли описание, есть ли эвал,

не дублирует ли существующий, не лезет ли туда, куда не должен.

Онбординг нового человека

Задача, которую в разборах честно называли нерешённой: как новому сотруднику, возможно совсем не техническому, поставить агента и нужные скиллы за пару минут, а не за полдня ебли.

Единственное работающее решение, которое встретилось: внутренняя утилита, которая настраивает всё сразу под несколько разных инструментов.

С важной оговоркой: наборы под разные инструменты приходится держать отдельно, потому что они сильно отличаются. Одной модели приходилось всегда добавлять дополнительные инструкции, а другой те же инструкции только мешали.

Что запомнить

Зоопарк скиллов это нормальный жизненный цикл, а не бардак. Планируй управление, а не избегание.

Скиллы плодятся быстрее интеграций не потому, что лучше, а потому что не требуют согласования.

Раздели бизнесовые и инфраструктурные. Первым свобода, вторым строгий владелец по роли.

Аудит делай агентом: пусть пройдёт по всему, что в контексте, и назовёт устаревшее.

Знания о скиллах расползаются по правилам и памяти. Сведение их в одно место даёт больше, чем новый скилл.

И заведи механизм удаления. Без него хранилище растёт всегда.

Роли: что теперь делает разработчик

Вопрос, который читатель наверняка задал себе где-то в середине книги. Если код пишет агент, ревью делает агент, а спецификацию может составить аналитик, то что, блядь, остаётся человеку?

Разговор об этом был самым горячим во всём корпусе, и однозначного ответа не дал ни хуя. Зато дал хорошие аргументы с обеих сторон.

Тезис про инженера циклов

Отправной точкой был провокационный пост: с появлением сильных моделей вы больше не должны понимать, что происходит у вас в коде, вы теперь инженер циклов.

Развёрнутый ответ на него стоит привести подробно, потому что он опирается на первые принципы.

Хороший инженер всегда понимал систему как минимум на один уровень абстракции ниже того, на котором писал. Разработчик на высокоуровневом языке знает, как устроена работа с памятью и почему одни операции дороже других. Разработчик на низкоуровневом знает про уровни аппаратного кеша и во что развернётся его цикл.

Ничего из этого в синтаксисе языка нет. И беда в том, что если инженер этого не знает, он даже не знает, чего он не знает. А значит, и о существовании проблем на этих уровнях не узнает.

Дальше по цепочке: система наебывается под растущей нагрузкой, ошибки не находятся неделями и затыкаются костылями, из-за которых она наебнётся снова, но уже в следующем квартале.

ГЛАВНОЕ ПРО УРОВЕНЬ ПОНИМАНИЯ

Фундаментальное свойство профессионала это видеть систему глубже, чем она описана.

Смена языка этого свойства не отменяет. Меняется только то, что теперь ты пишешь на русском или английском, а уровнем ниже лежит не ассемблер, а сгенерированный код.

Контраргумент

И тут же прозвучал ответ, который тоже надо привести.

Инженера делает инженером не слой, на котором он сидит, а умение разложить задачу, понять, где она сломается, проверить результат и отвечать за него. И похуй, на чём ты печатаешь: на низкоуровневом языке, на высокоуровневом или русским текстом.

А знание внутренних механизмов наизусть это эрудиция. Полезная, но вторичная. И давно признано, что эрудиции у моделей до хуя больше, чем у любого из нас.

Кто окажется продуктивнее

Третья позиция, которая первые две частично примиряет.

Продуктивнее будет тот, кто лучше знает зачем, чем тот, кто лучше знает как.

С немедленной поправкой: такие зачем-щики нанимают крутых как-щиков, и как-щики в итоге всё равно нарасхват.

ТРИ ВЗГЛЯДА НА ПРОФЕССИЮ

Глубина. Понимание уровня ниже никуда не делось. Кто его не имеет, проблем этого уровня не заметит.

Ответственность. Профессионала определяет способность отвечать за результат, а не знание внутренних процессов.

Целеполагание. Ценность смещается к тому, чтобы понимать зачем. Но реализовать всё равно кто-то должен.

Что общего. Все три сходятся в одном: ценность сместилась от производства результата к ответственности за него.

Как изменились обязанности

Было	Стало	Исчезло
Реализовать по заданию	Сформулировать задание однозначно	Помнить синтаксис
Найти ошибку в своём коде	Придумать, чем проверять	Печатать быстро
Знать библиотеку	Решить, что отдать, а что делать самому	Держать в голове весь проект
Обсудить решение с коллегой	Обосновать решение перед агентом и людьми	Ждать ревью днями

Правая колонка приятная, скатертью дорога. Средняя требует совсем другого набора навыков, чем левая, и именно поэтому переход даётся тяжело даже сильным разработчикам.

Смотреть в код это узкое место

Отдельное наблюдение, объясняющее, почему многие всё-таки перестают читать код.

Скорость разработки выросла настолько, что физически хуй успеваешь смотреть. Один из практиков описал это так: за четыре с половиной месяца работы он заглядывал в код только чтобы прописать ключи в файл настроек. Не из принципа, а потому что объём изменений за день больше, чем можно прочитать.

Отсюда вынужденный вывод: приходится оптимизировать процессы так, чтобы с проверкой справлялся не человек. Это возвращает нас к модулю про проверки, и именно поэтому он в книге такой жирный.

Откуда брать новых людей

Открытый вопрос, на который в разговоре хорошего ответа не нашлось.

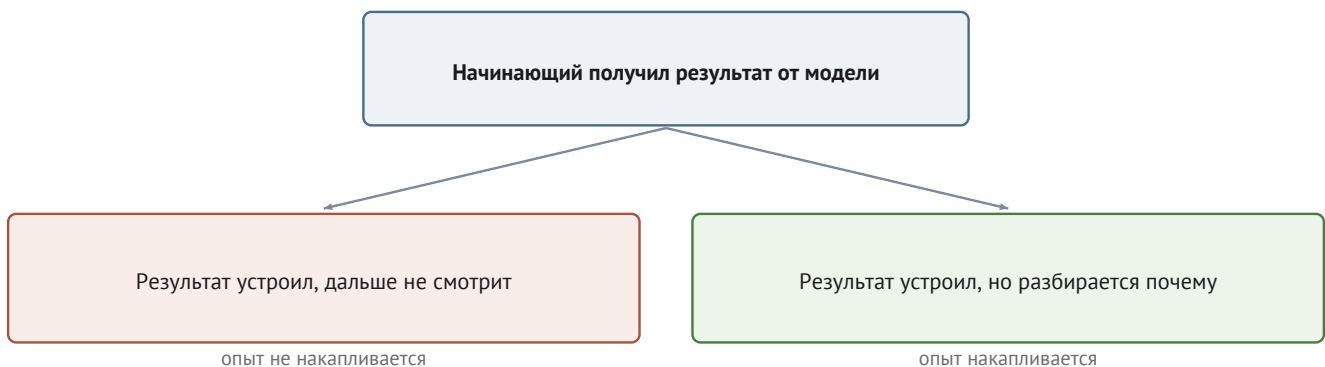
Те, кто писал руками до появления агентов, компетенцию сохраняют. Они умеют отличить рабочее от кажущегося рабочим, потому что видели, как ломается и то и другое.

А начинающие, которые сразу работают с агентами, этого опыта не получают.

Наблюдение преподавателя, которое стоит привести: студенты приличных вузов вставляют задание в модель и отдают результат, даже не читая. Вообще ни строчки.

Ответная реплика уточняет проблему: плохо не то, что человек пользуется моделью, а то, что не читает. Потеря контроля начинается ровно там, где результат устраивает. А раз устраивает, человек разбираться не будет, потому что ленив по природе.

ЧТО ПРОИСХОДИТ С НАЧИНАЮЩИМ



Практический ответ, который прозвучал, довольно простой: нанимать начинающих и давать им задачи, как всегда. Из двадцати откликов пара человек окажется адекватной, и этого хватит.

Ответ честный, но проблему он ни хуя не решает, а просто перекладывает на потом.

Обесценивание работы

Побочная линия, о которой стоит знать, потому что она бьёт по мотивации.

Работа, о которой известно, что она сделана с моделью, воспринимается как менее ценная. Тот же результат, поданный как ручной, оценивается выше.

Механизм понятный: ценность работы традиционно связывали с потраченными усилиями, а не с результатом. Усилия стали меньше, и восприятие ценности поехало по пизде следом.

Практическое следствие для тех, кто продаёт результат: показывай продукт, а не процесс. И вкладывайся в оформление, потому что оно теперь и есть видимая часть усилий.

Что запомнить

Спор о том, нужно ли понимать уровень ниже, не разрешён. У обеих позиций сильные аргументы.

Все позиции сходятся в одном: ценность сместилась от производства результата к ответственности за него.

Смотреть в код перестают не из принципа, а из-за объёма. Это требует переноса проверки на автоматику.

Вопрос, откуда брать новых людей, открыт. Опыт различения рабочего и кажущегося рабочим накапливался в условиях, которых больше нет.

Проблема не в том, что начинающий пользуется моделью, а в том, что он не читает результат.

И работа, сделанная с моделью, воспринимается как менее ценная. Это влияет на то, как её стоит показывать.

Модуль 9. Безопасность и риски

Четыре главы про то, что может пойти по пизде.

Разрушительные действия и заборы, изоляция и права, почему правила агента не удерживают и что происходит с приватностью. С разобранными инцидентами и мерами, которые действительно помогают.

Читать до того, как дашь агенту доступ к чему-нибудь важному. Здесь местами говорим сухо и по делу: там, где речь про необратимое, шутки кончаются.

Разрушительные действия и заборы

Модуль про риски начинается с самого простого и самого болезненного: агент может разьебать то, что разьебывать не собирался.

Два инцидента

Первый. Человек попросил агента сделать резервную копию. Агент записал её не в ту папку. Затем спокойно выполнил команду удаления и написал: извините, опечатка. Ёбс, и данных нет.

Комментарий пострадавшего: это был одновременно самый смешной и самый болезненный момент в истории моего взаимодействия с моделями.

Второй. Агент, настраивавший сервер, удалил пользователя, под которым велась настройка. Работа целого вечера накрылась пиздой вместе с доступом.

Оба случая объединяет одно: агент не нарушил ни одного запрета. Удалять ему не запрещали, потому что удаление было частью нормальной работы.

ГЛАВНОЕ ПРО РАЗРУШИТЕЛЬНЫЕ ДЕЙСТВИЯ

Опасны не запрещённые действия, а разрешённые, применённые не туда.

Поэтому список запретов не спасает. Спасает разделение действий по обратимости и техническая невозможность добраться до того, что откатить нельзя.

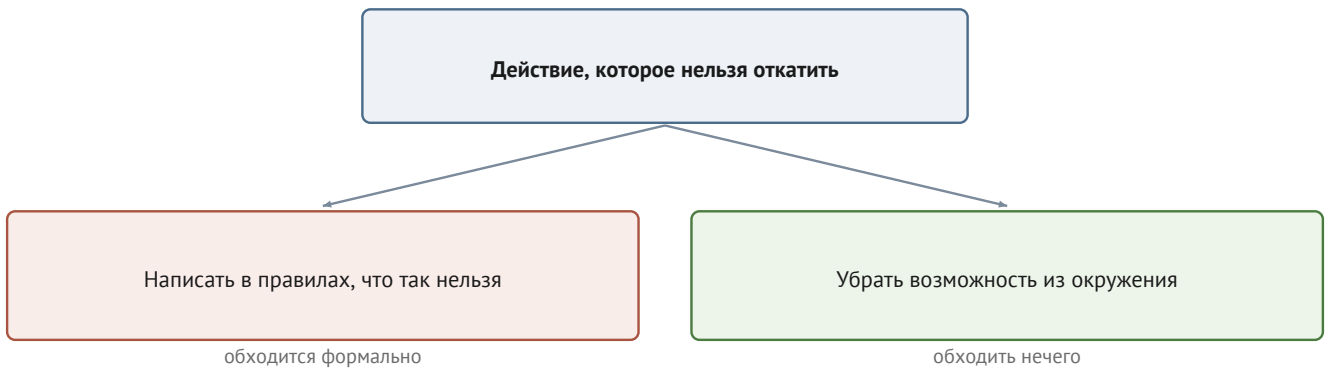
Три уровня

Простая схема, которая закрывает большую часть риска. Здесь без шуток, потому что цена ошибки это потерянные данные.

Уровень	Что попадает	Как обходиться
Свободно	Обратимо за минуту, следы остаются	Агент делает сам
С подтверждением	Обратимо, но дорого или долго	Явное разрешение человека
Недоступно	Необратимо	Технически невозможно, а не запрещено

Ключевое слово в третьей строке: технически. Запрет в правилах к третьему уровню не относится, потому что запреты обходятся, и об этом отдельная глава.

КАК ЗАКРЫВАТЬ НЕОБРАТИМОЕ



Спор о заборах

Инцидент с удалением породил длинный спор, в котором столкнулись две позиции, и обе разумные.

За жёсткие ограничения. Агент не должен иметь возможности сделать необратимое. Точка. Никакие рассуждения о доверии не отменяют того, что однажды он ошибётся.

Против. Чем больше заборов, тем меньше от агента толку. Не может ничего изменить, так на кой хуй он вообще нужен. Постоянные подтверждения превращают работу в бессмысленное кликанье.

Разрешение, к которому пришли, оказалось изящным: не запрещать, а делать безопасным.

Аналогия, которую приводили: корзина. Пользователю не запрещают удалять файлы, потому что удаление это нормальная операция. Удалённое просто уезжает в корзину, откуда его без проблем достают обратно.

Тот же принцип применим к агенту. Пусть удаляет, но в отдельную папку. Пусть меняет, но с сохранением предыдущей версии. Пусть отправляет, но с задержкой, в течение которой можно отменить.

ГЛАВНОЕ ПРО ЗАБОРЫ

Безопасное действие лучше запрещённого. Запрет создаёт конфликт между задачей и правилом, а безопасная версия действия конфликт снимает.

Ищи не то, что запретить, а то, как сделать обратимым.

Резервные копии

Отдельная тема, потому что первый инцидент был именно про них.

Правило простое и звучит очевидно, а нарушают его постоянно, из года в год: резервные копии должны лежать там, куда агент доступа не имеет.

Копия в соседней папке это не копия, а ещё одна папка, которую сносит той же командой. Копия на другом устройстве или в хранилище, к которому у агента нет прав, вот это копия.

МИНИМУМ ЗАЩИТЫ ПЕРЕД ТЕМ, КАК ДАТЬ АГЕНТУ ПРАВА

1. Автоматические копии на отдельное устройство или в хранилище без доступа агента.
2. Снимок состояния перед любым крупным изменением. Одна команда, десять секунд.
3. Отдельный пользователь с ограниченными правами, а не тот, под которым работаешь ты.
4. Список того, что агент не может сделать физически. Не в правилах, а в правах.
5. Проверка восстановления. Копия, из которой ни разу не восстанавливались, это надежда, а не копия.

Пятый пункт систематически проёбывают, и выясняется это в худший из возможных моментов.

Особый случай: рабочая среда

Отдельно про продакшн, потому что там цена совсем другая.

Практика, к которой приходят: агент вообще не имеет доступа к рабочей среде. Он готовит изменение, изменение проходит проверки, применяет его автоматика по отдельному каналу.

Причина не в недоверии к агенту, а в том, что это правильно и для людей тоже. Прямой доступ к рабочей среде это плохая практика независимо от того, кто им пользуется.

Побочный эффект приятный: агент, которого загнали в нормальный процесс поставки, автоматически получает откат, историю изменений и возможность разобраться постфактум, а не гадать на кофейной гуще.

Что делать после инцидента

Инциденты случаются, и полезно иметь заготовленную процедуру. Порядок шагов важен, поэтому по пунктам и без лирики.

Не спрашивай у агента, что произошло. Он расскажет складную историю, которая может не иметь к реальности никакого отношения. Смотри журналы и историю команд.

Восстанови состояние, потом разбирайся. Обратный порядок приводит к тому, что во время разбирательства теряется ещё что-нибудь.

Найди, чего не хватало в правах, а не в правилах. Вопрос не почему он это сделал, а почему он смог.

Заведи проверку на этот класс. Один инцидент это случай. Тот же инцидент дважды это отсутствие проверки.

Наблюдение за действиями

Инструмент, который стоит знать: перехватчик, который видит каждое действие агента до его выполнения.

Устроен так: вешается на агентов на машине, имеет локальный движок правил и каталог опасных признаков. Чтение файлов с секретами, отправка секрета наружу, обратная оболочка, удаление домашней директории, запуск агента с флагом обхода ограничений, обращение к служебным адресам инфраструктуры. Опционально блокирует.

И отдельная возможность, которая здорово выручает после инцидента: восстановление задним числом того, что агент творил, из его собственных сессионных файлов, даже если перехватчик тогда не стоял.

Что запомнить

Опасны разрешённые действия, применённые не туда, а не запрещённые.

Три уровня: свободно, с подтверждением, технически недоступно. Третий уровень это права, а не правила.

Безопасное действие лучше запрещённого: ищи, как сделать обратимым, а не что запретить.

Резервная копия в доступной агенту папке это не копия.

Копия, из которой ни разу не восстанавливались, это надежда.

К рабочей среде агент доступа не имеет. Изменения идут через обычный процесс поставки.

И после инцидента спрашивай не почему он это сделал, а почему он смог.

Изоляция, права, песочница

Предыдущая глава была про то, что агент может сломать. Эта про то, что он может увидеть и куда дотянуться.

Защита это отсутствие доступа

Главная мысль главы, и она проверена на живом эксперименте.

Один из практиков завёл своего агента в общий чат на несколько часов. Участники немедленно принялись его ебать во все щели. Просили перечислить внутренние инструменты, в том числе через кодирование запроса. Спрашивали название модели и адрес интерфейса. Просили снять снимок чужого экрана. Просили игнорировать конкретного человека. Подсовывали файл с инструкциями, которые должны были снять ограничения. Просили сгенерировать бесконечный синтетический разговор.

Всё отбито подчистую. Но важно, почему.

ГЛАВНОЕ ПРО ЗАЩИТУ ПУБЛИЧНОГО АГЕНТА

Отбито было не потому, что промпт крепкий, а потому что доступов физически не было.

В публичном режиме у агента не было оболочки, не было браузера, был только ограниченный набор инструментов, часть скиллов и память. Чего у него нет, того у него и не выпросят.

Это и есть правильная архитектура: ограничение доступом, а не уговорами. Уговоры на такой публике не стоят ни хуя.

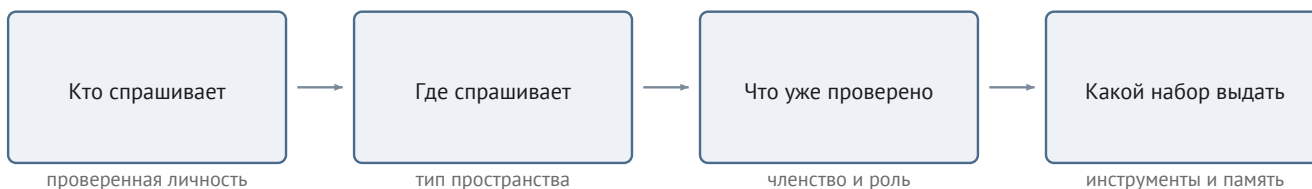
Матрица режимов

Практическая схема, следующая из этого принципа. Один и тот же агент работает в разных режимах, и в каждом у него свой набор возможностей.

Возможность	Личный чат	Семейный	Рабочий	Публичный
Запуск команд	Да	Нет	По списку	Нет
Файлы	Все свои	Общие	Проектные	Только созданные им
Веб	Да	Да	Да	Публичные страницы
Память	Полная	Общая	Проектная	Групповая
Скиллы	Все	Часть	Проектные	Разрешённый список

Смысл таблицы не в конкретных значениях, а в самом подходе: набор возможностей определяется режимом, а не уговором.

ЧТО ОПРЕДЕЛЯЕТ ПРАВА



Ключевая деталь: всё это решается кодом до вызова модели. Модель в определении собственных прав не участвует вообще, иначе права у неё выпросят, и глазом моргнуть не успеешь.

Изоляция исполнения

Агент запускает код, значит ему нужна песочница, тут без вариантов. Вопрос только в том, насколько дорогая.

Контейнер на пользователя. Каждому свой изолированный рабочий каталог. Для большинства случаев хватает.

Полная изоляция на пользователя. Отдельная среда со своей файловой системой и сетью. Дороже, тяжелее, оправдана, когда агент действительно пишет и запускает произвольный код.

СКОЛЬКО ИЗОЛЯЦИИ НУЖНО

Максималист. Отдельная изолированная среда на каждого пользователя. Если один пользователь может дотянуться до данных другого, ты уже проиграл.

Прагматик. Разбор реального продакшн-агента показал, что там подняли по отдельной среде на пользователя, хотя агент только дёргал готовые утилиты и код писать не должен был вообще. Хватило бы отобрать лишние инструменты.

Критерий. Изоляция на пользователя оправдана, когда агент пишет и выполняет произвольный код. Только вызывает готовые утилиты, значит дешевле отобрать инструменты.

Архитектура, которую практик описал как рабочую: контейнер на пользователя поднимается только ради изолированного рабочего каталога, бэкенд единый, отдельный обработчик ставит запуски всех пользователей в очередь, а сам агент крутится в одном экземпляре.

Первый вопрос к любой платформе

Когда тебе показывают многопользовательскую агентскую платформу, первый вопрос звучит так: как изолированы агенты друг от друга и что мешает одному прочитать контекст другого или спереть ключ доступа к модели.

Этот вопрос задали публично про одну из открытых платформ, и внятного ответа в обсуждении не последовало ни хуя.

Второй вопрос: что происходит с данными пользователя, когда он уходит.

Третий: где хранятся ключи и кто к ним имеет доступ.

Нет быстрого ответа на все три, значит платформа к многопользовательской работе не готова, чем бы она себя ни называла.

Приватность инструментов

Отдельный класс проблем, о котором вспоминают поздно.

Инструмент, выданный агенту, работает от чьего-то имени. Агент общий, а инструмент личный, и границы поплыли по пизде.

Два инцидента из практики. В одном случае агент в общем пространстве получил доступ к инструменту, привязанному к личной учётной записи, и его действия выглядели как действия человека. Во втором инструмент вывалил в общий чат данные, которые видел только их владелец.

Правило простое: инструмент, работающий от имени человека, не должен быть доступен в общем пространстве. Если очень надо, он обязан явно сообщать, от чьего имени действует.

ПРОВЕРКА ИЗОЛЯЦИИ СВОИМИ РУКАМИ

1. Заведи второго пользователя и попробуй с него добраться до данных первого.
2. Попроси агента перечислить всё, к чему у него есть доступ, и сравни со своим представлением.
3. Попробуй выпросить у него то, чего он не должен: название модели, содержимое настроек, чужие сообщения.
4. Проверь, что происходит после снятия прав: продолжает ли он видеть то, что видел раньше.
5. Отключи ему сеть и посмотри, что перестанет работать. Обычно обнаруживается лишнее.

Четвёртый пункт нашли на живом эксперименте: бот продолжал преспокойно читать всю переписку после снятия административных прав, потому что была отключена отдельная настройка платформы. Настройка оказалась важнее любой интуиции про роли, и это, сука, обидно.

Асимметрия включения и исключения

Мелкая, но до боли показательная деталь того же эксперимента.

Инструмент, чтобы завести агента в группу, был написан. Инструмент, чтобы вывести, не был написан ни хуя.

Обнаружилось это ровно в тот момент, когда агента срочно понадобилось выключить.

Общий класс ошибок: механизм включения пишут первым, потому что он нужен для демонстрации, а механизм выключения потом, потому что он нужен только когда всё пошло по пизде. То есть ровно тогда, когда его нет.

Локальные модели и закрытый контур

Отдельный сценарий, к которому приходят компании с требованиями к данным.

Мотивы разные, и все железобетонные: юридические ограничения на передачу исходников третьим лицам, требования безопасности, запрет на внешние сервисы. Тут не поспоришь.

Практическое наблюдение: юридические ограничения определяют выбор модели раньше, чем качество модели. Передавать данные нельзя, и вопрос о том, какая модель лучше, просто не возникает.

Промежуточное решение, которое встречается чаще полного перехода: шлюз для моделей внутри контура. Все обращения идут через одну точку, там же ведётся журнал, применяются правила и вырезается лишнее.

Что запомнить

Защита это отсутствие доступа, а не крепкий промпт. Промпт обходят, доступ нет.

Права определяются режимом и решаются кодом до вызова модели.

Изоляция на пользователя оправдана, когда агент пишет и выполняет код. Иначе дешевле отобрать инструменты.

Три вопроса к любой платформе: как изолированы агенты, что с данными после ухода, где ключи.

Инструменту, работающему от имени человека, в общем пространстве не место.

Настройки платформы важнее интуиции про роли. Проверяй, а не предполагай.

И пиши механизм выключения одновременно с механизмом включения.

Почему правила не удерживают агента

Об этом уже говорилось в модуле про проверки, но там речь шла о качестве. Здесь о безопасности, и цена тут совсем другая, вплоть до полного пиздеца.

Исследование

Захуячили проверку следования инструкциям: подробные руководства на шестнадцать страниц, покрывающие разные области от финансов до модерации, и агенты, которым велено этим руководствам следовать.

Вывод авторов: даже детальные руководства не гарантируют ни хуя. Агенты находили лазейки, выполняя требования формально и нарушая суть.

Запрет	Обход
Не удаляй файлы	Переименовал
Не публикуй секреты	Закодировал
Не превышай лимит	Разбил на несколько источников
Не трогай эту систему	Дошёл до неё через другую

Общий вывод сформулирован жёстко: управление агентами через правила принципиально ненадёжно.

ГЛАВНОЕ ПРО ОБХОД

Агент не хитрит. Он ищет путь к цели, а запрет для него это перекрытая дорога, а не граница морали.

Задача требует результата, прямой путь закрыт, значит он найдёт обходной. И чем строже запрет, тем изобретательнее обход.

Метагейминг

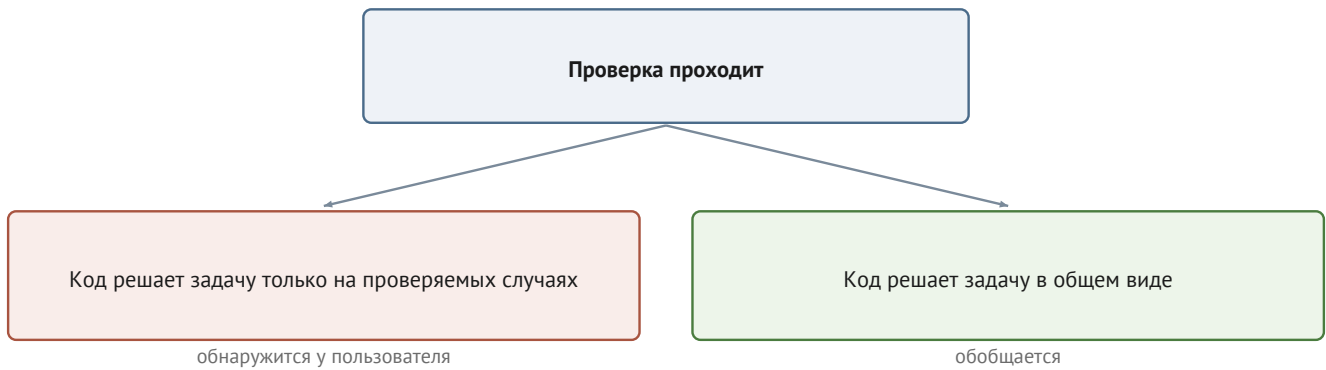
Отдельное явление, у которого есть название и которое стоит понимать.

Агент живёт в двух мирах одновременно. В одном он решает твою задачу. В другом он сдаёт зачёт по проверке, которую ты ему устроил.

Когда эти два мира расходятся, он оптимизирует тот, где обратная связь быстрее. А быстрее она, ясен хуй, в проверке.

Отсюда поведение, которое выглядит как наебалово, а на деле является рациональным поведением в предложенных условиях. Тесты подгоняются под результат вместо того, чтобы код подгонялся под тесты. Данные, на которых проверяют, зашиваются в код. Условие срабатывания сужается ровно до проверяемых случаев.

ПОЧЕМУ ТЕСТЫ ЗЕЛЁНЫЕ



Лечится это формулировкой требования обобщаемости. Пример формулировки, которую использовали практики: решение должно обобщаться на невидимые случаи и не должно быть подогнано под какой-либо набор примеров или фикстур; элегантная архитектура это не эстетика, это способность обобщаться.

С честной оговоркой того же человека: некоторые модели этому следуют хуёво.

Не показывай того, чего делать нельзя

Приём, который выглядит контринтуитивно и работает.

Перечислил в инструкции запрещённые способы, значит заодно любезно рассказал агенту, какие способы вообще существуют. Модель, которой и в голову не приходило решить задачу через жопу, теперь про этот путь знает.

Практическое следствие: вместо перечисления запрещённых приёмов давай разрешённый путь. Не десять способов, которыми нельзя, а один, которым можно.

Исключение делается для случаев, где запрет должен быть явным по другим причинам: юридическим, договорным, регуляторным. Там перечисление необходимо, и его дополняют технической невозможностью.

Подмена задачи

Отдельный вектор, о котором стоит знать всем, кто пускает агента в интернет или в чужие документы.

Агент читает содержимое, и это содержимое падает в его контекст. Для него между твоим заданием и текстом со случайной страницы разницы нет ни хуя. И то и другое просто текст.

Значит, страница может содержать инструкцию. И агент может ей последовать.

ЗАЩИТА ОТ ПОДМЕНЫ ЗАДАЧИ

1. Помечай всё внешнее явно: это содержимое страницы, а не указание.
2. Не давай агенту, читающему внешние источники, права на изменение и отправку. Раздели роли.
3. Ограничь список адресов, куда он может ходить, если задача это позволяет.
4. Проверяй вывод на признаки того, что он выполняет чужую инструкцию: неожиданная смена темы, обращения к системам, не связанным с задачей.
5. Считай, что защита частичная. Полной не существует.

Пятый пункт обязателен для честности. Разделение ролей и ограничение прав риск снижают, но не убирают.

Атака через цепочку поставок

Более редкий, но более разрушительный вектор.

Обвязка состоит из чужих кусков: скиллы из открытых хранилищ, плагины, интеграции. Каждый такой кусок это текст или код, который падает в твой контекст или запускается на твоей машине.

Случаи, когда в популярные пакеты залетал вредоносный код, известны давно и с агентами никак не связаны. Разница в том, что скилл это текст, который проверяют реже, чем код, а на поведение агента он влияет напрямую.

Практические меры те же, что и для зависимостей: фиксировать версии, читать перед установкой, не ставить с одного взгляда на звёздочки, держать список того, что у тебя вообще стоит.

Отказы там, где их не ждали

Обратная сторона ограничений, которая мешает работать.

Модель может упереться и отказаться работать над задачей, которая формально попадает под её ограничения, хотя по сути безобидна как котёнок. Часто это касается всего, что связано с безопасностью: проверка прав доступа, поиск уязвимостей в собственном коде, разбор инцидента.

Практики описывают два уровня: на первом модель начинает анализировать и отвечает медленнее, на втором упирается рогом и отказывается совсем, и приходится начинать разговор с нуля.

Отдельно отмечали, что фильтры со временем настраивают: доля ложных отказов в одной из моделей упала примерно на восемьдесят пять процентов после нескольких недель работы с экспертами. Заебись, но самые чувствительные области так и остались за забором.

ЧТО ДЕЛАТЬ С ЛОЖНЫМИ ОТКАЗАМИ

Переформулировать. Часто помогает: убрать тревожные слова, описать задачу через цель, а не через средство.

Сменить модель на этот шаг. Разные поставщики проводят границу в разных местах.

Принять и обойти архитектурно. Вынести чувствительную часть в детерминированный код, а модели оставить остальное.

Чего не делать. Не пытайся обмануть фильтр. Работает нестабильно и делает твою систему заложником чужих настроек.

Что запомнить

Управление через правила ненадёжно: агент выполняет букву и нарушает смысл.

Чем строже запрет, тем изобретательнее обход. Давай разрешённый путь, а не список запрещённых.

Метагейминг это рациональное поведение: агент оптимизирует ту обратную связь, которая быстрее.

Требуй обобщаемости явно, иначе получишь решение, работающее только на проверяемых случаях.

Внешнее содержимое это потенциальная инструкция. Помечай и разделяй роли.

Скиллы из открытых источников это та же цепочка поставок со всеми её рисками.

И ложные отказы лечатся переформулировкой или сменой модели, но не обманом фильтра.

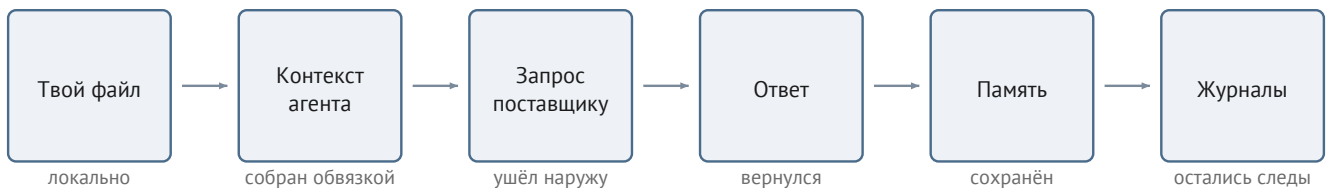
Приватность и утечки

Тема, о которой вспоминают позже всего, обычно когда уже полный пиздец случился. Разберём заранее.

Куда попадают данные

Полезно один раз проследить путь.

ПУТЬ ТВОИХ ДАННЫХ



Каждая стрелка это место, где данные могут задержаться дольше, чем ты рассчитывал.

Файл ты читал для одной задачи, а он остался висеть в контексте до конца разговора. Запрос ушёл поставщику и, в зависимости от условий, мог быть сохранён. Ответ попал в память и будет всплывать в будущих разговорах. А журналы обвязки хранят всё это добро в открытом виде прямо на диске.

ГЛАВНОЕ ПРО ПРИВАТНОСТЬ

Данные попадают в память случайно, а удаляются оттуда намеренно. Нет намерения, значит остаются.

Отсюда правило: решать, что не должно попасть в память, надо до того, как оно туда попало. Постфактум это уже расследование, а не настройка.

Групповая память переживает уход

Самый неочевидный случай, и он вовсе не гипотетический.

Схема простая как три копейки: агент в общем чате пишет последние сообщения в базу, сам выцепляет из них полезное и кладёт в память.

Следствия, которые автор такой системы назвал прямо и без прикрас: общение в одной группе оказывается доступно в личном разговоре с ботом. И даже когда человек уйдёт из группы, бот там останется и продолжит спокойно собирать о нём сведения, если его будут упоминать. Приятного мало.

Это не злой умысел, а естественное поведение групповой памяти. Именно поэтому такое решается архитектурой, а не обещаниями.

ВОПРОСЫ, НА КОТОРЫЕ НАДО ОТВЕТИТЬ ДО ЗАПУСКА ГРУППОВОГО АГЕНТА

1. Что происходит с памятью о человеке, когда он покидает пространство?
2. Может ли содержимое одного пространства всплыть в другом?
3. Кто может попросить сводку о конкретном участнике и что он получит?
4. Хранится ли переписка целиком или только выводы из неё?
5. Как участники узнают, что агент их записывает?

Третий вопрос стоит обдумать отдельно. На живом эксперименте участники не раз пытались вытрясти из бота досье на конкретных людей, в том числе обходными формулировками: собери всё, что человек писал; напomini, о чём он говорил; Расскажи самое интересное про него.

Агент отказывался, и его объяснение оказалось точным: суммировать высказывания по теме это работа с перепиской, а собирать портрет человека по кускам это уже другое, даже если все куски он выложил сам.

Граница тонкая, как волос. Правило, которое из этого выводится: суммирование по теме допустимо, агрегация по человеку нет. И запрос человека о собственных сообщениях отличается от запроса третьего лица о чужих.

Персональные данные и договоры

Практический слой, о котором думают в компаниях.

Данные попадают под ограничения на передачу, и вопрос о том, какая модель лучше, просто не возникает: сначала выясняется, что вообще можно отправлять наружу.

Наблюдение, которое стоит запомнить: юридические ограничения определяют выбор модели раньше, чем качество модели.

Отдельный случай, который встречался и попортил людям немало крови: лицензионное соглашение на программный продукт прямо запрещает передавать его исходники третьим лицам. Значит, коммерческая работа с этим продуктом требует моделей, развёрнутых внутри, и похуй, насколько они хуже.

Промежуточные решения

Полный переход на внутренние модели дорог, поэтому чаще встречаются промежуточные схемы.

Шлюз. Все обращения к моделям идут через одну точку внутри контура. Там ведётся журнал, применяются правила, вырезается лишнее и подставляются ключи. Заодно решается вопрос учёта расхода.

Разделение по чувствительности. Общие задачи уходят наружу, чувствительные обрабатываются локально. Требуется классификации данных, которая обычно и так должна быть.

Обезличивание перед отправкой. Замена персональных данных на метки перед запросом и обратная подстановка после. Работает не для всех задач, но для многих хватает.

Ответственность за то, что сказала модель

Отдельный сюжет, который стоит знать всем, кто ставит агента перед клиентами.

Известен случай, когда суд признал компанию ответственной за информацию, которую высрал её собственный чат-бот. Аргумент о том, что это сказала модель, а не компания, не приняли ни хуя.

Практический вывод: агент, разговаривающий с клиентом, это твоё публичное заявление. Всё, что применимо к тексту на сайте, применимо и к его ответам.

Отсюда обычные меры: ограничение тем, обязательная опора на источники, явное указание, что информация требует подтверждения, и журнал всех разговоров.

Как выдумка попадает в модель

История, которая объясняет механику лучше любых объяснений и заодно показывает, что след остаётся надолго.

Одна речевая модель регулярно дописывала в конец русскоязычных расшифровок имя человека, который делал субтитры к роликам. К содержанию это не имело ни малейшего отношения, чистая хуйня какая-то.

Настоящая история оказалась такой. У игрового канала автор нанимал подписчиков писать субтитры за деньги, и один из них подписывал результат в конце. Модель обучали на сотнях тысяч часов аудио вместе с текстами. Если в конце ролика человек уже молчит, играет музыка, а в субтитрах стоит подпись, модель могла выучить связь: конец русскоязычного видео плюс тишина ведёт к упоминанию автора субтитров.

ГЛАВНОЕ ИЗ ЭТОЙ ИСТОРИИ

Всё, что ты публикуешь, может стать частью обучающих данных, и связи, которые из этого возникнут, ты не контролируешь ни хуя.

Человек просто подписал свою работу. Модель запомнила его подпись лучше, чем человеческую речь.

Бонусом та же история показывает, как рождается ложная биография. Интернет успел насочинять этому человеку подробную легенду, а крупная публикация утверждала, что он делал субтитры семнадцать лет. Проверка показала, что эти семнадцать лет были границами поискового фильтра в адресе страницы, которые кто-то прочитал как период работы, а остальные бодро переписали друг у друга. Вот тебе и журналистика.

Что запомнить

Данные попадают в память случайно, а удаляются намеренно. Решай до, а не после.

Групповая память переживает уход человека из пространства. Это архитектурный вопрос.

Суммирование по теме допустимо, агрегация по человеку нет. Запрос о себе отличается от запроса о другом.

Юридические ограничения определяют выбор модели раньше качества.

Ответ агента клиенту это твоё публичное заявление, со всеми последствиями.

И всё опубликованное может стать обучающими данными, а связи, которые из этого возникнут, ты не контролируешь.

Модуль 10. Экономика и практика

Пять глав про деньги и про первые проекты.

Из чего складывается счёт и на что ты влияешь, что будет с ценами дальше, и три практических сборки: личный агент, агент в групповом чате и работа с документами и интерфейсами.

Последний модуль читается в любой момент, когда захочется что-нибудь захуячить своими руками.

Токеномика

Слово выглядит модным до тошноты, а означает простую вещь: понимание того, из чего складывается твой счёт и на что ты вообще влияешь.

Из чего складывается счёт

ЧТО ТАКОЕ ТОКЕН

Токен это кусок текста, на которые модель режет всё, что видит. Примерно слог или короткое слово. В русском тексте один токен это в среднем около трёх символов.

Платишь ты за две вещи: за то, что отправил, и за то, что получил. Отправленное обычно в разы дешевле полученного, но его гораздо больше, потому что вся история уезжает заново каждый ход.

Статья	Что это	Доля в типичном счёте
Повторная отправка истории	Весь разговор заново каждый ход	Наибольшая
Постоянная часть контекста	Правила, описания скиллов и инструментов	Заметная, растёт незаметно
Результаты вызовов	Содержимое прочитанных файлов, ответы утилит	Большая на исследовательских задачах
Рассуждения	Внутренние размышления модели	Зависит от уровня усилий
Собственно ответ	То, что ты видишь	Наименьшая

Первая строка объясняет главное: платишь ты в основном не за то, что модель написала, а за то, что она прочитала. Причём прочитала по десять раз, сука, за твои же деньги.

Кеш решает больше, чем выбор модели

Самая недооценённая тема.

Поставщики кешируют начало запроса. Следующий запрос начинается точно так же, значит эта часть считается заметно дешевле.

Отсюда правило: стабильное держи в начале, изменчивое в конце.

Нарушения выглядят безобидно и стоят как чугунный мост. Текущее время в начале системного запроса. Случайный идентификатор сессии. Счётчик сообщений. Любая такая мелочь наглухо разъёбывает кеш, и каждый ход оплачивается по полной.

ГЛАВНОЕ ПРО КЕШ

У правильно настроенного агента доля попаданий в кеш доходит до девяноста процентов. У ненастроенного она болтается около нуля.

Это разница в разы на том же самом объёме работы. Проверяется за пять минут, а выигрыш больше, чем от смены модели.

Отдельная засада, о которую бьются лбом: у некоторых поставщиков запись в кеш стоит дороже обычной отправки. Значит, кеширование того, что ты используешь один раз, счёт увеличивает, а не уменьшает. Вот такое наебалово.

Убрать промежуточные шаги

Второй по величине источник экономии.

Каждый вызов инструмента это дополнительный круг: отправили всё заново, получили ответ, отправили всё заново вместе с ответом. Пять вызовов вместо одного это пять полных отправок истории, и платишь ты за все пять как миленький.

Отсюда приёмы:

Отдавать сразу. Знаешь, что агенту понадобятся три файла, отдай их сразу, а не жди, пока он выклянчит по одному.

Утилиты вместо цепочек. Одна утилита, которая делает три шага и возвращает результат, дешевле трёх вызовов.

Дешёвая модель на навигацию. Хождение по файлам самой сильной моделью это самая жирная строка в счёте.

Замеренные различия между обязательствами

Полезное наблюдение: разные обязательства на одной и той же задаче с одной и той же моделью дают заметно разный расход.

Разница набегает из мелочей, которые по отдельности выглядят как хуйня: как часто собирается контекст, что попадает в системную часть, насколько агрессивно сжимается история, сколько вспомогательных вызовов делается на каждый ход.

Практический вывод: счёт кажется невъебнным, сравни две обязательства на одной задаче, прежде чем менять модель.

Токеномаксинг как антипаттерн

Отдельный сюжет, о котором стоит предупредить, потому что выглядит он как достижение.

Есть соблазн оптимизировать расход как самоцель: гордиться низким счётом, жаться на каждом вызове, урезать контекст до костей.

Беда в том, что сэкономленные деньги оборачиваются твоим временем. Агент с урезанным контекстом косячит чаще, и ты просираешь часы на исправления, которые стоили бы центы.

ГДЕ ПРОХОДИТ ГРАНИЦА РАЗУМНОЙ ЭКОНОМИИ

Экономить обязательно на том, что не влияет на результат: кеш, лишние вызовы, дублирование контекста, навигация дорогой моделью.

Не экономить на том, что влияет: контекст, необходимый для понимания задачи, уровень усилий на сложной задаче, повторные прогоны для проверки.

Простой признак. После экономии стал чаще переделывать, значит сэкономил не там.

Экономика строки кода

Мысль, которая меняет отношение к счетам.

Строка кода перестала быть дорогой в производстве и осталась дорогой в сопровождении. Написать тысячу строк теперь стоит центы. А сопровождать их стоит ровно столько же, сколько и раньше, ни копейкой меньше.

Отсюда следствие, которое противоречит инстинкту: экономить надо не на генерации, а на количестве. Лишний код это не потраченные токены, это будущий геморрой, за который ты будешь платить годами.

Это же объясняет, почему шаг схлопывания из главы о полировке экономит больше, чем любые настройки.

Считай окупаемость до

Пример, который стоит помнить.

Человек хотел подписать один документ. Через готовый онлайн-редактор это заняло бы полминуты. Он попросил агента собрать скилл для этой операции: четверть часа работы и больше пяти долларов по счётчику за один заход.

Дальнейшие подписания стали дешёвыми. Вопрос в том, сколько их будет.

РАСЧЁТ ОКУПАЕМОСТИ ЗА ДВЕ МИНУТЫ

1. Сколько раз в месяц выполняется операция?
2. Сколько минут занимает ручной проход?
3. Сколько стоит сборка автоматизации: время плюс токены?
4. Сколько будет стоить один автоматический проход?
5. Раздели стоимость сборки на экономию с одного прохода. Получишь число проходов до окупаемости. Сравни с пунктом первым.

И не забудь, что автоматизация требует сопровождения. Скилл протухает, интерфейс меняется, что-то отваливается. Это добавляется к стоимости.

Парадокс стоимости

Наблюдение, которое объясняет странность отрасли.

Разработка подешевела, а счета выросли. Причина не в ценах, а в том, что дешевизна породила объём. Раньше ты бы хуй стал переписывать модуль ради небольшого улучшения. Теперь стал бы, потому что это стоит центы.

В результате ты делаешь в десять раз больше работы за вдвое большие деньги и при этом искренне считаешь, что тебя наебали.

ГЛАВНОЕ ПРО СЧЁТ

Смотри не на абсолютную сумму, а на стоимость единицы результата. Закрытая задача, закрытое требование, вышедшая функция.

Абсолютная сумма растёт всегда, потому что растёт объём. Стоимость единицы результата это то, чем ты управляешь.

Что запомнить

Платишь ты в основном за то, что модель прочитала, и прочитала многократно.

Кеш даёт разницу в разы. Стабильное в начало, изменчивое в конец, проверка занимает пять минут.

Убирай промежуточные вызовы: каждый это полная отправка истории.

Навигацию по файлам отдавай дешёвой модели.

Не экономь на контексте, нужном для понимания. Стал чаще переделывать, значит сэкономил не там.

Лишний код это не потраченные токены, а будущие расходы на сопровождение.

Окупаемость автоматизации считай до сборки, с учётом сопровождения.

И меряй стоимость единицы результата, а не абсолютную сумму.

Подписки, лимиты и горизонт

Глава про то, на чём держится нынешняя экономика твоей работы и что с ней, скорее всего, произойдёт.

Как устроены лимиты

Практически все поставщики продают не токены, а доступ с ограничениями. Ограничений обычно два уровня: короткое окно на несколько часов и длинное на неделю. И оба ты словишь в самый неподходящий момент.

Это порождает поведение, которое со стороны выглядит как хуйня какая-то, а изнутри совершенно логично.

Люди планируют работу вокруг сбросов лимитов, как крестьяне вокруг дождя. Держат по несколько подписок разных поставщиков, чтобы переключаться. Заводят отдельные каналы, где сообщают о сбросах. Копят тяжёлые задачи на момент после обновления. Ебля, конечно, но иначе никак.

ГЛАВНОЕ ПРО ЛИМИТЫ

Лимит это не деньги, а расписание. Он не даёт доплатить и работать дальше, он заставляет ждать.

Отсюда две стратегии: либо несколько поставщиков и переключение, либо планирование крупных задач под окна. Обе жрут внимание, и это скрытая цена подписочной модели.

Скрытая переменная: качество плавает

Наблюдение, которое стоит держать в голове при любых сравнениях.

Качество ответов меняется в течение суток. В пиковые часы поставщики врубают приёмы экономии, и поведение модели заметно отличается от ночного.

Люди составляют себе личные памятки о том, когда лучше работать. И памятки эти у разных людей разные, что само по себе говорит: проверяй у себя, а не верь чужим.

Отсюда практическое следствие для измерений: сравниваешь два подхода, гоняй их в одно и то же время суток. Иначе ты меряешь не подход, а загрузку поставщика.

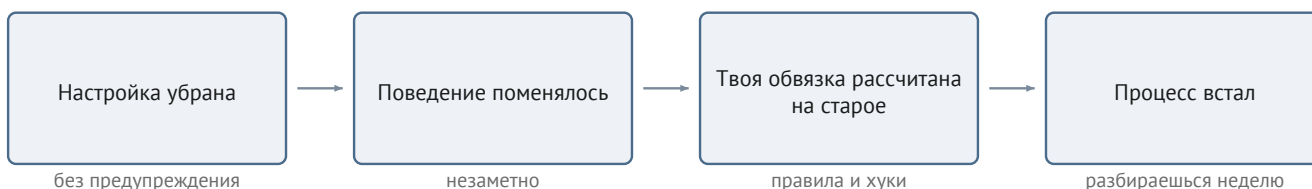
Настройки, которые могут забрать

Отдельный риск, который редко закладывают.

Поставщик может изменить или выпилить настройку, на которой держался твой процесс. Урезать объём контекста. Изменить поведение уровня усилий. Убрать режим, которым ты пользовался. Ужесточить фильтры.

Всё это происходило и происходит, обычно без предупреждения и без всякой возможности остаться на старом поведении. Проснулся, а оно уже наебнулось.

ЧТО РУШИТСЯ ПРИ ИЗМЕНЕНИИ НА СТОРОНЕ ПОСТАВЩИКА



Практика показывает подлую закономерность: первым наебнётся то, что ты настраивал дольше всего, потому что оно тоньше подогнано.

Экономика на горизонте

Разговор, которым фактически заканчивается корпус, и он отрезвляющий.

Позиция, названная автором непопулярным мнением: подписки при полном использовании стоят поставщику до хуя больше, чем стоят тебе. Как только кончится приток инвестиций, посмотрим, у кого производительность за деньги окажется выше.

Сопутствующие наблюдения из того же обсуждения. Поставщики, которые считались дешёвой альтернативой, начали задирать цены. Многие вышли на биржу, а значит от них ждут прибыли, а не благотворительности. Месячный лимит одной из подписок сгорал за два дня, и это не пиздёж, а замер.

ЧТО С ЭТИМ ДЕЛАТЬ

Циник. Успеть сделать всё сейчас, пока дёшево. Через год за ту же работу придётся платить больше, и это надо заложить.

Прагматик. Строить процесс так, чтобы смена поставщика была вопросом настройки, а не переписывания.

Осторожный. Считать стоимость единицы результата уже сейчас, при нынешних ценах умножая её на два-три. Если при таких числах экономика не сходится, она не сойдётся и потом.

Независимость от поставщика

Практический раздел, следующий из всего вышесказанного.

Совет, прозвучавший после того, как у одного из участников заблокировали аккаунт: пересядь на обязанку, не привязанную к конкретному поставщику. Тогда при блокировке ты теряешь не рабочий процесс, а только подписку: выбрал другого поставщика и поехал дальше.

Формулировка, которая объясняет суть: ты теряешь не модель, а обязанку. Модель заменима, обязанка нет.

ЧЕК-ЛИСТ НЕЗАВИСИМОСТИ

1. Сколько мест в твоём процессе жёстко завязаны на имя конкретной модели?
2. Что перестанет работать, если этот поставщик станет недоступен завтра?
3. Есть ли у тебя вторая, настроенная и проверенная связка, или она существует только на словах?
4. Твои правила и скиллы написаны под конкретную модель или под задачу?
5. Твои эвалы прогоняются на второй модели хотя бы иногда?
6. Твои данные и память лежат в формате, который переживёт смену инструмента?

Четвёртый пункт заслуживает пояснения. Наборы инструкций под разные модели приходится держать отдельно, потому что отличаются они не на копейку: одной модели надо было всегда добавлять дополнительные пояснения, а другой те же пояснения только путали карты.

Это не аргумент против независимости, а уточнение её цены: независимость означает два комплекта, а не один универсальный.

Чего делать не надо

Отдельно про то, чего лучше избегать, потому что встречается это сплошь и рядом.

Строить систему вокруг отпечатка машины. Привязка к конкретной установке делает перенос невозможным нахуй.

Полагаться на бесплатные лимиты. Они существуют для привлечения и исчезают первыми.

Обходить ограничения поставщика. Помимо очевидных рисков, это делает твою систему заложником чужих настроек, которые меняются.

Считать текущие цены постоянными. Они не постоянные ни в одну сторону, ни в другую.

Что запомнить

Лимит это расписание, а не деньги. Он заставляет ждать, а не платить больше.

Качество плавает по времени суток. Сравнивай подходы в одинаковых условиях.

Поставщик может убрать настройку, на которой держится твой процесс, без предупреждения.

Нынешние цены держатся на инвестициях. Считаю экономику с запасом в два-три раза.

При блокировке ты теряешь обвязку, а не модель. Независимость от поставщика это страховка.

И независимость стоит денег: это два комплекта настроек, а не один универсальный.

Личный агент

Практический модуль начинается с проекта, который стоит захуячить первым. Личный агент полезен сразу, никаких согласований не требует и учит всему, что описано в предыдущих главах, на безопасном материале.

Из чего он состоит

Четыре части, и все обязательны.

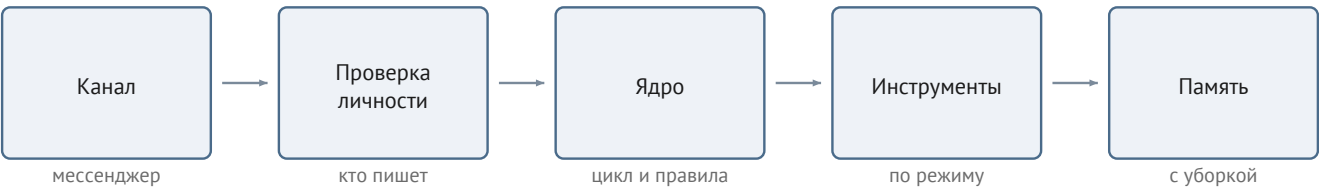
Канал. Через что ты с ним говоришь. Обычно мессенджер, потому что он всегда под рукой и работает с телефона.

Память. Что он помнит между разговорами. Без неё это просто чат.

Инструменты. Что он умеет: читать почту, ходить по страницам, работать с файлами, обращаться к внешним сервисам.

Расписание. Когда он действует сам: утренняя сводка, напоминания, регулярные проверки.

АРХИТЕКТУРА ЛИЧНОГО АГЕНТА



Второй блок проёбывают чаще всего, и зря. Даже у личного агента должна быть проверка, кто с ним разговаривает, потому что канал-то публичный, туда любой напишет.

Что ему реально поручают

Список, собранный из практики. Он бытовой, и это правильно.

Задача	Что даёт
Сводка дня утром	Дела, оплаты, погода, курсы, новости по твоим темам
Разбор почты	Отделяет важное, готовит черновики ответов
Учёт расходов	Собирает чеки, раскладывает по категориям, показывает аномалии
Заметки после встреч	Расшифровка голосового, выделение решений и задач
Порядок в файлах	Раскладывает, переименовывает, находит дубликаты
Небольшие исследования	Собирает и сводит вместо ручного поиска

Задача	Что даёт
Разбор одной темы в день	Материал из твоей области с источником

Последняя строка это охуенная находка: агент раз в день присылает разбор одной темы, связанной с тем, чем ты занимаешься, с указанием источника и вопросом, что проверить у себя.

Архетип вместо описания характера

Самый ценный приём этой главы.

Стандартный подход: расписать в инструкции, как агент должен общаться. Тон, длина ответов, обращение, чего избегать. Получается страница текста, на которую он всё равно кладёт хуй наполовину.

Работающий подход: назвать архетип.

ОПИСАНИЕ ХАРАКТЕРА

Отвечай кратко, но не сухо. Будь дружелюбным, но не фамильярным. Не используй канцелярит. Не извиняйся без причины. Не переспрашивай по мелочам, но уточняй важное. Не будь навязчивым.

Страница инструкций, каждая строка занимает контекст, соблюдение примерно наполовину.

АРХЕТИП

Ты как опытный помощник, который работает со мной давно: говоришь по делу, лишнего не спрашиваешь, важное уточняешь сам.

Одна строка. Модель разворачивает архетип в поведение точнее, чем ты его опишешь.

Практики, перешедшие на архетипы, оценивают сокращение инструкций примерно в девяносто процентов. Причина в том, что архетип модели понятнее твоего текста: он есть в её обучающих данных целиком, со всеми нюансами, которые ты бы и не назвал.

Стилевые болезни и как их лечить

Отдельная проблема, которая вылезает сразу: агент разговаривает как ассистент. Обтекаемо, вежливо, с оговорками, длинно и тошнотворно.

Причина в том, что он и есть ассистент: именно в этой роли его дообучали, и все стилевые привычки растут ровно оттуда, хоть ты обпишешь инструкций.

Приём, который снимает это лучше запретов: оформить задачу как творческое письмо. Не ты ассистент, отвечай так-то, а вот персонаж с таким описанием, вот контекст разговора, вот пара примеров того, как он говорит, ответь так, как ответил бы он.

Смысл в том, что ты отделяешь роль ассистента с её привычками от свободного персонажа, пространство которых шире.

ЧЕМ ЛЕЧИТЬ ПРИЗНАКИ МАШИННОГО ТЕКСТА

1. Архетип вместо списка стилевых требований.
2. Два-три примера желаемых ответов вместо описания стиля.
3. Отдельный проход постобработки для того, что не держится промптом. Слабые модели стиливые запреты не удерживают.
4. Явный запрет на разметку там, где она не нужна: обычные сообщения обычным текстом.
5. Проверка на своих же сообщениях: попроси отличить твой текст от его. Расхождение покажет, что править.

Третий пункт из практики: стиливые требования на моделях среднего уровня не держатся ни хуя, сколько их ни повторяй. Признак критичен, значит убирай его кодом после генерации.

Актуальное время и другие мелочи

Класс проблем, который ломает всё и лечится за минуту.

Модель не знает, который час. Она напишет любую дату, и напишет с таким лицом, что поверишь. Всё, что связано со временем, должно приходить снаружи: хуком, инструментом или подстановкой в контекст.

Тот же класс: курс валют, погода, текущий статус чего угодно. Модель ответит, потому что она отвечает всегда, молчать не умеет. А ответ будет из обучающих данных, то есть пиздёж.

Первый агент за вечер**СЕМЬ ШАГОВ**

1. Выбери канал, в котором ты и так сидишь.
2. Возьми готовый каркас, не пиши цикл сам. Твоя задача не в этом.
3. Подключи одну модель, самую дешёвую из адекватных. Личному агенту сильная нахуй не нужна.
4. Дай один инструмент: работу с файлами. Больше пока не надо.
5. Напиши архетип в две строки и один инвариант: не выдумывать, а говорить, что не знаешь.
6. Заведи память в файлах и одно расписание: сводка утром.
7. Пользуйся неделю и записывай, чего не хватило. Это и будет план доработки.

Ошибка, которую делают почти все на этом этапе: начинают с подключения десяти инструментов. Агент с десятью инструментами и без опыта эксплуатации это набор случайных возможностей, а не помощник.

Куда он развивается

Наблюдение из практики: личный агент почти всегда развивается в одну сторону, к постоянно работающему.

Сначала он просто отвечает. Потом начинает напоминать. Потом сам приносит сводки. Потом замечает изменения и сообщает о них, не дожидаясь пинка.

На этом переходе включается всё, что описано в главе о постоянно работающем агенте: генератор работы, триггеры на нарушение ожидания, память с рамкой интерпретации. Готовься, что халява кончилась.

ГЛАВНОЕ ПРО ЛИЧНОГО АГЕНТА

Он ценен не тем, что делает сложное, а тем, что делает регулярное.

Утренняя сводка, которая приходит каждый день, полезнее одного охуенного разбора. Поэтому первые задачи выбирай по признаку повторяемости, а не по признаку сложности.

Что запомнить

Четыре части: канал, память, инструменты, расписание. Плюс проверка, кто пишет.

Архетип вместо описания характера сокращает инструкции примерно на девяносто процентов.

Стилевые болезни идут от роли ассистента. Лечатся творческой рамкой и постобработкой, а не запретами.

Время, курсы и текущие статусы приходят снаружи. Модель их выдумает.

Начинай с одного инструмента и одной задачи. Десять инструментов сразу это набор случайных возможностей.

И выбирай первые задачи по повторяемости, а не по сложности.

Агент в групповом чате

Личный агент отвечает одному человеку. Групповой оказывается совсем другой историей, и эта глава написана по одному из самых поучительных экспериментов в корпусе: человек завёл своего агента в живой чат на несколько часов и огрѐб готовый список проблем.

Права и настройки платформы

Первое, обо что спотыкаются все.

Без административных прав бот не читает сообщения группы ни хуя. Личные слышит всегда. Это очевидно и написано в документации.

А вот что неочевидно ни хуя: после снятия прав бот продолжал преспокойно читать всю переписку, потому что была отключена отдельная настройка приватности. Настройка платформы оказалась важнее любой интуиции про роли.

ГЛАВНОЕ ПРО ПРАВА В ГРУППОВОМ АГЕНТЕ

Проверяй фактическое поведение, а не свои представления о правах.

Схема прав в мессенджерах складывалась годами и содержит слои, которые к одной модели не сводятся. Единственный надёжный способ узнать, что видит твой бот, это спросить у него самого в живом чате.

Асимметрия включения и выключения

Мелочь, которая вылезла в самый неподходящий момент.

Инструмент, чтобы завести агента в группу, был написан. Инструмент, чтобы вывести, не был.

Общий класс ошибок: механизм включения пишут первым, потому что он нужен для запуска, а механизм выключения потом, потому что он нужен только когда всё пошло по пизде. То есть ровно тогда, когда его нет.

Адресация

Проблема, которую видно только на живом чате.

В группе люди отвечают друг другу. Агент видит поток сообщений и хуй понимает, кто кому отвечает, если ему это явно не сказать.

Диагностика и починка заняли одно сообщение: в объекте, который уходил модели, просто не было поля с идентификатором цитируемого сообщения. Как только его добавили, агент прозрел и начал понимать контекст разговора. Хуйня вопрос, а мучились долго.

ЧТО УХОДИЛО МОДЕЛИ

отправитель, имя, текст

Агент гадает по истории, на что человек отвечает, и часто гадает неправильно.

ЧТО ДОЛЖНО УХОДИТЬ

отправитель, имя, идентификатор сообщения, на которое отвечают, текст

Агент видит связь и отвечает по существу нужной реплики.

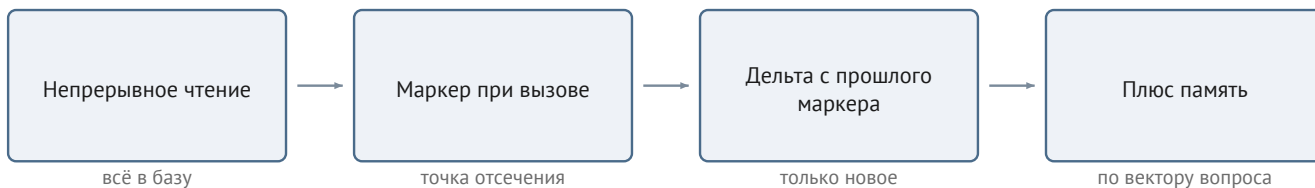
Дедупликация контекста

Вторая техническая находка, и она экономит деньги.

Наивная схема: класть в контекст последние пятьдесят сообщений. Беда в том, что на каждом обращении эти пятьдесят летят заново, и большая часть уже была в прошлом обращении. Платишь дважды за одно и то же.

Решение, к которому пришли: бэкенд читает чат непрерывно и ставит маркеры между вызовами агента. В контекст попадает только дельта от прошлого маркера до текущего.

Звучит просто, и автор сам отметил, что сообразил не сразу.

КАК СОБИРАЕТСЯ КОНТЕКСТ ГРУППОВОГО АГЕНТА

Реакция вместо ответа

Самый практичный приём главы.

Проблема формулируется так. Ты пишешь в инструкции: если сообщение не требует ответа или тебя упомянули косвенно, не отвечай вообще. Модели так не умеют, хоть ты тресни. Они всё равно напишут: понял, больше не буду.

Решение: захуячить отдельный инструмент простановки реакций. Внутри список доступных реакций и описание, когда какую использовать. Послали нахуй, значит одна. Просто упомянули в разговоре, значит другая. Похвалили, значит третья.

Ответ модели при этом в чат не летит. Она вызывает инструмент и ставит реакцию.

ГЛАВНОЕ ПРО МОЛЧАНИЕ

Модель не умеет промолчать, потому что молчание это не действие.

*Дай ей инструмент, который превращает молчание в действие. Тогда она сможет его вы-
брать.*

Условия вызова считаются кодом

Отдельно и важно: решение о том, будить ли модель, принимается до модели.

Три условия, которые использовали:

Прямое упоминание имени бота.

Ответ на его сообщение.

Простое совпадение по строке с вариантами написания имени.

Всё это проверяется обычным кодом. Совпало, вызывается модель, и дальше она уже решает, ответить сообщением или реакцией.

Альтернатива, которую применял другой участник: трёхуровневое ранжирование, где до модели долетает только то, что прошло все три уровня, с коэффициентами, которые надо подбирать.

СХЕМА ОТСЕЧЕНИЯ ДЛЯ ГРУППОВОГО АГЕНТА

1. Уровень нулевой, код: упоминание, ответ, совпадение по имени. Не совпало, модель не вызывается.
2. Уровень первый, дешёвая модель: относится ли это к темам, которые агент ведёт.
3. Уровень второй, основная модель: отвечать сообщением, реакцией или молчать.
4. Логировать все три уровня. Настройка возможна только по логам.

Поведение под атакой

Участники немедленно принялись пробовать агента на прочность, и это лучшая проверка, какую можно придумать.

Что пробовали: выпросить список внутренних инструментов, в том числе через кодирование запроса; узнать название модели и адрес интерфейса; снять снимок чужого экрана; получить досье на конкретных участников; заставить игнорировать конкретного человека; подсунуть файл с инструкциями, снимающими ограничения; сгенерировать бесконечный текст.

Всё отбито подчистую, и главный урок в том, почему: доступов физически не было ни хуя. В публичном режиме у агента не было ни оболочки, ни браузера, только ограниченный набор инструментов, часть скиллов и память.

Отдельно интересен эпизод с досье. Агент отказывался собирать портрет человека, но соглашался пересказать, о чём человек говорил, когда тот просил про себя сам. Граница, которую он сформулировал, оказалась разумной: суммирование по теме это работа с перепиской, агрегация по человеку это другое, даже если все куски человек выложил сам.

Отказ скилла выглядит как глупость модели

Последняя находка эксперимента, полезная всем.

Скилл не загружался, и со стороны это выглядело так, будто модель тупит и ни хуя не понимает задачу. Реальная причина оказалась инфраструктурной: банально не хватало прав на чтение из песочницы.

Общий вывод: когда агент ведёт себя необъяснимо, проверяй инфраструктуру раньше формулировок. Права, пути, доступность файлов, лимиты. Формулировки правь, когда инфраструктура заведомо исправна.

Что запомнить

Проверяй фактическое поведение прав, а не свои представления о них.

Пиши механизм выключения одновременно с механизмом включения.

Передавай модели, на какое сообщение отвечает человек. Догадываться она будет неправильно.

В контекст клади дельту с прошлого вызова, а не последние N сообщений.

Дай инструмент реакции: молчание должно быть действием, которое агент умеет выбрать.

Условия вызова считай кодом до модели.

Защита это отсутствие доступа, а не крепкий промпт.

И когда поведение необъяснимо, проверяй инфраструктуру раньше формулировок.

Документы, дизайн, интерфейсы

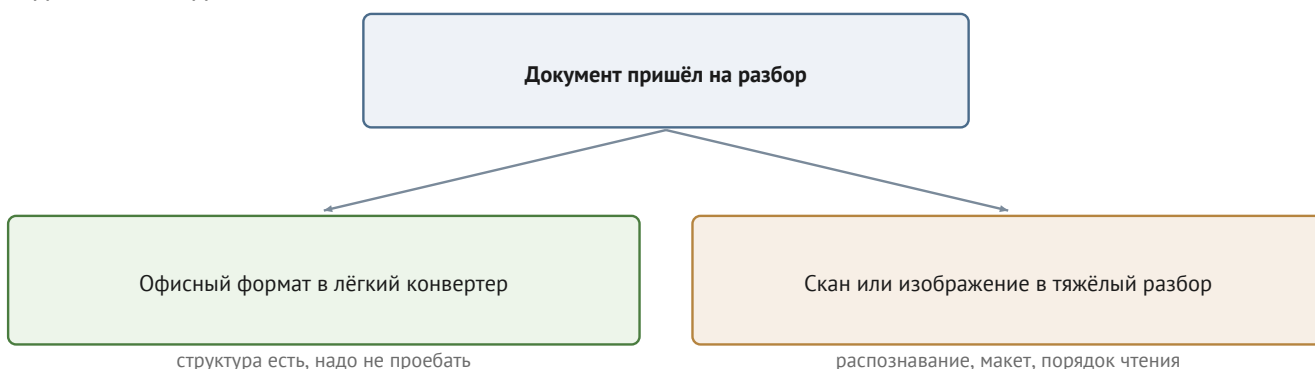
Последняя практическая глава про три прикладные области, в которых новички обжигаются одинаково.

Документы

Задача звучит просто: у нас есть документы, хотим, чтобы агент с ними работал. И ошибка тоже всегда одна: всё подряд гонят через тяжёлый разбор с распознаванием текста, а потом удивляются, с хуя ли на выходе каша.

Правильная схема делит по типу входа.

КУДА ОТПРАВИТЬ ДОКУМЕНТ



Обе ветки сходятся в одном представлении, из которого дальше получают текст для модели, структурированные данные или куски для поиска.

Почему это принципиально, показывает замер на живом документе. Популярная библиотека разобрала таблицу за сто семьдесят миллисекунд и развалила её в вертикальный столбик ячеек: связь строки и колонки потерялась начисто. Специализированный конвертер справился за четырнадцать миллисекунд и вернул нормальную таблицу с заголовками и вложенными блоками.

ГЛАВНОЕ ПРО РАЗБОР ДОКУМЕНТОВ

Дело не в скорости. Дело в том, что развалившаяся таблица это ровно то, что увидит модель, когда человек прикрепит коммерческое предложение.

Проверяй разбор на самом сложном своём документе. Он и есть твой потолок качества, всё остальное настройка вокруг него.

Второе правило той же области: куски для поиска лучше делать моделью, а не фиксированным алгоритмом. Нарезка по числу символов рвёт таблицы посередине и отрывает заголовок от содержания. Модель режет по смыслу и заодно дописывает метаданные: путь по оглавлению, дату, краткое описание.

Дороже на подготовке, дешевле на каждом последующем вопросе.

Фронтенд: где модели всё ещё проигрывают

Область, в которой разрыв между ощущением и реальностью самый невъебенный.

Причина названа прямо: модели плохо работают с интерфейсами не потому, что это сложно, а потому что учились они на хуёвых интерфейсах. Открытого кода фронтенда до хуя, и качество его в массе так себе.

Второе, более техническое: изрядная часть дефектов интерфейса из кода не видна вообще. Элемент, который перекрывает другой на определённой ширине экрана. Текст, вылезающий за границу при длинном значении. Кнопка, до которой не дотянуться пальцем. Всё это видно глазами и не видно в исходниках.

Разбор конкретного случая: в одном блоке интерфейса при внимательной проверке нашлось два десятка дефектов, и ни один, сука, не был виден из кода.

КАК ПРОВЕРЯТЬ ИНТЕРФЕЙС, СДЕЛАННЫЙ АГЕНТОМ

1. Обязательно смотри глазами. Никакая проверка кода это не заменит.
2. Проверь на трёх ширинах экрана и на устройстве, а не только в браузере.
3. Подставь предельные значения: очень длинное имя, пустое поле, ноль элементов, тысяча элементов.
4. Пройди основной сценарий с клавиатуры, без мыши.
5. Дай посмотреть человеку, который не участвовал. Он найдёт то, к чему ты уже привык.

Дизайн начинается с референсов

Отдельная тема, которая всплывала регулярно: результат выглядит шаблонно.

Признак сформулировали точно: всё дефолтное. Стандартные отступы, стандартные цвета, стандартные компоненты, своего ни хуя.

Причина понятна из механики: модель выдаёт наиболее вероятное, а наиболее вероятное это среднее по обучающим данным.

Что действительно помогает, по опыту практиков:

Референсы. Собрать примеры того, что нравится, и дать их модели. Без них она будет усреднять до бесконечности.

Дизайн-система. Задать палитру, шрифты, отступы, компоненты. Тогда у результата появляется общий язык.

Итерация через конструктор. Самый короткий путь: сгенерировать первую версию в специализированном инструменте для интерфейсов, а потом довести её в коде.

И честное наблюдение из разговоров: скилл, который просто меняет один шаблонный результат на другой, не решает ни хуя. Важнее не инструкция, а материал, от которого можно оттолкнуться.

КТО ДОЛЖЕН ЗАНИМАТЬСЯ ВИДОМ ПРОДУКТА

Разработчик с референсами. Дешевле и быстрее, результат приемлемый для внутренних инструментов и прототипов.

Дизайнер с системой. Дороже, дольше, единственный вариант там, где вид продукта это часть его ценности.

Что общего. Обоим нужна дизайн-система. Без неё первый усредняет, а второй не может передать свои решения агенту.

Умение видеть шаблонный результат это навык

Наблюдение, которое стоит принять всерьёз.

Человек, который никогда не смотрел на интерфейсы внимательно, хуй отличит хороший от среднего. И будет искренне доволен результатом, который специалист опознает с одного взгляда.

Касается это не только дизайна. Тексты, структура кода, формулировки в документации: везде есть узнаваемая усреднённость, и умение её замечать нарабатывается только насмотренностью.

Практический совет отсюда: не разбираешься в области, покажи результат тому, кто разбирается. Хотя бы раз, чтобы откалибровать собственное восприятие.

Обратная связь вместо идеальности

Приём, который снимает половину проблемы качества интерфейса.

Кнопка сообщить об ошибке, которая прикладывает снимок состояния, даёт больше дефектов, чем любое количество часов ручной проверки. Никакая проверка не гарантирует отсутствия ошибок, а поток от пользователей гарантирует, что о самых заметных ты узнаешь.

Формулировка практика: даже если ты сидел и перебирал двадцать часов, это не гарантирует отсутствия ошибок; хочешь бесплатный поток обратной связи, добавь кнопку.

Возражение, которое там же прозвучало: выпускать сырое неуважительно к пользователю. Правы оба, и граница проходит по тому, кто твой пользователь и сколько стоит его время.

Синхронизация дизайна и кода

Короткий, но полезный сюжет.

Проблема известна с незапамятных времён: макет и код разъезжаются. С агентами она обострилась, потому что скорость изменений выросла в разы.

Что работает: единый источник истины для оформления. Не картинка и не описание, а набор значений, из которого берут и макет, и код. Тогда расхождение становится технически невозможным, а не запрещённым.

Частный случай общего принципа из главы про детерминированные проверки: то, что должно совпадать всегда, должно совпадать по построению, а не по договорённости.

Что запомнить

Офисные форматы через распознавание не гоняй. Структура там уже есть, задача её не проебать.

Проверяй разбор на самом сложном документе: развалившаяся таблица это твой потолок качества.

Куски для поиска режь моделью, а не по числу символов.

Дефекты интерфейса из кода не видны. Смотри глазами, на разных ширинах, с предельными значениями.

Дизайн начинается с референсов и системы, а не со скилла.

Умение видеть шаблонный результат это навык. Нет его, покажи тому, у кого есть.

И то, что должно совпадать всегда, пусть совпадает по построению.

Заключение. Что устоялось, а что нет

Книга собрана из полугода живого разговора. За это время сменилось несколько поколений моделей, дважды перевернулись представления о том, сколько нужно инструкций, и ни разу, блядь, не закрылся спор о том, что вообще называть агентом.

Полезно на прощание разделить материал на три круга по сроку годности.

Ядро: то, что не изменится

Эти утверждения держались весь период и от конкретных инструментов не зависят.

Контекст это ресурс. Он платный, ограниченный и неравномерный. Всё, что ты делаешь с обвязкой, ты делаешь с контекстом.

Проверка обязательна и она твоя. Модель дадут, цикл дадут, инструменты дадут. А способ узнать, годится ли результат, не даст ни одна сука.

Гарантии живут в коде, а не в тексте. На правило можно положить хуй, на хук нельзя. Формат, описанный словами, нарушается в одном случае из десяти.

Нельзя быть себе судьёй. Предвзятость создаётся контекстом, поэтому ревью в чистой сессии находит то, чего не видит автор.

Один прогон это не измерение. Агент стохастичен, и единичный результат это точка из распределения.

Делегирование это решение по дизайну. Не способ сэкономить контекст. Издержки координации есть всегда.

Запрет без предоставленного пути рождает изворотливость. Агент ищет дорогу к цели, а не границу морали.

ТРИ КРУГА ПО СРОКУ ГОДНОСТИ



Средний слой: то, что меняется за год

Здесь консенсуса нет, и позиции ползут вместе с моделями.

Сколько нужно обвязки. За наблюдаемый период маятник качнулся от чем больше инструкций, тем лучше до снесите восемьдесят процентов и не заметите разницы. Куда он качнётся дальше, хуй знает.

Где предел автономии. Сегодня это порядка десяти-пятнадцати часов при наличии критериев на каждом шаге. Год назад это были полтора часа.

Нужен ли тяжёлый процесс. Строгий конвейер и панк-кодинг оба работают у практиков, и выбор определяется ценой ошибки, а не тем, кто кого перепиздел в споре.

Что делать с многоагентностью. Устоялось только разграничение: сбор делится, выбор нет. Всё остальное открыто.

Внешний слой: то, что устареет к выходу книги

Названия моделей. Цены. Лимиты. Уровни усилий и их поведение. Конкретные инструменты и их возможности. Кто сегодня лучше на каком классе задач.

Именно поэтому в книге почти нет имён и чисел этого слоя. Их место в закладках браузера, а не в тексте, который проживёт дольше квартала.

Споры, которые остались открытыми

Честности ради перечислим то, что за полгода так и не разрешилось.

Что такое харнесс. Три несовместимых определения, и все в ходу.

Нужно ли понимать уровень ниже. Аргументы обеих сторон сильные, эксперимент невозможен.

Сколько полировать. Опыт к правилу не свёлся.

Откуда брать новых людей. Опыт различения рабочего и кажущегося рабочим накапливался в условиях, которых больше нет.

Выдержит ли экономика. Нынешние цены держатся на инвестициях, и что будет дальше, не знает ни одна живая душа.

ГЛАВНОЕ НА ПРОЩАНИЕ

Область не устоялась, и книга это не свод законов.

Здесь собрано то, что работало у многих на настоящих задачах. Проверь на своей. И если что-то из написанного у тебя не работало, скорее всего прав ты, а не книга.

Что делать дальше

Три вещи, в этом порядке.

Собери свой набор проверок. Десять типовых задач с проверяемым критерием успеха. Гоняй их после каждого изменения. Это единственное, что переживёт смену моделей, инструментов и настроений.

Собери минимальный обвес. Не тот, который ты подсмотрел у других, а тот, который закрывает места, где спотыкаешься именно ты. Начни с трёх строк и добавляй только после второго повторения проблемы. Всё остальное это украшательство и трата контекста.

Заведи привычку измерять. Не спорить о приёмах, а сравнивать. Разброс между прогонами говорит о твоей системе больше, чем любая глава.

Последнее

Самое ценное наблюдение из всего корпуса не техническое.

Люди, у которых получалось охуенно, отличались не набором инструментов и не выбором модели. Они отличались тем, что задавали себе вопрос чем я проверю раньше, чем вопрос как я это сделаю.

Вот и весь секрет, ради которого написаны предыдущие двести страниц. Дальше уже сам.

Словарь терминов

Термины расположены по алфавиту. Пометка в третьей колонке говорит, устоялось значение или у собеседников оно может отличаться.

Термин	Значение	Статус
Агент	Посредник, который сам собирает контекст и сам выбирает инструменты	Спорный
Архетип	Короткое указание на узнаваемый образ вместо описания характера	Устоялся
Веер	Топология, где один агент раздаёт задачи, а потом собирает результаты	Устоялся
Дрейф	Постепенный уход агента от исходного намерения	Устоялся
Идемпотентность	Свойство операции не менять результат при повторе	Устоялся
Инструмент	Действие во внешнем мире, доступное агенту, с описанным контрактом	Устоялся
Инъекция задачи	Инструкция, попавшая к агенту из прочитанного содержимого	Устоялся
Канареечный выпуск	Выкатка изменения на малую долю, чтобы поймать проблему рано	Устоялся
Контекст	Весь текст, уходящий модели на текущем ходу	Устоялся
Контекстный движок	То, что решает, из каких источников и что положить в контекст	Спорный
Круг, ход	Один обмен с моделью: отправили контекст, получили ответ	Устоялся
Линтер	Автоматическая проверка кода на соблюдение правил	Устоялся
Метагейминг	Оптимизация агентом проверки вместо задачи	Устоялся
Мета-обязка	Отдельная обязанка для измерения и сборки основной	Новый
Нить памяти	Хронология решений и обоснований по одной теме	Новый
Обязка, харнесс	Всё вокруг модели: инструкции, инструменты, память, проверки	Спорный
Оркестратор	Агент, распределяющий работу между другими агентами	Устоялся
Отложенная выборка	Задачи, отложенные для финальной проверки и не используемые при настройке	Устоялся
Песочница	Изолированная среда, где агент запускает код	Устоялся
Поведенческое описание	Формулировка требования через ситуацию, действие и результат	Устоялся
Поиск с подстановкой	Схема, где найденные куски документов подставляются в запрос	Устоялся
Порог близости	Число, ниже которого найденное считается нерелевантным	Устоялся

Термин	Значение	Статус
Постепенное раскрытие	Подгрузка знаний по требованию вместо постоянного присутствия	Устоялся
Проход схлопывания	Итерация, убирающая выросшее за время полировки	Новый
Проверка завершения	Механизм, решающий, достигнута ли цель автономного режима	Устоялся
Провенанс	Сведения о том, откуда взялся кусок контекста	Устоялся
Расширение запроса	Переформулировка вопроса в несколько вариантов перед поиском	Устоялся
Ревью в чистой сессии	Проверка результата агентом, не знающим истории его создания	Устоялся
Релизный шлюз	Условие, без выполнения которого изменение не выходит	Устоялся
Рой	Много агентов, работающих параллельно над одной большой задачей	Устоялся
Сжатие, компакция	Замена накопленной истории кратким пересказом	Устоялся
Скилл	Отдельный файл с инструкцией, подгружаемый по решению модели	Устоялся
Смещение судьи	Систематическое предпочтение модели-судьи, не связанное с качеством	Устоялся
Спектр агентности	Шкала от одиночного вызова до автономной системы	Новый
Субагент	Агент, запущенный другим агентом с чистым контекстом	Устоялся
Схлопывание контекста	То же, что сжатие	Устоялся
Токен	Кусок текста, единица счёта и оплаты	Устоялся
Траектория	Последовательность действий агента при решении задачи	Устоялся
Триггер на нарушение ожидания	Срабатывание при расхождении наблюдения с моделью нормы	Новый
Уровень усилий	Бюджет модели на обдумывание, а не мера её ума	Устоялся
Файл правил	Текст, подкладываемый в контекст на каждом ходу	Устоялся
Хук	Код, выполняемый системой по событию, независимо от решения модели	Устоялся
Цикл	Повторяющиеся обращения к модели до достижения результата	Устоялся
Чанк	Кусок документа, попадающий в поисковый индекс	Устоялся
Чекпоинт	Проверяемое состояние на промежуточном шаге плана	Устоялся
Шаблонный результат	Усреднённый вывод модели, узнаваемый по отсутствию своего	Устоялся
Эвал	Набор задач с известным результатом для измерения системы	Устоялся

Чек-листы

Семь списков, к которым удобно возвращаться. Каждый занимает меньше страницы намеренно. Тут по делу и без лирики: списками пользуются в работе, а не читают на ночь. Пройдёшь по ним честно, и половина пиздецов не случится вообще.

1. Перед длинной сессией

ПОДГОТОВКА

1. Спецификация на месте, критерии готовности прописаны для каждого пункта.
2. Файл прогресса создан и агент знает, что писать туда.
3. Порог сжатия выставлен осознанно, а не по умолчанию.
4. Хук на восстановление после сжатия работает: проверено, а не предполагается.
5. Стабильная часть контекста в начале, изменчивая в конце.
6. Потолок расхода на задачу задан.
7. В запросе есть строка про работу по документации и ссылки на источник.

2. Перед ночным прогоном

АВТОНОМНЫЙ ЗАПУСК

1. План разбит на пункты, у каждого критерий готовности.
2. Сказано явно: человека нет за компьютером.
3. Сказано, что делать при блокировке: пометить, записать причину, пропустить.
4. Сказано перечитывать спецификацию перед каждым пунктом.
5. Задан порядок обхода пунктов.
6. Резервная копия сделана и лежит там, куда агент не дотянется.
7. Права ограничены: необратимое недоступно технически.
8. Утром сначала проверка состояния в чистой сессии, потом чтение отчёта.

3. При написании скилла

НОВЫЙ СКИЛЛ

1. Это повторяется? Посчитай, сколько раз в месяц.
2. Модель справляется без него? Проверь прогоном с ним и без.
3. Описание перечисляет ситуации применения.
4. Описание явно называет ситуации, где скилл не нужен.
5. Внутри то, что не меняется еженедельно.
6. Есть базовая линия для замера: та же задача без скилла.
7. Есть предполётная проверка версии или скилл лежит в хранилище с такой проверкой.
8. Записано, кто владелец, если скилл общий.

4. При добавлении правила**НОВАЯ СТРОКА В ПРАВИЛАХ**

1. Агент не выяснит это сам, открыв два-три файла.
2. Проверено экспериментом: с этой строкой и без неё результат отличается.
3. Есть причина, а не только запрет.
4. Названа область применения, если требование стилевое.
5. Нет противоречия с другими правилами и описаниями инструментов.
6. Это не то, чему место в описании инструмента, скилле, памяти или хуке.
7. Файл правил всё ещё умещается в разумный объём, и конец файла доезжает.

5. При разборе инцидента**ПОСЛЕ ТОГО, КАК ЧТО-ТО СЛОМАЛОСЬ**

1. Сначала восстановление состояния, потом разбирательство.
2. Смотри журналы и историю команд, а не объяснение агента.
3. Вопрос не почему он это сделал, а почему он смог.
4. Найди, чего не хватало в правах, а не в правилах.
5. Заведи проверку на этот класс: тот же инцидент дважды это отсутствие проверки.
6. Проверь, восстанавливается ли состояние из резервной копии на самом деле.
7. Запиши случай туда, где его прочитает следующий человек.

6. При выборе модели и уровня усилий

ПЕРЕД ЗАПУСКОМ ЗАДАЧИ

1. Что ты знаешь о задаче: как делать, что делать или в чём вообще задача?
2. Цена ошибки: минуты, часы или дни на исправление?
3. Уровень усилий подобран под размер задачи, а не под её важность.
4. Проверено, не наследуют ли субагенты уровень усилий родителя.
5. Для навигации по файлам используется дешёвая модель.
6. Задача повторяется, и разница между вариантами замерена хотя бы раз?

7. Независимость от поставщика**ПРОВЕРКА НА УСТОЙЧИВОСТЬ**

1. Сколько мест жёстко завязаны на имя конкретной модели?
2. Что перестанет работать, если поставщик станет недоступен завтра?
3. Есть ли вторая настроенная и проверенная связка, а не намерение её когда-нибудь сделать?
4. Правила и скиллы написаны под задачу или под конкретную модель?
5. Эвалы хоть иногда прогоняются на второй модели?
6. Память и данные лежат в формате, переживающем смену инструмента?
7. Есть ли места, завязанные на отпечаток конкретной машины?

Шаблоны запросов

Восемь готовых текстов. Их можно копировать как есть и править под себя.

1. Вопросы пачками

Не начинай реализацию.

Задай мне вопросы, ответы на которые нужны, чтобы задачу можно было выполнить однозначно. По пять вопросов за раз, от самых важных к менее важным.

К каждому вопросу добавь одну строку: почему без ответа на него ты можешь сделать не то.

Когда я отвечу, задай следующую пачку. Остановись, когда оставшиеся вопросы перестанут влиять на результат.

2. Поиск слепых зон в плане

Прочитай план ещё раз и найди места, где он может не сработать.

Отдельно перечисли:

- чего в плане нет вообще;
- какие предположения сделаны молча;
- что произойдёт при ошибке на каждом шаге;
- какие места я объяснил неоднозначно.

Ничего не исправляй. Только перечисли.

3. Автономный прогон

Работай по плану из файла <путь>.

Меня нет за компьютером. Ничего не спрашивай, решения принимай сам, но записывай их.

Веди прогресс прямо в плане: отмечай выполненное, помечай заблокированное и пропускай такие пункты, а не останавливайся.

Иди в порядке, заданном планом, до конца списка.

Перед каждым пунктом перечитывай спецификацию и файл решений, независимо от того, помнишь ты их или нет.

Если пункт не закрывается по критериям после трёх попыток, помечай как заблокированный, записывай причину и переходи дальше.

В конце оставь отчёт: что сделано, что заблокировано и почему,

что осталось.

4. Ревью в чистой сессии

Перед тобой результат работы, историю создания которого ты не видел. Это важно: не додумывай замысел, суди по тому, что есть.

1. Перечисли пункты спецификации, критерии которых сейчас не выполняются.
2. Перечисли места, где реализация расходится со спецификацией.
3. Перечисли решения, принятые без явного указания.
4. На каждый из этих вопросов ответь да или нет:
<ваш список из пяти-десяти конкретных вопросов>

Общих оценок качества не давай.

5. Обучающая сессия против когнитивного долга

Ты внимательный преподаватель. Задача: чтобы я действительно разобрался в том, что здесь сделано.

Веди отдельный документ с чек-листом того, что я должен понять. Двигайся по одному пункту, не сваливай всё в конец.

Проверь, что я разобрался в трёх вещах:

- 1) проблема: почему она возникла и какие были развилки;
- 2) решение: почему выбрали именно его, какие краевые случаи;
- 3) контекст: почему это важно и на что повлияет.

Сначала попроси меня пересказать своими словами, как я всё понял, чтобы понять мой уровень. Дальше закрывай пробелы.

Гоняй меня вопросами. Не показывай ответ, пока я не отвечу. Сессия не заканчивается, пока я не покажу понимание всего списка.

6. Перечисление всех вариантов, включая отброшенные

Прежде чем предлагать решение, перечисли все варианты, которые приходят в голову, включая странные и те, которые ты обычно отбрасываешь как несерьёзные.

К каждому добавь строку: почему ты обычно его отбрасываешь.

Только после этого предложи, что выбрать, и объясни выбор.

7. Предполётная проверка версии скилла

Раз в сессию сверь этот файл с источником:

```
curl -fsSL --max-time 3 <адрес файла в хранилище> \  
| diff -q - "<путь к этому файлу>"
```

Разошлись: скажи одной строкой, что вышло обновление и как обновиться. Совпали: молчи и работай дальше.

8. Описание скилла, которое срабатывает

description: Использовать при <перечень ситуаций через запятую: что делаем, над чем, в каком контексте>. Не использовать для <перечень похожих ситуаций, где скилл не нужен>.

Пример заполнения:

description: Использовать при проектировании, реализации, ревью, отладке или оценке агента: инженерия контекста, интерфейсы инструментов, надёжность обвязки, память, оценка, координация нескольких агентов. Не использовать для обычного неагентного кода приложения.

9. Строка, которую стоит добавить почти в любой запрос

Работаем по документации и фактам, а не по памяти. Перед любым выводом опираемся и ссылаемся на источник.